

LAPORAN KERJA PRAKTEK
PENGEMBANGAN BACKEND SISTEM INFORMASI MANAJEMEN
PRODUK BARU UNTUK MENDUKUNG PENGELOLAAN DATA
DAN MONITORING PRODUK BERBASIS WEB PADA PT
PETROKIMIA GRESIK



Oleh:

Henokh Yeremia Olbrain Perangin Angin

1462300075

PROGRAM SARJANA
PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS 17 AGUSTUS 1945 SURABAYA

2026

LEMBAR PENGESAHAN

LAPORAN KERJA PRAKTEK

**PENGEMBANGAN BACKEND SISTEM INFORMASI MANAJEMEN PRODUK
BARU UNTUK MENDUKUNG PENGELOLAAN DATA DAN MONITORING
PRODUK BERBASIS WEB PADA PT PETROKIMIA GRESIK**

Sebagai salah satu syarat untuk melaksanakan Kerja Praktek

Oleh :

Henokh Yeremia Olbrain Perangin Angin

1462300075



Surabaya, 25 Juni 2026

Koordinator KP,

Dosen Pembimbing



Anton Breva Yunanda S.T., M.MT

Ahmad Habib, S.Kom., M.M., M.Kom

NPP. 20450020554

NPP. 20460150665

Mengetahui,

Ka, Program Studi Teknik Informatika

Puteri Noraisya Primandari S.ST., M.IM.

NPP. 20460170736

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmat-Nya sehingga kegiatan magang dan penyusunan laporan ini dapat diselesaikan dengan baik.

Laporan Kerja Praktek ini disusun sebagai salah satu syarat dalam menyelesaikan program studi Teknik Informatika di [Universitas]. Laporan ini membahas kegiatan magang yang dilaksanakan di PT Petrokimia Gresik, Departemen Manajemen Produk Baru, dengan fokus pada pengembangan backend Sistem Informasi Manajemen Produk Baru (Satu Peta Pasar).

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada:

1. **Ahmad Habib, S.Kom., M.M., M.Kom** selaku dosen pembimbing yang telah memberikan arahan dan bimbingan dalam penyusunan laporan ini.
2. **Erwin Indra AVP Product Management** selaku pembimbing lapangan di PT Petrokimia Gresik yang telah memberikan bimbingan selama kegiatan magang.
3. Pimpinan dan staf Departemen Manajemen Produk Baru PT Petrokimia Gresik yang telah memberikan kesempatan dan dukungan selama kegiatan magang.
4. Kedua orang tua dan keluarga yang selalu memberikan doa dan dukungan.
5. Teman-teman yang telah memberikan semangat dan bantuan selama kegiatan magang.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, saran dan kritik yang membangun sangat diharapkan untuk perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi semua pihak yang berkepentingan.

Surabaya, 25 Juni 2026



Henokh Yerima Olbrain Perangin Angin

DAFTAR ISI

KATA PENGANTAR.....	3
BAB 1	10
PENDAHULUAN	10
1.1 Latar Belakang.....	10
1.2 Identifikasi Masalah	11
1.3 Rumusan Masalah	11
1.4 Tujuan Magang.....	12
1.5 Manfaat.....	12
1.6 Batasan Masalah	13
1.7 Metode Pengumpulan Data	13
1.8 Sistematika Penulisan.....	14
BAB II	16
GAMBARAN UMUM	16
2.1 Profil Perusahaan.....	16
2.1.1 Sejarah dan Bidang Usaha.....	16
2.1.2 Departemen Manajemen Produk Baru	16
2.2 Gambaran Umum Sistem	16
2.2.1 Nama dan Tujuan Sistem.....	16
2.2.2 Pengguna Sistem	17
2.2.3 Modul Utama Sistem.....	17
2.2.4 Status Implementasi.....	18
2.3 Proses Bisnis Sistem.....	20
2.3.1 Alur Pengelolaan Data Master.....	20
2.3.2 Alur Pencatatan Potensi dan Penjualan	20
2.3.3 Alur Pendataan Kios dan Assignment Produk	20

2.3.4 Alur Visualisasi Peta	20
2.4 Landasan Teori	20
2.4.1 React dan TanStack	20
2.4.2 TypeScript	21
2.4.3 oRPC dan Zod	21
2.4.4 PostgreSQL dan Drizzle ORM	21
2.4.5 Better Auth	22
2.4.6 Leaflet dan GeoJSON	22
2.4.7 Build dan Deployment	22
2.4.8 Arsitektur Full-Stack Monolith	23
3.1 Persiapan Lingkungan dan Tools	25
3.1.1 Prasyarat Sistem	25
3.1.2 Instalasi Dependency	25
3.1.3 Konfigurasi Environment	25
3.1.4 Menjalankan Database	26
3.1.5 Menjalankan Development Server	26
3.1.6 Perintah Penting	26
3.2 Kegiatan Survei Lapangan dan Observasi Wilayah	27
3.2.1 Tujuan Kegiatan Lapangan	27
3.2.2 Lokasi yang Dikunjungi	27
3.2.3 Aktivitas Observasi	28
3.2.4 Hubungan dengan Kebutuhan Data Sistem	28
3.2.5 Manfaat Observasi Lapangan	28
3.3 Arsitektur Sistem	31
3.3.1 Arsitektur Full-Stack Monolith	31
3.3.2 Lapisan Aplikasi	31
3.3.3 Alur Request	31

3.3.4 Perbedaan dengan Deskripsi Template.....	32
3.4 Struktur Proyek dan Pembagian Layer.....	33
3.4.1 Struktur Root	33
3.4.2 Struktur Aplikasi (`apps/web`)	34
3.4.3 Policy Dependency	34
3.5 Perancangan Database	37
3.5.1 Gambaran Umum	37
3.5.2 Domain Autentikasi.....	37
3.5.3 Domain Wilayah dan Lahan	37
3.5.4 Domain Komoditas.....	38
3.5.5 Domain Produk dan Potensi	38
3.5.6 Domain Kios.....	39
3.5.7 Domain Penjualan	39
3.5.8 Domain Demo	39
3.5.9 Catatan Integritas Database	39
3.6 Perancangan API	41
3.6.1 oRPC Router.....	41
3.6.2 Public dan Protected Procedure.....	41
3.6.3 Context	42
3.6.4 Struktur Endpoint per Modul.....	42
3.6.5 Validasi Input	43
3.7 Authentication dan Authorization	45
3.7.1 Konfigurasi Better Auth	45
3.7.2 Route Guard.....	46
3.7.3 Protected Procedure.....	46
3.7.4 Role Pengguna.....	46
3.7.5 Gap Authorization	46

3.8 Hasil Implementasi Modul Bisnis	51
3.8.1 Modul Wilayah (Province dan Regency)	51
3.8.2 Modul Lahan (Land Type, Province Land, Regency Land).....	54
3.8.3 Modul Komoditas (Commodity Type, Province Commodity, Regency Commodity).....	55
3.8.4 Modul Produk (Product Type, Product Brand, Product Dosage)	58
3.8.5 Modul Potensi (Province Potential, Regency Potential)	61
3.8.6 Modul Penjualan (Sales Realization, Daily Sales)	63
3.8.7 Modul Stall	67
3.8.8 User Management.....	71
3.8.9 Dashboard Admin.....	71
3.8.10 Visualisasi Peta.....	72
3.8.11 Landing Page Publik.....	74
3.9 Pengujian Sistem	74
3.9.1 Pengujian API.....	74
3.9.2 Pengujian Authentication	75
3.9.3 Quality Check	75
3.9.4 Production Build.....	75
3.9.5 Keterbatasan Pengujian	75
3.10 Build, Testing, dan Deployment.....	77
3.10.1 Build	77
3.10.2 Linting dan Formatting.....	77
3.10.3 Deployment	77
3.10.4 Environment Variable	78
3.11 Analisis dan Temuan	79
3.11.1 Status Modul.....	79
3.11.2 Gap Authorization	79

3.11.3 Mismatch Schema-Migration	79
3.11.4 Missing Foreign Key	79
3.11.5 Unique Constraint.....	79
3.11.6 Coupling Modul Stall	80
3.11.7 Standardisasi.....	80
3.11.8 Rekomendasi	80
3.12 Kontribusi Mahasiswa	80
3.12.1 Bukti Repository.....	80
3.12.2 Commit Modul Stall	81
3.12.3 Commit Konfigurasi Deployment	81
3.12.4 Kegiatan Analisis dan Dokumentasi	81
3.12.5 Batasan Kontribusi	81
4.1 Kesimpulan.....	83
4.1.1 Kesimpulan Arsitektur Sistem.....	83
4.1.2 Kesimpulan Database	83
4.1.3 Kesimpulan Modul Bisnis	84
4.1.4 Kesimpulan Kontribusi.....	84
4.2 Keterbatasan	84
4.3 Saran Pengembangan.....	85
4.3.1 Saran Jangka Pendek	85
4.3.2 Saran Jangka Menengah	85
4.3.3 Saran Jangka Panjang	85
4.3.4 Saran untuk Kegiatan Magang	86
4.4 Penutup	86
DAFTAR PUSTAKA.....	87
LAMPIRAN	88
Lampiran A — Logbook Kegiatan.....	88

Lampiran B — Screenshot Tambahan.....	89
Lampiran C — Source Code Representatif	89
C.1 Router oRPC Utama	89
C.2 Schema Database	90
C.3 Route Guard Admin.....	90
C.4 Better Auth Configuration	90
C.5 Stall Module — Assignment Product Brand	91
Lampiran D — Surat Keterangan Magang.....	91
Lampiran E — Daftar Lengkap Endpoint API.....	91
Public Endpoints.....	91
Protected Endpoints (Admin)	91
Lampiran F — Daftar Lengkap Tabel Database	95
F.1 Domain Autentikasi.....	95
F.2 Domain Wilayah	95
F.3 Domain Lahan	96
F.4 Domain Komoditas	96
F.5 Domain Produk dan Potensi	97
F.6 Domain Kios	98
F.7 Domain Penjualan	99
F.8 Domain Demo	99

BAB 1

PENDAHULUAN

1.1 Latar Belakang

PT Petrokimia Gresik merupakan perusahaan pupuk terbesar di Indonesia yang memiliki peran strategis dalam mendukung ketahanan pangan nasional. Dalam menjalankan fungsi pengembangan produk, Departemen Manajemen Produk Baru bertanggung jawab mengelola data pasar yang mencakup informasi wilayah, lahan pertanian, komoditas unggulan, produk pupuk, potensi pasar, jaringan kios, dan data penjualan. Pengelolaan data ini menjadi penting untuk mendukung analisis pasar, perencanaan strategis, dan monitoring produk secara tepat dan akurat.

Sebelum sistem ini dikembangkan, data pasar yang dikelola Departemen Manajemen Produk Baru masih tersebar dalam berbagai format dan belum terintegrasi dalam satu platform terpusat. Analisis geografis terhadap data pasar sulit dilakukan karena data disajikan dalam bentuk tabel yang tidak dilengkapi visualisasi spasial. Selain itu, konsistensi data sulit dijaga karena belum ada mekanisme validasi dan kontrol akses yang terstandarisasi. Kondisi ini mendorong perlunya sebuah aplikasi berbasis web yang mampu menyatukan pengelolaan data pasar sekaligus menyediakan visualisasi geografis untuk membaca persebaran dan potensi pasar.

Sistem Informasi Manajemen Produk Baru hadir sebagai solusi untuk menjawab kebutuhan tersebut. Aplikasi ini diberi nama Satu Peta Pasar (PASAR — Platform Analisis Strategi dan Realisasi Pasar), yang bertujuan memetakan dan mengelola data pasar secara terpusat. Lingkup data yang dikelola meliputi provinsi, kabupaten, jenis lahan, komoditas, produk, potensi pasar, kios, dan penjualan. Sistem menyediakan peta interaktif untuk visualisasi data spasial serta panel administrasi untuk pengelolaan data master.

Dalam pengembangan aplikasi, backend memegang peranan krusial dalam menyediakan API yang konsisten, validasi data yang ketat, mekanisme authentication dan authorization, serta integrasi dengan database. Backend juga menjadi jembatan antara antarmuka pengguna dan data yang tersimpan, sehingga keandalan, keamanan, dan kemudahan pemeliharaan backend menjadi faktor penentu keberhasilan sistem secara keseluruhan.

Kegiatan magang ini menjadi sarana penerapan pengetahuan Teknik Informatika di lingkungan industri. Melalui kegiatan ini, mahasiswa dapat mempelajari arsitektur full-stack, perancangan database, implementasi API, mekanisme keamanan, dan proses deployment pada proyek nyata. Selain itu, kegiatan ini juga memberikan kesempatan untuk menuangkan hasil analisis dalam bentuk dokumentasi teknis yang bermanfaat bagi perusahaan.

1.2 Identifikasi Masalah

Berdasarkan latar belakang yang telah diuraikan, beberapa permasalahan yang teridentifikasi adalah sebagai berikut:

1. Data pasar yang dikelola Departemen Manajemen Produk Baru masih tersebar dan belum terintegrasi dalam satu sistem yang terpusat, sehingga menyulitkan proses pencarian, pembaruan, dan pelaporan data.
2. Analisis geografis data pasar sulit dilakukan dalam bentuk tabel, padahal informasi spasial seperti persebaran potensi pasar dan lokasi kios sangat penting dalam pengambilan keputusan bisnis.
3. Diperlukan pengelolaan data yang konsisten dengan validasi input dan kontrol akses untuk menjaga kualitas dan keamanan data.
4. Dokumentasi arsitektur backend, database, dan API diperlukan sebagai landasan pengembangan dan pemeliharaan sistem di masa mendatang.
5. Status kesiapan setiap modul bisnis perlu dipetakan untuk mengetahui area yang masih dapat dikembangkan atau disempurnakan.

1.3 Rumusan Masalah

Berdasarkan identifikasi masalah di atas, rumusan masalah yang menjadi fokus kegiatan magang ini adalah sebagai berikut:

1. Bagaimana struktur arsitektur backend dan alur data pada sistem aplikasi Satu Peta Pasar?
2. Bagaimana perancangan database untuk mendukung pengelolaan data pasar pada sistem?
3. Bagaimana implementasi authentication dan authorization pada sistem?
4. Bagaimana implementasi API untuk setiap modul bisnis pada sistem?

5. Bagaimana tingkat kesiapan setiap modul bisnis dalam sistem?
6. Apa temuan dan rekomendasi untuk pengembangan sistem selanjutnya?

1.4 Tujuan Magang

1. Tujuan yang ingin dicapai melalui kegiatan magang ini adalah sebagai berikut:
Memahami arsitektur full-stack dan alur data dari antarmuka pengguna hingga database pada aplikasi Satu Peta Pasar.
2. Mempelajari perancangan database untuk domain data pasar menggunakan PostgreSQL dan Drizzle ORM.
3. Menganalisis dan mendokumentasikan struktur backend, API, authentication, dan setiap modul bisnis pada sistem.
4. Menyusun dokumentasi teknis arsitektur, database, API, dan modul sebagai referensi pengembangan dan pemeliharaan sistem.
5. Mengimplementasikan pengembangan atau pelengkapan modul sesuai ruang lingkup yang ditetapkan, khususnya pada modul Stall dan assignment Product Brand.

1.5 Manfaat

Kegiatan magang ini diharapkan memberikan manfaat bagi berbagai pihak sebagai berikut:

Bagi mahasiswa:

- Menerapkan pengetahuan arsitektur full-stack, perancangan database, implementasi API, dan mekanisme authentication dalam proyek nyata di lingkungan industri.
- Memperoleh pengalaman dalam menganalisis source code, mendokumentasikan sistem, dan menyusun rekomendasi pengembangan.

Bagi perusahaan:

- Memperoleh dokumentasi teknis backend yang mencakup arsitektur, database, API, authentication, dan status modul sebagai referensi pengembangan dan pemeliharaan sistem.
- Mendapatkan identifikasi area pengembangan potensial dan rekomendasi perbaikan kualitas sistem.

Bagi akademik:

- Menyediakan bahan studi kasus pengembangan backend berbasis TypeScript, oRPC, dan Drizzle ORM.

- Memberikan gambaran penerapan praktik pengembangan perangkat lunak di lingkungan industri pupuk dan pertanian.

1.6 Batasan Masalah

Agar pembahasan lebih terfokus, kegiatan magang ini dibatasi pada hal-hal berikut:

1. Analisis dilakukan terhadap source code, dokumentasi, dan riwayat Git repository pada branch yang diperiksa.
2. Fokus utama adalah backend yang mencakup database, API oRPC, authentication, dan modul bisnis.
3. Pembahasan frontend dibatasi pada aspek yang berkaitan langsung dengan alur data backend, seperti pemanggilan API dan mekanisme route guard.
4. Modul yang dibahas meliputi wilayah (province dan regency), lahan (land type, province land, regency land), komoditas (commodity type, province commodity, regency commodity), produk (product type, product brand, product dosage), potensi (province potential, regency potential), penjualan (sales realization, daily sales), kios (stall), user management, peta interaktif, dan authentication.
5. Klaim kontribusi implementasi mahasiswa hanya digunakan untuk modul yang memiliki bukti perubahan pada riwayat repository, yaitu modul Stall dan assignment Product Brand. Modul lainnya diposisikan sebagai objek analisis dan dokumentasi.
6. Tidak semua fitur pada setiap modul memiliki tingkat kesiapan yang sama; beberapa modul masih bersifat read-only atau memiliki endpoint yang belum aktif.

1.7 Metode Pengumpulan Data

Metode yang digunakan dalam pengumpulan data untuk penyusunan laporan ini adalah sebagai berikut:

Studi dokumentasi:

Mempelajari dokumen proyek yang tersedia, seperti README, GUIDEBOOK,

BACKEND.md, arch.md, dan dokumentasi lainnya yang menjelaskan arsitektur, konfigurasi, dan penggunaan sistem.

Observasi source code:

Menelusuri struktur route, procedure oRPC, schema Drizzle ORM, konfigurasi aplikasi, dan mekanisme authentication pada source code untuk memahami implementasi teknis setiap modul.

Pemeriksaan database:

Menganalisis schema database, file migration, relasi tabel, foreign key, dan constraint untuk memahami struktur dan integritas data.

Analisis riwayat Git:

Memeriksa riwayat commit, author, tanggal perubahan, dan file yang berubah untuk mengidentifikasi kontribusi pengembangan dan melacak evolusi sistem.

1.8 Sistematika Penulisan

Laporan ini disusun dalam empat bab dengan sistematika sebagai berikut:

BAB I — Pendahuluan:

Berisi latar belakang, identifikasi masalah, rumusan masalah, tujuan magang, manfaat, batasan masalah, metode pengumpulan data, dan sistematika penulisan.

BAB II — Gambaran Umum:

Berisi profil perusahaan, gambaran umum sistem Satu Peta Pasar, proses bisnis sistem, dan landasan teori yang mendukung pembahasan laporan.

BAB III — Pelaksanaan Kerja Praktek:

Berisi persiapan lingkungan kerja, analisis sistem, perancangan database dan API, implementasi authentication, hasil implementasi setiap modul bisnis, pengujian sistem, build dan deployment, analisis temuan, serta kontribusi mahasiswa.

BAB IV — Kesimpulan dan Saran:

Berisi kesimpulan dari seluruh kegiatan magang, keterbatasan, serta saran pengembangan untuk perusahaan dan kegiatan magang selanjutnya.

BAB II

GAMBARAN UMUM

2.1 Profil Perusahaan

2.1.1 Sejarah dan Bidang Usaha

PT Petrokimia Gresik merupakan perusahaan pupuk terbesar dan terlengkap di Indonesia yang didirikan pada tanggal 10 Juli 1972. Perusahaan ini berlokasi di Gresik, Jawa Timur, dan bergerak di bidang produksi pupuk serta bahan kimia untuk sektor pertanian dan industri. Sebagai anggota holding perusahaan pupuk Indonesia (PT Pupuk Indonesia Holding Company), PT Petrokimia Gresik memiliki peran strategis dalam mendukung program ketahanan pangan nasional melalui penyediaan pupuk bersubsidi maupun non-subsidi.

Visi perusahaan adalah menjadi perusahaan kimia dan pupuk yang unggul di tingkat regional dan berkelanjutan. Misi perusahaan meliputi menyediakan produk dan jasa yang berkualitas, mengelola sumber daya secara efisien dan bertanggung jawab, serta memberikan kontribusi optimal bagi pemangku kepentingan.

2.1.2 Departemen Manajemen Produk Baru

Departemen Manajemen Produk Baru merupakan unit kerja di bawah PT Petrokimia Gresik yang bertanggung jawab dalam pengembangan produk pupuk baru, baik bersubsidi maupun non-subsidi. Departemen ini melakukan riset pasar, analisis potensi wilayah, pemetaan distribusi, dan monitoring realisasi penjualan produk. Dalam menjalankan fungsinya, departemen mengelola data pasar yang mencakup informasi wilayah pertanian, komoditas unggulan, produk pesaing, jaringan kios, dan data penjualan.

Struktur pimpinan departemen terdiri dari Project Manager (PM) dan Senior Manager Department I (SMD I).

2.2 Gambaran Umum Sistem

2.2.1 Nama dan Tujuan Sistem

Sistem yang menjadi objek kegiatan magang adalah **Satu Peta Pasar (PASAR — Platform Analisis Strategi dan Realisasi Pasar)**. Sistem ini merupakan aplikasi berbasis

web yang bertujuan untuk memetakan dan mengelola data pasar secara terpusat guna mendukung pengelolaan data dan monitoring produk di Departemen Manajemen Produk Baru PT Petrokimia Gresik.

Tujuan utama sistem adalah menyediakan platform terintegrasi yang mampu mengelola data wilayah, lahan, komoditas, produk, potensi pasar, kios, dan penjualan dalam satu kesatuan. Sistem dilengkapi peta interaktif untuk visualisasi geografis data pasar serta panel administrasi untuk pengelolaan data master.

2.2.2 Pengguna Sistem

Sistem memiliki empat kategori pengguna dengan hak akses yang berbeda:

Pengguna	Peran dan Akses
Viewer	Role yang terdefinisi pada kode, memiliki akses terbatas ke halaman tertentu. Tidak dapat mengakses panel admin. **[PERLU VERIFIKASI hak akses final]**
Guest	Role default bagi pengguna baru. Dapat mengakses halaman publik tetapi tidak dapat masuk ke panel admin.
Pengunjung umum	Mengakses halaman publik seperti beranda, peta, informasi potensi produk, realisasi penjualan, dan data kios tanpa perlu login.

2.2.3 Modul Utama Sistem

Sistem terdiri dari modul-modul utama sebagai berikut:

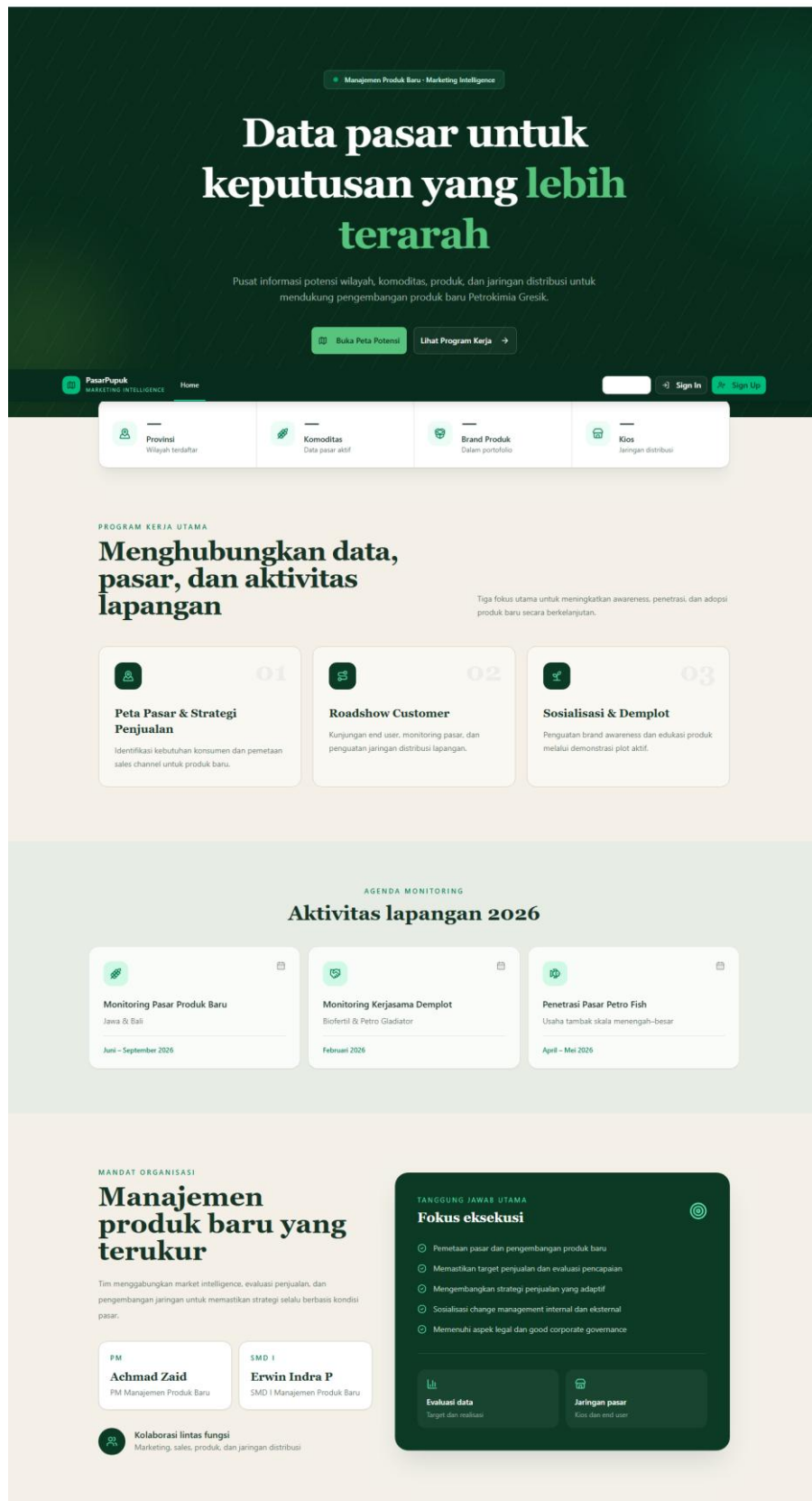
Modul	Fungsi
Authentication	Login dan registrasi berbasis email/password, manajemen session, dan route guard.
Marketing Map	Peta interaktif untuk visualisasi data potensi pasar, batas wilayah, dan persebaran kios.
Dashboard Admin	Ringkasan data berupa cards statistik, progress bar potensi, dan daftar produk terbaru.
Province Management	CRUD data provinsi (kode, nama, luas wilayah, tahun).
Regency Management	CRUD data kabupaten/kota yang terhubung dengan provinsi induk.
Land Management	CRUD jenis lahan dan data luas lahan per provinsi/kabupaten.
Commodity Management	CRUD jenis komoditas serta data komoditas per provinsi/kabupaten.

Product Management	CRUD jenis produk, brand produk, dan dosis produk.
Potential Management	Penyajian data potensi brand produk per provinsi/kabupaten.
Sales Management	CRUD realisasi penjualan dan penjualan harian.
Stall Management	CRUD data kios dan assignment brand produk ke kios.
User Management	Pengelolaan data pengguna dan peran (role).
Internationalization	Dukungan bahasa Inggris dan Indonesia melalui Lingui.

2.2.4 Status Implementasi

Tingkat kesiapan setiap modul belum sepenuhnya seragam. Beberapa modul telah memiliki operasi CRUD penuh, beberapa masih bersifat read-only, dan terdapat modul yang tabel databasenya telah tersedia tetapi endpoint aktifnya belum ditemukan.

Pembahasan lebih rinci mengenai status setiap modul disajikan pada BAB III laporan ini.



Gambar 2.1 Program kerja Departemen Manajemen Produk Baru

2.3 Proses Bisnis Sistem

2.3.1 Alur Pengelolaan Data Master

Proses bisnis pengelolaan data master dimulai dari pendataan wilayah sebagai entitas geografis tertinggi. Data provinsi dan kabupaten menjadi referensi bagi seluruh data operasional lainnya. Setelah wilayah terdefinisi, sistem mencatat data lahan (jenis lahan dan luasnya per wilayah) dan data komoditas (jenis komoditas unggulan per wilayah). Data produk kemudian dikelola dengan hierarki dari jenis produk, brand produk, hingga dosis produk untuk komoditas tertentu.

2.3.2 Alur Pencatatan Potensi dan Penjualan

Modul potensi menyimpan data potensi pasar suatu brand produk pada tingkat provinsi dan kabupaten. Data ini digunakan sebagai acuan dalam perencanaan strategis pengembangan produk. Sementara itu, modul penjualan mencatat realisasi penjualan dan data penjualan harian untuk memantau capaian dibandingkan dengan rencana (RKAP).

2.3.3 Alur Pendataan Kios dan Assignment Produk

Proses pendataan kios mencakup pencatatan informasi lokasi (provinsi, kabupaten, koordinat geografis), data pemilik, dan kontak. Setiap kios dapat di-assign dengan satu atau lebih brand produk melalui relasi many-to-many, sehingga sistem dapat mengetahui produk apa saja yang tersedia di setiap kios.

2.3.4 Alur Visualisasi Peta

Data potensi pasar divisualisasikan melalui peta interaktif berbasis Leaflet. Layer peta mencakup batas wilayah Indonesia dalam format GeoJSON, visualisasi choropleth untuk data potensi, serta marker untuk lokasi kios. Pengguna dapat memfilter data berdasarkan tahun, tingkat administrasi, dan jenis data yang ingin ditampilkan.

2.4 Landasan Teori

2.4.1 React dan TanStack

React merupakan library JavaScript untuk membangun antarmuka pengguna berbasis komponen. Dalam sistem Satu Peta Pasar, React digunakan bersama TanStack Start sebagai framework full-stack yang memungkinkan server-side rendering dan routing berbasis file. TanStack Router menangani navigasi client-side dengan dukungan loader,

context, dan route guard. TanStack Query digunakan untuk mengelola server state, caching, fetching, dan mutation data dari API.

2.4.2 TypeScript

TypeScript adalah superset JavaScript yang menambahkan sistem tipe statis. TypeScript digunakan di seluruh lapisan sistem untuk menjaga type safety antara frontend, API, dan database. Penggunaan TypeScript memungkinkan deteksi kesalahan pada tahap kompilasi, dokumentasi kode yang lebih baik, dan pengalaman pengembangan yang lebih produktif melalui autocompletion dan type checking.

2.4.3 oRPC dan Zod

oRPC merupakan library untuk membuat kontrak Remote Procedure Call (RPC) yang type-safe antara client dan server. Setiap procedure oRPC mendefinisikan input, output, dan handler secara eksplisit. Input procedure divalidasi menggunakan Zod, yaitu library validasi skema data untuk TypeScript. Zod memungkinkan pendefinisian skema validasi yang dapat digunakan bersama antara lapisan form frontend dan validasi API backend.

oRPC mendukung dua jenis procedure:

`publicProcedure` — dapat diakses tanpa session, digunakan untuk endpoint publik seperti health check dan data peta.

`protectedProcedure` — memerlukan session pengguna yang valid, digunakan untuk endpoint administratif.

2.4.4 PostgreSQL dan Drizzle ORM

PostgreSQL adalah sistem manajemen basis data relasional objek yang digunakan sebagai database utama sistem. Drizzle ORM adalah TypeScript ORM yang menyediakan query builder type-safe, definisi schema deklaratif, dan sistem migrasi database. Schema Drizzle mendefinisikan tabel, kolom, foreign key, dan constraint secara eksplisit dalam kode TypeScript.

Komponen utama Drizzle ORM yang digunakan meliputi:

- **Schema** — definisi tabel, relasi, dan constraint dalam kode TypeScript.

- **Migration** — file SQL yang dihasilkan dari perubahan schema untuk menjaga konsistensi database.
- **Drizzle Kit** — alat CLI untuk generate, migrate, dan push schema ke database.

2.4.5 Better Auth

Better Auth adalah library authentication untuk TypeScript yang menyediakan manajemen user, session, dan account. Sistem Satu Peta Pasar menggunakan Better Auth dengan adapter Drizzle ORM untuk menyimpan data autentikasi ke PostgreSQL. Metode autentikasi yang digunakan adalah email dan password dengan session yang dikelola melalui cookie.

Mekanisme keamanan mencakup:

- **Session-based authentication** — pengguna yang login mendapatkan session token yang disimpan dalam cookie.
- **Route guard** — halaman admin memeriksa session dan role pengguna sebelum memberikan akses.
- **Protected procedure** — API procedure memvalidasi session sebelum memproses request.

2.4.6 Leaflet dan GeoJSON

Leaflet merupakan library JavaScript sources terbuka untuk menampilkan peta interaktif berbasis web. React Leaflet menyediakan binding React untuk Leaflet sehingga peta dapat diintegrasikan sebagai komponen React. GeoJSON adalah format data geospasial berbasis JSON yang digunakan untuk merepresentasikan fitur geografis seperti batas wilayah dan titik lokasi.

Dalam sistem, GeoJSON digunakan untuk menyimpan data batas provinsi dan kabupaten Indonesia. Data ini disimpan dalam folder `public/data/` dan dimuat oleh Leaflet untuk ditampilkan sebagai layer peta. Data choropleth diperoleh dari API oRPC yang menyediakan data potensi pasar per wilayah.

2.4.7 Build dan Deployment

Proses build dan deployment sistem menggunakan perangkat sebagai berikut:

Teknologi	Fungsi
-----------	--------

Vite	Development server dan production bundling.
Turborepo	Menjalankan task monorepo secara paralel dengan caching.
Biome / Ultracite	Linting, formatting, dan pemeriksaan kualitas kode.
Vitest	Framework pengujian unit dan integrasi.
Docker	Kontainerisasi PostgreSQL dan aplikasi.
Netlify	Target deployment utama dengan TanStack Start.
Vercel	Alternatif platform deployment.
GitHub Actions	Workflow otomatisasi, termasuk ping Supabase keep-alive.

2.4.8 Arsitektur Full-Stack Monolith

Sistem Satu Peta Pasar menerapkan arsitektur full-stack monolith pada workspace `apps/web`. Berbeda dengan deskripsi template proyek yang membayangkan backend Hono terpisah pada folder `apps/server`, implementasi aktif menempatkan frontend dan backend dalam satu aplikasi TanStack Start. Server route di `apps/web/src/routes/api` mengekspos endpoint untuk oRPC, OpenAPI, dan Better Auth, sehingga seluruh lapisan aplikasi terintegrasi dalam satu kesatuan deployment.

```

1  {
2    "name": "web",
3    "private": true,
4    "type": "module",
5    "scripts": {
6      "dev": "bunx --bun vite dev",
7      "start": "bun run .output/server/index.mjs",
8      "build": "bunx --bun vite build",
9      "serve": "bunx --bun vite preview",
10     "test": "vitest run",
11     "format": "biome format",
12     "lint": "biome lint",
13     "check": "biome check",
14     "check-types": "tsc --noEmit",
15     "clean": "rm -rf .output .nitro .tanstack dist node_modules src/routeTree.gen.ts",
16     "lingui:extract": "lingui extract --clean",
17     "lingui:compile": "lingui compile",
18     "ui": "bunx --bun shadcn@latest --",
19     "db:push": "drizzle-kit push",
20     "db:studio": "drizzle-kit studio",
21     "db:generate": "drizzle-kit generate",
22     "db:migrate": "drizzle-kit migrate",
23     "db:start": "docker compose up -d",
24     "db:watch": "docker compose up",
25     "db:stop": "docker compose stop",
26     "db:down": "docker compose down"
27   },
28   "dependencies": {
29     "@hookform/resolvers": "^5.4.0",
30     "@lingui/core": "5.4.1",
31     "@lingui/react": "5.4.1",
32     "@orpc/client": "^1.7.11",
33     "@orpc/json-schema": "^1.7.11",
34     "@orpc/openapi": "^1.7.11",
35     "@orpc/server": "^1.7.11",
36     "@orpc/tanstack-query": "^1.7.11",
37     "@orpc/zod": "^1.7.11",
38     "@radix-ui/react-avatar": "^1.1.10",
39     "@radix-ui/react-collapsible": "^1.1.12",
40     "@radix-ui/react-dialog": "^1.1.15",
41     "@radix-ui/react-dropdown-menu": "^2.1.15",
42     "@radix-ui/react-label": "^2.1.8",
43     "@radix-ui/react-popover": "^1.1.15",
44     "@radix-ui/react-progress": "^1.1.8",
45     "@radix-ui/react-scroll-area": "^1.2.10",
46     "@radix-ui/react-select": "^2.2.5",
47     "@radix-ui/react-separator": "^1.1.7",
48     "@radix-ui/react-slider": "^1.3.5",
49     "@radix-ui/react-slot": "^1.2.3",
50     "@radix-ui/react-switch": "^1.2.5",
51     "@radix-ui/react-tooltip": "^1.2.8",
52     "@t3-oss/env-core": "^0.12.0",
53     "@tailwindcss/vite": "^4.1.11",
54     "@tanstack/react-form": "^1.19.0",
55     "@tanstack/react-query": "5.84.2",
56     "@tanstack/react-query-devtools": "5.84.2",
57     "@tanstack/react-router": "1.131.2",
58     "@tanstack/react-router-devtools": "1.131.2",
59     "@tanstack/react-router-ssr-query": "1.131.2",
60     "@tanstack/react-start": "1.131.2",
61     "@types/react-grid-layout": "^1.3.5",
62     "better-auth": "1.3.4",
63     "class-variance-authority": "^0.7.1",
64     "clsx": "^2.1.1",
65     "date-fns": "^4.1.0",
66     "dotenv": "^17.2.1",
67     "drizzle-orm": "^0.44.4",
68     "jotai": "^2.15.2",
69     "leaflet": "^1.9.4",
70     "lucide-react": "^0.556.0",
71     "pg": "^8.16.3",
72     "radix-ui": "^1.4.3",
73     "react": "^19.1.1",
74     "react-dom": "^19.1.1",
75     "react-grid-layout": "^1.5.2",
76     "react-hook-form": "^7.76.1",
77     "react-leaflet": "^5.0.0",
78     "react-resizable": "^3.0.5",
79     "recharts": "^3.3.0",
80     "tailwind-merge": "^3.3.1",
81     "tailwindcss": "^4.1.11",
82     "tw-animate-css": "^1.3.6",
83     "uuid": "^13.0.0",
84     "zod": "^4.4.3"
85   },
86   "devDependencies": {
87     "@lingui/babel-plugin-macro": "5.4.1",
88     "@lingui/cli": "5.4.1",
89     "@lingui/vite-plugin": "5.4.1",
90     "@testing-library/dom": "^10.4.1",
91     "@testing-library/react": "^16.3.0",
92     "@types/bun": "^1.2.20",
93     "@types/leaflet": "^1.9.21",
94     "@types/pg": "^8.15.5",
95     "@types/react": "^19.1.9",
96     "@types/react-dom": "^19.1.7",
97     "@vitejs/plugin-react": "^4.7.0",
98     "babel": "^6.23.0",
99     "drizzle-kit": "^0.31.4",
100    "jsdom": "^26.1.0",
101    "typescript": "^5.9.2",
102    "vite": "^7.1.1",
103    "vite-tsconfig-paths": "^5.1.4",
104    "vitest": "^3.2.4",
105    "web-vitals": "^4.2.4"
106  }
107 }
108

```

Gambar 2.2 *Dependency utama aplikasi pada package.json*

BAB 3

PELAKSANAAN KERJA PRAKTEK

3.1 Persiapan Lingkungan dan Tools

3.1.1 Prasyarat Sistem

Sebelum memulai kegiatan pengembangan dan analisis, perlu disiapkan lingkungan kerja dengan prasyarat sebagai berikut:

1. **Bun** — Runtime JavaScript dan package manager yang digunakan untuk menjalankan script, mengelola dependency, dan menjalankan development server.
2. **PostgreSQL** — Database relasional yang digunakan sebagai penyimpanan data utama aplikasi. Database dapat dijalankan secara lokal maupun melalui Docker.
3. **Git** — Version control system yang digunakan untuk melacak perubahan source code dan berkolaborasi dalam pengembangan.

3.1.2 Instalasi Dependency

Setelah repository dikloning, langkah pertama yang dilakukan adalah menginstal seluruh dependency proyek menggunakan perintah:

[codeblock]

Perintah ini mengunduh dan memasang seluruh package yang didefinisikan dalam ``package.json`` workspaces. Struktur monorepo dengan Turborepo memastikan bahwa dependency diinstal untuk seluruh workspace secara efisien.

3.1.3 Konfigurasi Environment

Proyek menyediakan file ``.env.example`` sebagai template konfigurasi lingkungan. Variabel lingkungan yang perlu dikonfigurasi meliputi:

- ``DATABASE_URL`` — URL koneksi PostgreSQL.
- ``BETTER_AUTH_SECRET`` — Secret key untuk Better Auth.
- ``BETTER_AUTH_URL`` — Base URL aplikasi untuk authentication.

- `'CORS_ORIGIN'` — Origin yang diizinkan untuk CORS.
- `'VITE_BETTER_AUTH_URL'` — URL authentication untuk sisi client.

3.1.4 Menjalankan Database

Database PostgreSQL dapat dijalankan menggunakan Docker Compose yang telah disediakan:

[code block]

Setelah database berjalan, skema database dibuat dan diperbarui menggunakan Drizzle ORM:

[code block]

3.1.5 Menjalankan Development Server

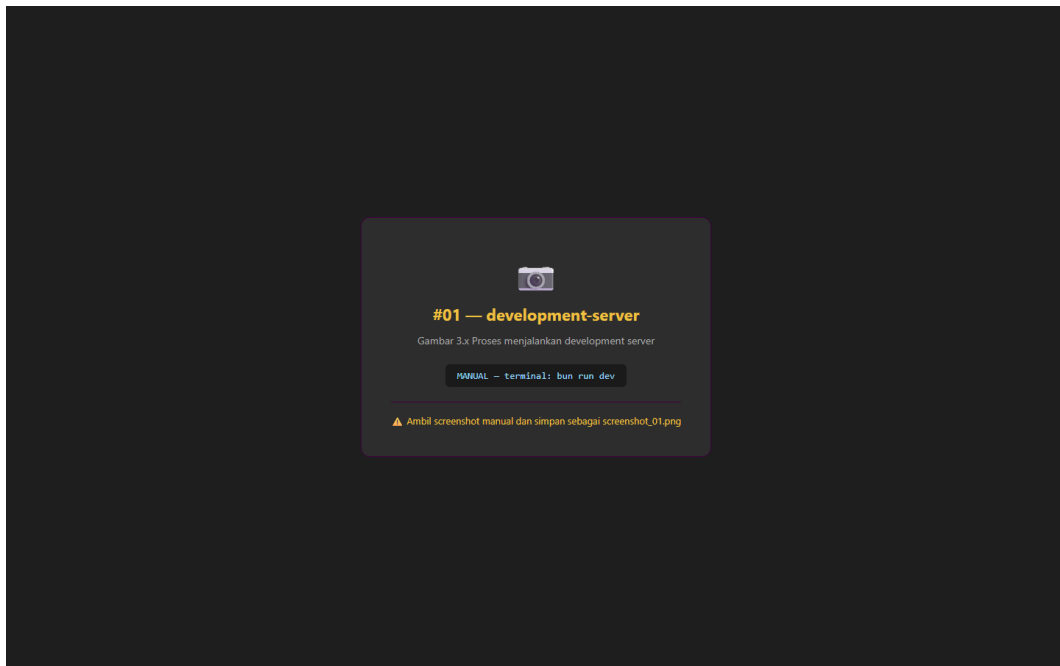
Development server dijalankan dengan perintah:

[code block]

Perintah ini menjalankan aplikasi dalam mode pengembangan dengan fitur hot module replacement. Aplikasi dapat diakses melalui browser pada alamat yang ditentukan oleh konfigurasi Vite.

3.1.6 Perintah Penting

Perintah	Fungsi
<code>'bun run build'</code>	Menjalankan production build.
<code>'bun run check'</code>	Menjalankan linting dan formatting check.
<code>'bun run db:push'</code>	Mendorong perubahan schema ke database.
<code>'bun run db:generate'</code>	Menghasilkan file migration dari schema.
<code>'bun run db:migrate'</code>	Menjalankan migrasi database.
<code>'bun run db:studio'</code>	Membuka Drizzle Studio untuk eksplorasi database.
<code>'bun run check-types'</code>	Menjalankan type checking TypeScript.



Gambar 3.16 Proses menjalankan development server

3.2 Kegiatan Survei Lapangan dan Observasi Wilayah

3.2.1 Tujuan Kegiatan Lapangan

Selama kegiatan magang, penulis berkesempatan mengikuti survei lapangan ke beberapa wilayah binaan PT Petrokimia Gresik. Kegiatan ini bertujuan untuk mengamati secara langsung kondisi pasar, potensi pertanian, dan aktivitas penjualan di lapangan. Pemahaman kontekstual yang diperoleh dari kegiatan ini menjadi acuan dalam memvalidasi data yang dikelola dalam Sistem Informasi Manajemen Produk Baru. **[PERLU VERIFIKASI tujuan spesifik kegiatan survei]**

3.2.2 Lokasi yang Dikunjungi

Kegiatan survei lapangan dilaksanakan di empat wilayah, yaitu:

1. **Bojonegoro** — Observasi kondisi pertanian dan potensi pasar di wilayah Jawa Timur bagian barat.
2. **Banyuwangi** — Kunjungan ke kawasan pertanian di ujung timur Pulau Jawa.
3. **Jember** — Observasi sentra produksi pertanian dan perkebunan.
4. **Lumajang** — Pemantauan aktivitas distribusi produk dan kondisi kios.

[PERLU VERIFIKASI tanggal kunjungan dan aktivitas spesifik per wilayah]

3.2.3 Aktivitas Observasi

Aktivitas yang dilakukan selama survei lapangan meliputi:

- Kunjungan ke kios atau lokasi penjualan untuk mengamati proses bisnis di lapangan.
- Pencatatan data geografis dan informasi lapangan yang relevan dengan data potensi pasar.
- Observasi jenis komoditas unggulan dan kebutuhan pupuk di masing-masing wilayah.
- Dokumentasi kondisi lapangan sebagai bahan verifikasi data sistem.

3.2.4 Hubungan dengan Kebutuhan Data Sistem

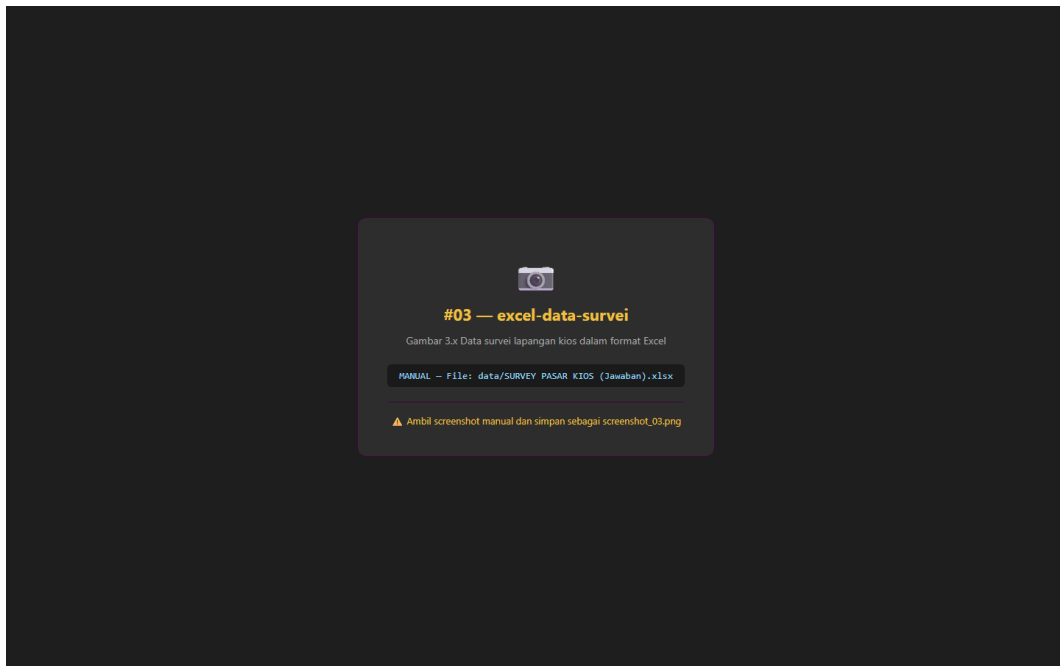
Data yang diamati di lapangan memiliki keterkaitan langsung dengan data yang dikelola dalam sistem:

- Informasi kondisi pasar menjadi acuan validasi data potensi provinsi dan kabupaten pada sistem.
- Data kios dan lokasi geografis mendukung modul Stall dan visualisasi peta interaktif.
- Observasi komoditas per wilayah memperkuat data pada modul Province Commodity dan Regency Commodity.
- Hasil survei digunakan dalam proses import data kios dari file Excel ke database melalui script `scripts/seed-stalls.ts`.

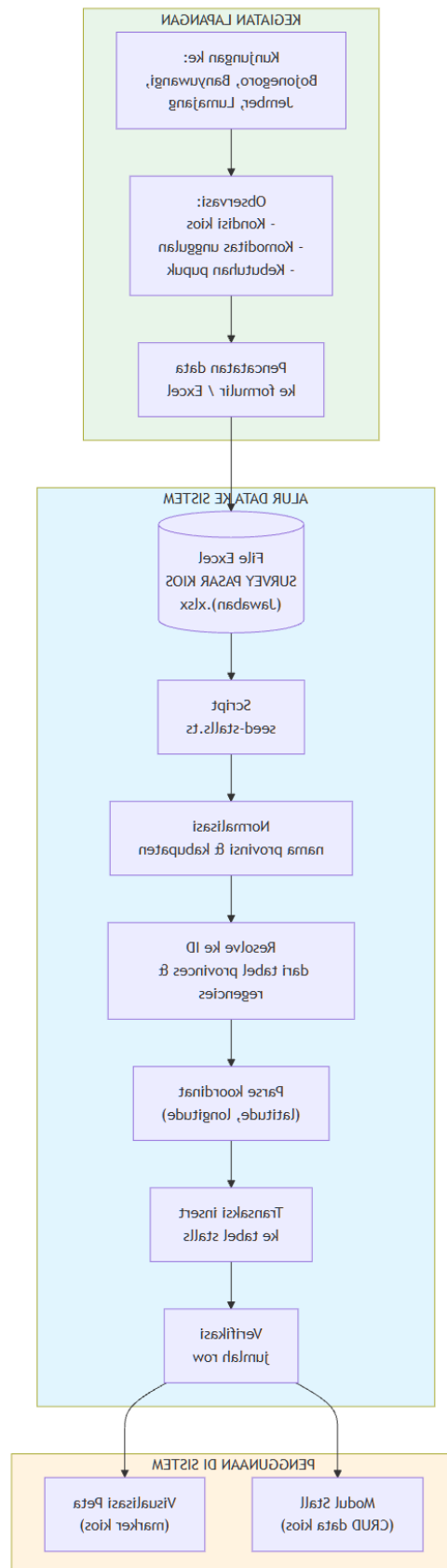
3.2.5 Manfaat Observasi Lapangan

Kegiatan observasi lapangan memberikan manfaat sebagai berikut:

- Memberikan pemahaman langsung tentang alur distribusi produk dari gudang ke kios dan ke petani.
- Membantu interpretasi data potensi pasar yang tersimpan dalam sistem.
- Menjembatani kesenjangan antara representasi data digital dengan kondisi aktual di lapangan.
- Mendukung rekomendasi pengembangan fitur berdasarkan kebutuhan pengguna lapangan.



Gambar 3.17 Data survei lapangan kios dalam format Excel



Gambar 3.15 Diagram alur data survei lapangan ke dalam sistem

3.3 Arsitektur Sistem

3.3.1 Arsitektur Full-Stack Monolith

Sistem Satu Peta Pasar menerapkan arsitektur full-stack monolith pada workspace `apps/web`. Berbeda dengan deskripsi template proyek yang membayangkan backend Hono terpisah pada folder `apps/server`, implementasi aktif menempatkan frontend dan backend dalam satu aplikasi TanStack Start. Seluruh lapisan aplikasi — mulai dari antarmuka pengguna, routing, API, authentication, hingga akses database — terintegrasi dalam satu kesatuan deployment.

3.3.2 Lapisan Aplikasi

Frontend:

- React 19 sebagai library komponen antarmuka.
- TanStack Router untuk file-based routing, loader, dan route guard.
- TanStack Query untuk pengelolaan server state dan caching.
- Tailwind CSS untuk styling utility-first.
- Leaflet dan React Leaflet untuk visualisasi peta interaktif.

Backend:

- Server route TanStack Start di `apps/web/src/routes/api` mengekspos endpoint.
- oRPC sebagai boundary API type-safe yang mendefinisikan procedure.
- Zod untuk validasi input setiap procedure.
- Better Auth untuk authentication dan session management.

Database:

- PostgreSQL sebagai database relasional utama.
- Drizzle ORM untuk query type-safe, definisi schema, dan migrasi.

3.3.3 Alur Request

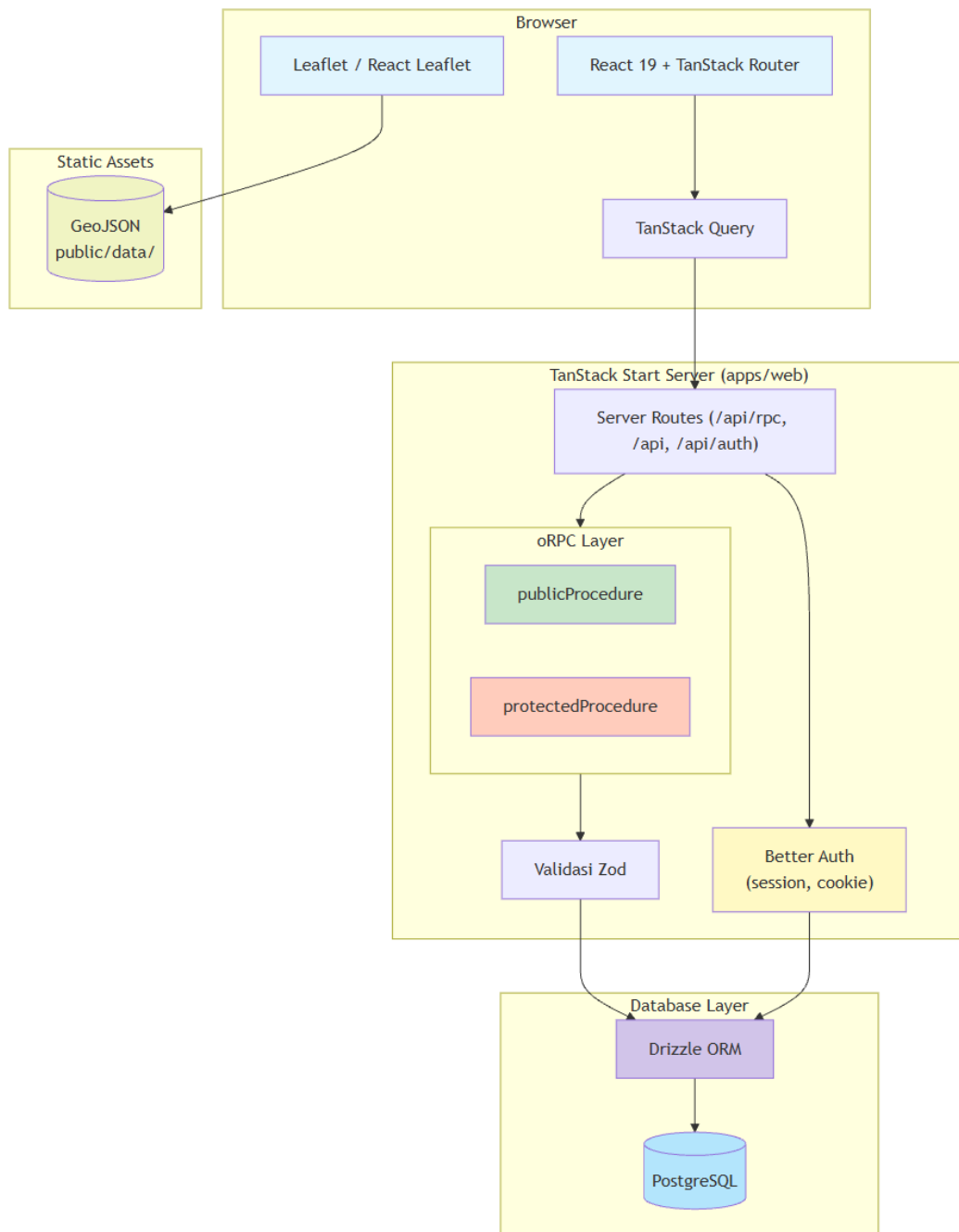
Alur request dari antarmuka pengguna hingga database dapat digambarkan sebagai berikut:

[code block]

Data peta statis (GeoJSON) dimuat langsung dari folder `public/data/` oleh Leaflet tanpa melalui API.

3.3.4 Perbedaan dengan Deskripsi Template

Dokumentasi template proyek (`AGENTS.md`) menjelaskan struktur dengan backend Hono terpisah pada `apps/server`. Setelah dilakukan penelusuran source code, ditemukan bahwa struktur tersebut tidak sesuai dengan implementasi aktif. Folder `apps/server` tidak ditemukan, dan seluruh backend dijalankan dalam `apps/web` menggunakan server route TanStack Start. Laporan ini menggunakan kondisi implementasi aktif sebagai acuan pembahasan.



Gambar 3.1 Diagram arsitektur full-stack monolith Satu Peta Pasar

3.4 Struktur Proyek dan Pembagian Layer

3.4.1 Struktur Root

Struktur direktori root proyek terdiri dari:

- `apps/` — Container workspace aplikasi, dengan `apps/web` sebagai aplikasi aktif.
- `data/` — Berkas data pendukung, termasuk file Excel survei lapangan.

- `scripts/` — Script utilitas, termasuk `seed-stalls.ts` untuk import data kios.
- `docs/` — Dokumentasi proyek dan laporan magang.
- Konfigurasi root: `package.json`, `turbo.json`, `bun.lock`, `docker-compose.yml`,
`.env.example`.

3.4.2 Struktur Aplikasi (`apps/web`)

Struktur direktori `apps/web/src/` mengikuti pola route-located feature modules:

Direktori	Fungsi
`lib/`	Infrastruktur bersama: auth, database, Lingui, oRPC, TanStack Query.
`lib/auth/`	Konfigurasi Better Auth, client auth, middleware guard.
`lib/db/`	Koneksi database, schema Drizzle, dan migrasi.
`lib/db/schema/`	Schema tabel per domain: auth, map-product, sale, stall, todo.
`lib/orpc/`	Context, client, procedure, schema, dan komposisi router oRPC.
`routes/`	File-based routes: halaman publik, API, auth, admin, map.
`routes/admin/`	Layout, guard, navigasi, dan seluruh modul administrasi.
`routes/admin/*/-app/`	Procedure/use case backend (get, create, update, delete).
`routes/admin/*/-domain/`	Schema dan tipe data domain.
`routes/admin/*/-components/`	Tabel, form, dialog komponen presentasi.
`public/data/`	GeoJSON batas Indonesia dan kabupaten.

3.4.3 Policy Dependency

Alur dependensi antar lapisan mengikuti arah berikut:

[code block]

Struktur ini memastikan bahwa setiap lapisan memiliki tanggung jawab yang jelas dan dependensi hanya mengalir satu arah. Komponen UI tidak mengakses database secara langsung, melainkan melalui procedure oRPC yang menjembatani komunikasi client-server.



Marketing Map

Admin Panel



← Back to Home

Overview

📈 Dashboard

Marketing Map

📈 Regions >

📈 Lands >

📈 Commodities >

📈 Products >

Potential

📈 Potential >

Sale

📈 Sales Overview >

Stall

Dashboard

Welcome

Product

3

Ad

Product

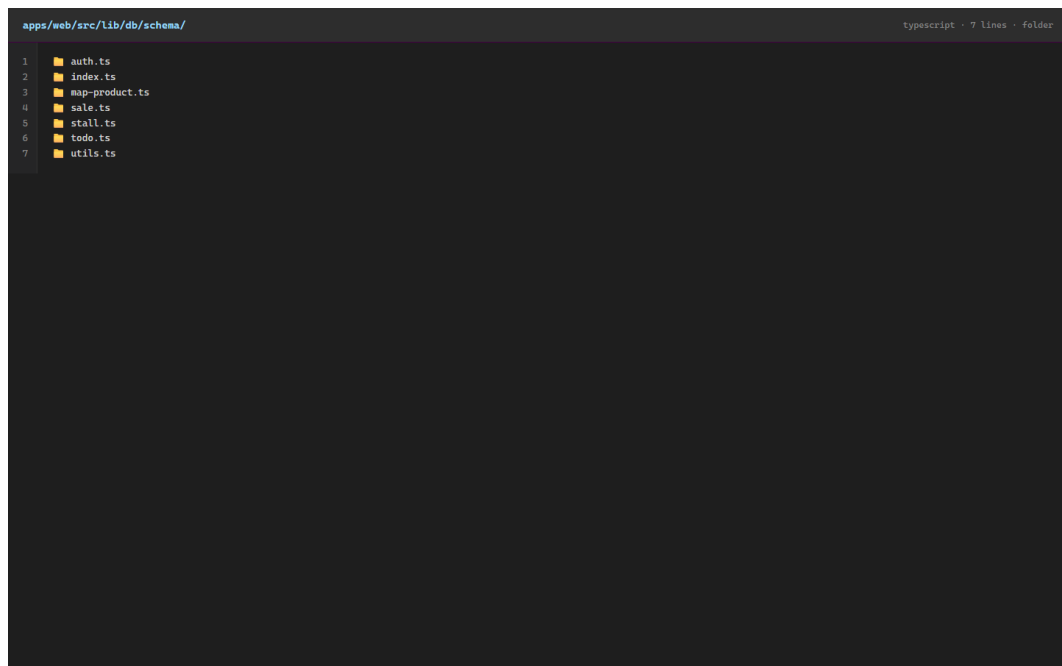
7

Proc

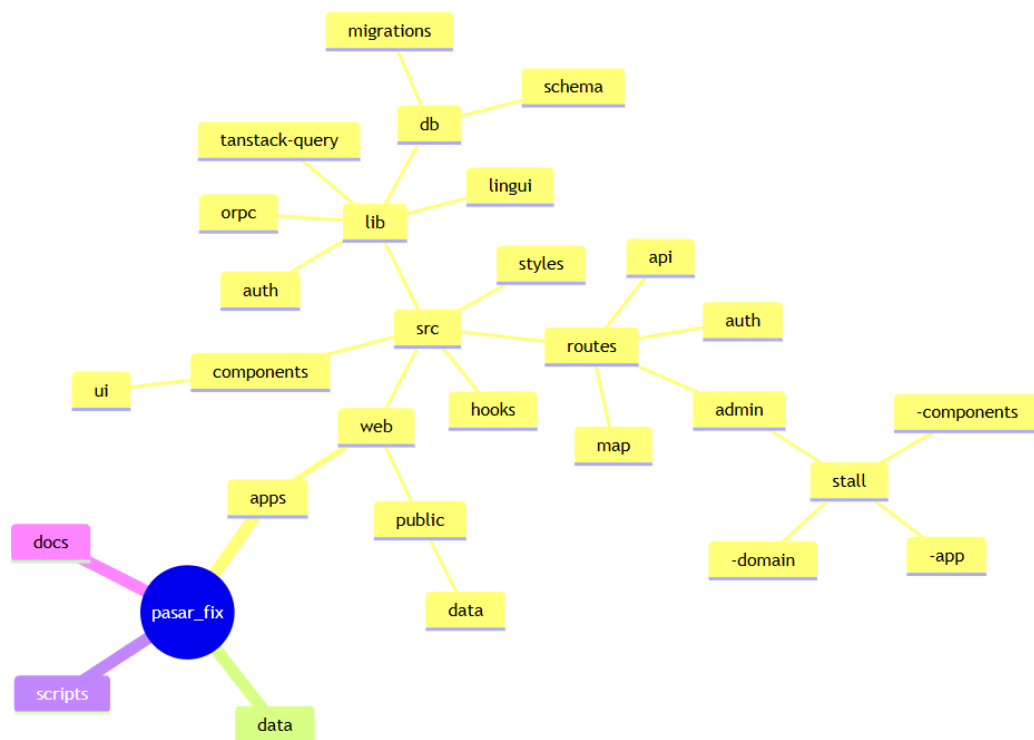
Product

Co

Gambar 3.18 Sidebar navigasi panel admin



Gambar 3.19 Struktur file schema Drizzle ORM



Gambar 3.2 Diagram struktur direktori dan pembagian layer proyek

3.5 Perancangan Database

3.5.1 Gambaran Umum

Database sistem Satu Peta Pasar terdiri dari 22 tabel yang mencakup domain autentikasi, wilayah, lahan, komoditas, produk, potensi, kios, penjualan, dan demo. Seluruh tabel menggunakan primary key bertipe UUID untuk menjamin keunikan identitas data secara global. Relasi antar domain dibangun menggunakan foreign key yang menghubungkan entitas-entitas terkait.

3.5.2 Domain Autentikasi

Domain autentikasi dikelola oleh Better Auth dan terdiri dari empat tabel:

Tabel	Fungsi	Primary Key	Foreign Key
`session`	Menyimpan session login beserta token dan masa berlaku.	`id`	`user_id` → `user.id`
`account`	Menyimpan credential dan password hash.	`id`	`user_id` → `user.id`
`verification`	Menyimpan token verifikasi.	`id`	-

Role pengguna yang didefinisikan meliputi `admin`, `viewer`, dan `guest`. Role default bagi pengguna baru adalah `guest`.

3.5.3 Domain Wilayah dan Lahan

Tabel	Fungsi	Primary Key	Foreign Key
`regencies`	Data master kabupaten/kota.	`id`	`province_id` → `provinces.id`
`land_types`	Data master jenis lahan.	`id`	-
`province_lands`	Luas lahan per provinsi per jenis lahan.	`id`	`province_id` → `provinces.id`, `land_type_id` → `land_types.id`
`regency_lands`	Luas lahan per kabupaten per jenis lahan.	`id`	`regency_id` → `regencies.id`, `land_type_id` → `land_types.id`

Relasi: satu provinsi memiliki banyak kabupaten. Satu jenis lahan digunakan oleh banyak provinsi dan kabupaten.

3.5.4 Domain Komoditas

Tabel	Fungsi	Primary Key	Foreign Key
`province_commodities`	Data komoditas per provinsi.	`id`	`province_id` → `provinces.id`, `commodity_type_id` → `commodity_types.id`
`regency_commodities`	Data komoditas per kabupaten.	`id`	`regency_id` → `regencies.id`, `commodity_type_id` → `commodity_types.id`

Relasi: satu jenis komoditas terhubung dengan satu jenis lahan. Setiap komoditas dapat dicatat pada tingkat provinsi dan kabupaten.

3.5.5 Domain Produk dan Potensi

Tabel	Fungsi	Primary Key	Foreign Key
`product_brands`	Data brand produk.	`id`	`product_type_id` → `product_types.id`
`product_dosages`	Dosis brand untuk komoditas.	`id`	`commodity_type_id` → `commodity_types.id`, `product_brand_id` → `product_brands.id`
`province_potentials`	Nilai potensi brand di provinsi.	`id`	`province_id` → `provinces.id`, `product_brand_id` → `product_brands.id`
`regency_potentials`	Nilai potensi brand di kabupaten.	`id`	`regency_id` → `regencies.id`, `product_brand_id` → `product_brands.id`

Relasi: hierarki produk dimulai dari jenis produk, kemudian brand produk, hingga dosis produk untuk komoditas tertentu. Potensi pasar dicatat pada tingkat provinsi dan kabupaten.

3.5.6 Domain Kios

Tabel	Fungsi	Primary Key	Foreign Key
`stall_product_brands`	Tabel penghubung many-to-many kios dan brand.	`id`	`stall_id` → `stalls.id`, `product_brand_id` → `product_brands.id`

Relasi: satu kios berada pada satu provinsi dan satu kabupaten. Satu kios dapat menjual banyak brand produk, dan satu brand produk dapat tersedia di banyak kios.

3.5.7 Domain Penjualan

Tabel	Fungsi	Primary Key	Foreign Key
`daily_sales`	Data penjualan harian per brand.	`id`	`product_brand_id` → `product_brands.id`, `province_id` (belum FK)

Relasi: setiap data penjualan terhubung dengan satu brand produk.

3.5.8 Domain Demo

3.5.9 Catatan Integritas Database

Berdasarkan hasil analisis, ditemukan beberapa catatan terkait integritas database:

1. **Missing foreign key:** Field `province\_id` pada tabel `daily\_sales` telah tersedia untuk menyimpan konteks wilayah, tetapi definisi foreign key belum ditemukan pada schema yang dianalisis. Penambahan constraint foreign key perlu didahului pemeriksaan data aktual.
2. **Indikasi mismatch schema-migration:** Pada tabel `sales\_realizations`, file migration memiliki typo `realizaton\_ytd`, sedangkan schema menggunakan `realization\_ytd`. Perbedaan ini perlu diverifikasi terhadap database aktual untuk memastikan konsistensi.
3. **Unique constraint:** Kombinasi unik seperti `(province\_id, land\_type\_id, year)` pada `province\_lands` dan `(stall\_id, product\_brand\_id)` pada `stall\_product\_brands` belum terdokumentasi secara eksplisit.
4. **Field `updated\_at`:** Tabel `product\_dosages` tidak memiliki field `updated\_at` pada schema yang dianalisis.

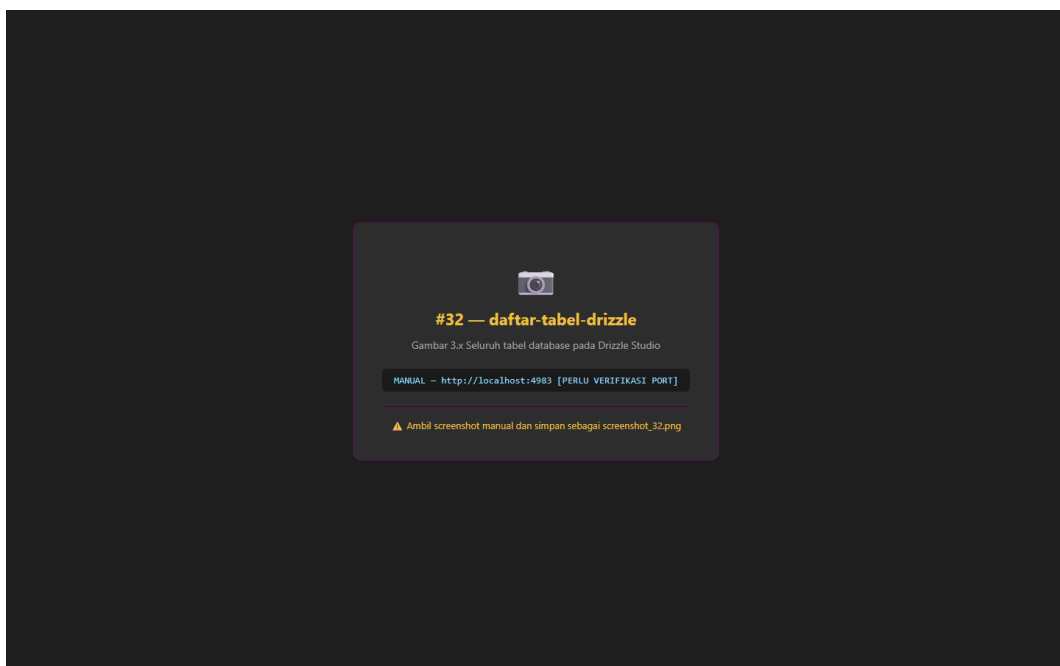
```

apps/web/src/lib/db/schema/auth.ts                                     typescript · 62 lines

1  import { boolean, pgTable, text, timestamp, uuid } from 'drizzle-orm/pg-core';
2  import { generateUUID } from './utils';
3
4  export const user = pgTable('user', {
5    id: uuid('id').primaryKey().$defaultFn(generateUUID),
6    name: text('name').notNull(),
7    email: text('email').notNull().unique(),
8    emailVerified: boolean('email_verified')
9      .$defaultFn(() => false)
10     .notNull(),
11    image: text('image'),
12    role: text('role')
13      .$defaultFn(() => 'guest')
14     .notNull(),
15    createdAt: timestamp('created_at')
16      .$defaultFn(() => new Date())
17     .notNull(),
18    updatedAt: timestamp('updated_at')
19      .$defaultFn(() => new Date())
20     .notNull(),
21  });
22
23  export const session = pgTable('session', {
24    id: uuid('id').primaryKey().$defaultFn(generateUUID),
25    expiresAt: timestamp('expires_at').notNull(),
26    token: text('token').notNull().unique(),
27    createdAt: timestamp('created_at').notNull(),
28    updatedAt: timestamp('updated_at').notNull(),
29    ipAddress: text('ip_address'),
30    userAgent: text('user_agent'),
31    userId: uuid('user_id')
32      .notNull()
33      .references(() => user.id, { onDelete: 'cascade' }),
34  });
35
36  export const account = pgTable('account', {
37    id: uuid('id').primaryKey().$defaultFn(generateUUID),
38    accountId: text('account_id').notNull(),
39    providerId: text('provider_id').notNull(),
40    userId: uuid('user_id')
41      .notNull()
42      .references(() => user.id, { onDelete: 'cascade' }),
43    accessToken: text('access_token'),
44    refreshToken: text('refresh_token'),
45    idToken: text('id_token'),
46    accessTokenExpiresAt: timestamp('access_token_expires_at'),
47    refreshTokenExpiresAt: timestamp('refresh_token_expires_at'),
48    scope: text('scope'),
49    password: text('password'),
50    createdAt: timestamp('created_at').notNull(),
51    updatedAt: timestamp('updated_at').notNull(),
52  });
53
54  export const verification = pgTable('verification', {
55    id: uuid('id').primaryKey().$defaultFn(generateUUID),
56    identifier: text('identifier').notNull(),
57    value: text('value').notNull(),
58    expiresAt: timestamp('expires_at').notNull(),
59    createdAt: timestamp('created_at').$defaultFn(() => new Date()),
60    updatedAt: timestamp('updated_at').$defaultFn(() => new Date()),
61  });
62

```

Gambar 3.20 Definisi tabel autentikasi pada schema Drizzle



Gambar 3.21 Seluruh tabel database pada Drizzle Studio

- ``healthCheck`` — Pengecekan status API.
- ``auth.getSession`` — Mendapatkan session pengguna saat ini.
- ``map.getProvinces`` — Data provinsi untuk peta.
- ``map.getStalls`` — Data kios untuk peta.

Protected procedure memerlukan session pengguna yang valid dan digunakan untuk endpoint administratif. Seluruh endpoint CRUD pada modul bisnis termasuk dalam kategori ini.

3.6.3 Context

Setiap procedure oRPC memiliki akses terhadap context yang menyediakan:

- **Session** — Informasi session pengguna dari Better Auth, digunakan untuk memvalidasi authentication.
- **Database (db)** — Instance Drizzle ORM untuk menjalankan query ke PostgreSQL.

3.6.4 Struktur Endpoint per Modul

Wilayah (Region):

- ``admin.region.province.get`, `create`, `update`, `delete``
- ``admin.region.regency.get`, `create`, `update`, `delete``

Lahan (Land):

- ``admin.land.land_type.get`, `create`, `update`, `delete``
- ``admin.land.province_land.get`, `create`, `update`, `delete``
- ``admin.land.regency_land.get`` (read-only)

Komoditas (Commodity):

- ``admin.commodity.commodity_type.get`, `create`, `update`, `delete``
- ``admin.commodity.province_commodity.get`` (read-only)
- ``admin.commodity.regency_commodity.get`, `create`, `update`, `delete``

Produk (Product):

- ``admin.product.product_type.get`, `create`, `update`, `delete``
- ``admin.product.product_brand.get`, `create`, `update`, `delete``

- `admin.product.product\_dosage.get`, `create`, `update`, `delete`

Potensi (Potential):

- `admin.potential.province\_potential.get` (read-only)
- `admin.potential.regency\_potential.\*` **[PERLU VERIFIKASI endpoint aktif]**

Penjualan (Sales):

- `admin.sale.sale\_overview.get`
- `admin.sale.sales\_realization.get`, `create`, `update`, `delete`
- `admin.sale.daily\_sales.get`, `create`, `update`, `delete`

Kios (Stall):

- `admin.stall.get`, `create`, `update`, `delete`
- `admin.stall.stall\_product\_brand.get`, `sync` (assignment brand)

Pengguna (User):

- `admin.user.get`, `getById`, `create`, `update`, `delete`

Dashboard:

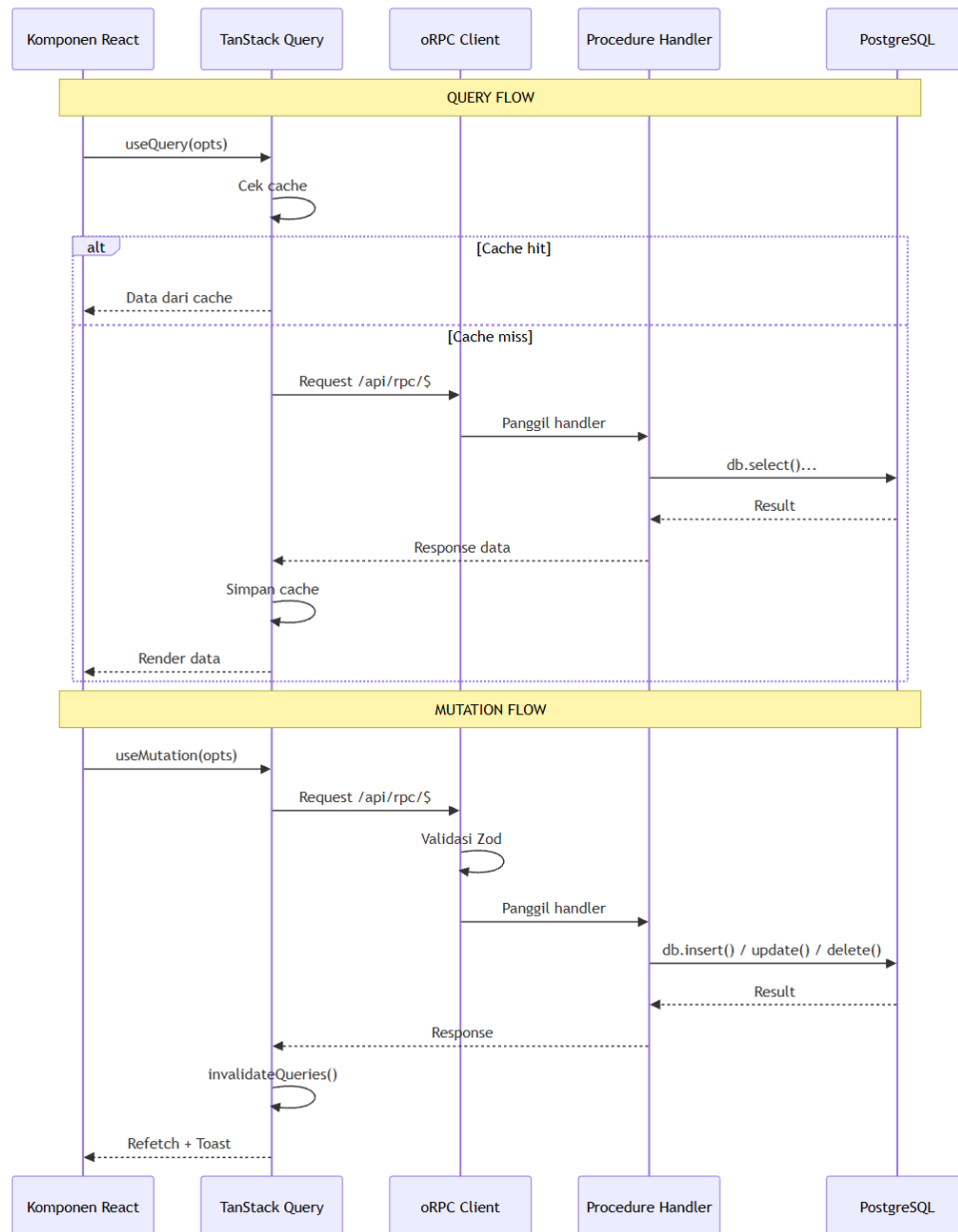
- `admin.dashboard.getSummary`

3.6.5 Validasi Input

Setiap procedure oRPC menggunakan Zod untuk memvalidasi input. Validasi mencakup tipe data, panjang karakter, format, dan constraint domain. Zod schema didefinisikan pada folder `-domain` masing-masing modul dan digunakan bersama antara validasi form frontend dan validasi API backend.

```
1 import { protectedProcedure, publicProcedure } from '@lib/orpc';
2 import { getDashboardSummary } from '@routes/admin/-app/get-dashboard-summary';
3 import { createCommodityType } from '@routes/admin/commodity/-app/create-commodity-type';
4 import { deleteCommodityType } from '@routes/admin/commodity/-app/delete-commodity-type';
5 import { getCommodityTypes } from '@routes/admin/commodity/-app/get-commodity-types';
6 import { updateCommodityType } from '@routes/admin/commodity/-app/update-commodity-type';
7 import { createProvinceCommodity } from '@routes/admin/commodity/province-commodity/-app/create-province-commodity';
8 import { deleteProvinceCommodity } from '@routes/admin/commodity/province-commodity/-app/delete-province-commodity';
9 import { getProvinceCommodities } from '@routes/admin/commodity/province-commodity/-app/get-province-commodities';
10 import { updateProvinceCommodity } from '@routes/admin/commodity/province-commodity/-app/update-province-commodity';
11 import { createRegencyCommodity } from '@routes/admin/commodity/regency-commodity/-app/create-regency-commodity';
12 import { deleteRegencyCommodity } from '@routes/admin/commodity/regency-commodity/-app/delete-regency-commodity';
13 import { getRegencyCommodities } from '@routes/admin/commodity/regency-commodity/-app/get-regency-commodities';
14 import { updateRegencyCommodity } from '@routes/admin/commodity/regency-commodity/-app/update-regency-commodity';
15 import { createLandType } from '@routes/admin/land/-app/create-land-type';
16 import { deleteLandType } from '@routes/admin/land/-app/delete-land-type';
17 import { getLandTypes } from '@routes/admin/land/-app/get-land-types';
18 import { updateLandType } from '@routes/admin/land/-app/update-land-type';
19 import { createProvinceLand } from '@routes/admin/land/province-land/-app/create-province-land';
20 import { deleteProvinceLand } from '@routes/admin/land/province-land/-app/delete-province-land';
21 import { getProvinceLands } from '@routes/admin/land/province-land/-app/get-province-lands';
22 import { updateProvinceLand } from '@routes/admin/land/province-land/-app/update-province-land';
23 import { createRegencyLand } from '@routes/admin/land/regency-land/-app/create-regency-land';
24 import { deleteRegencyLand } from '@routes/admin/land/regency-land/-app/delete-regency-land';
25 import { getRegencyLands } from '@routes/admin/land/regency-land/-app/get-regency-lands';
26 import { updateRegencyLand } from '@routes/admin/land/regency-land/-app/update-regency-land';
27 import { createProvincePotential } from '@routes/admin/potential/province-potential/-app/create-province-potential';
28 import { deleteProvincePotential } from '@routes/admin/potential/province-potential/-app/delete-province-potential';
29 import { getProvincePotentials } from '@routes/admin/potential/province-potential/-app/get-province-potentials';
30 import { updateProvincePotential } from '@routes/admin/potential/province-potential/-app/update-province-potential';
31 import { createRegencyPotential } from '@routes/admin/potential/regency-potential/-app/create-regency-potential';
32 import { deleteRegencyPotential } from '@routes/admin/potential/regency-potential/-app/delete-regency-potential';
33 import { getRegencyPotentials } from '@routes/admin/potential/regency-potential/-app/get-regency-potentials';
34 import { updateRegencyPotential } from '@routes/admin/potential/regency-potential/-app/update-regency-potential';
35 import { createProductType } from '@routes/admin/product/-app/create-product-type';
36 import { deleteProductType } from '@routes/admin/product/-app/delete-product-type';
37 import { getProductTypes } from '@routes/admin/product/-app/get-product-types';
38 import { updateProductType } from '@routes/admin/product/-app/update-product-type';
39 import { createProductBrand } from '@routes/admin/product/product-brand/-app/create-product-brand';
40 import { deleteProductBrand } from '@routes/admin/product/product-brand/-app/delete-product-brand';
41 import { getProductBrands } from '@routes/admin/product/product-brand/-app/get-product-brands';
42 import { updateProductBrand } from '@routes/admin/product/product-brand/-app/update-product-brand';
43 import { createProductDosage } from '@routes/admin/product/product-dosage/-app/create-product-dosage';
44 import { deleteProductDosage } from '@routes/admin/product/product-dosage/-app/delete-product-dosage';
45 import { getProductDosages } from '@routes/admin/product/product-dosage/-app/get-product-dosages';
46 import { updateProductDosage } from '@routes/admin/product/product-dosage/-app/update-product-dosage';
47 import { createProvince } from '@routes/admin/region/province/-app/create-province';
48 import { deleteProvince } from '@routes/admin/region/province/-app/delete-province';
49 import { getAllProvinces } from '@routes/admin/region/province/-app/get-all-provinces';
50 import { getProvinces } from '@routes/admin/region/province/-app/get-provinces';
51 import { updateProvince } from '@routes/admin/region/province/-app/update-province';
52 import { createRegency } from '@routes/admin/region/regency/-app/create-regency';
53 import { deleteRegency } from '@routes/admin/region/regency/-app/delete-regency';
54 import { getRegencies } from '@routes/admin/region/regency/-app/get-regencies';
55 import { updateRegency } from '@routes/admin/region/regency/-app/update-regency';
56 import { createSalesRealization } from '@routes/admin/sale/-app/create-sales-realization';
57 import { deleteSalesRealization } from '@routes/admin/sale/-app/delete-sales-realization';
58 import { getSalesOverview } from '@routes/admin/sale/-app/get-sales-overview';
59 import { getSalesRealizations } from '@routes/admin/sale/-app/get-sales-realizations';
60 import { updateSalesRealization } from '@routes/admin/sale/-app/update-sales-realization';
61 import { createDailySales } from '@routes/admin/sale/sale-daily/-app/create-daily-sales';
62 import { deleteDailySales } from '@routes/admin/sale/sale-daily/-app/delete-daily-sales';
63 import { getDailySales } from '@routes/admin/sale/sale-daily/-app/get-daily-sales';
64 import { updateDailySales } from '@routes/admin/sale/sale-daily/-app/update-daily-sales';
65 import { assignProductBrand } from '@routes/admin/stall/-app/assign-product-brand';
66 import { createStall } from '@routes/admin/stall/-app/create-stall';
67 import { deleteStall } from '@routes/admin/stall/-app/delete-stall';
68 import { getStallProductBrands } from '@routes/admin/stall/-app/get-stall-product-brand';
69 import { getStalls } from '@routes/admin/stall/-app/get-stalls';
70 import { updateStall } from '@routes/admin/stall/-app/update-stall';
71 import { createUser } from '@routes/admin/user/-app/create-user';
72 import { deleteUser } from '@routes/admin/user/-app/delete-user';
73 import { getUserById } from '@routes/admin/user/-app/get-user-by-id';
74 import { getUsers } from '@routes/admin/user/-app/get-users';
75 import { updateUser } from '@routes/admin/user/-app/update-user';
76 import { getSession } from '@routes/auth/-app/get-session';
77 import { getMapStalls } from '@routes/map/-app/get-map-stalls';
78 import { createTodo } from '@routes/todos/-app/create-todo';
79 import { deleteTodo } from '@routes/todos/-app/delete-todo';
80 import { getTodos } from '@routes/todos/-app/get-todos';
81 import { toggleTodo } from '@routes/todos/-app/toggle-todo';
82 /**
83  *
84  * Main ORPC Router
85  *
86  * This router provides a unified API interface for the entire application,
87  * following Clean Architecture principles with feature-based organization.
88  *
89  * **Architecture Benefits:**
90  * - **Unified Data Layer**: All API calls go through ORPC for consistency
91  * - **Type Safety**: End-to-end TypeScript inference from server to client
92  * - **Feature Organization**: Endpoints grouped by business domain (auth, todos, etc.)
93  * - **Clean Architecture**: Domain logic separated in feature modules (_api folders)
94  *
95  * **Better Auth Integration:**
96  * - Auth endpoints follow Better Auth conventions and terminology
97  * - Session management integrated with Better Auth context
98  * - Compatible with existing Better Auth patterns and middleware
99  */
100 export default {
101   /**
102    * Health check endpoint for monitoring
103    */
104   healthCheck: publicProcedure.handler() => {
105     return 'OK';
106   },
107   /**
108    * Authentication endpoints following Better Auth conventions
109    */
110   auth: {
111     getSession,
112   },
113   /**
114    * Protected data endpoint demonstrating auth-required functionality
115    */
116   privateData: protectedProcedure.handler(({ context }) => {
117     return {
118       message: 'This is private',
119       user: context.session?.user,
120     };
121   }),
122   /**
123    * Todo feature endpoints organized by domain
124    */
125   todos: {
126     getAll: getTodos,
127     create: createTodo,
128     toggle: toggleTodo,
129     toggle: toggleTodo,
130     toggle: toggleTodo,
131   },
132 }
```

Gambar 3.23 Struktur router oRPC utama



Gambar 3.6 Sequence diagram alur query dan mutation API

3.7 Authentication dan Authorization

3.7.1 Konfigurasi Better Auth

Better Auth dikonfigurasi menggunakan adapter Drizzle ORM untuk menyimpan data autentikasi ke PostgreSQL. Metode autentikasi yang digunakan adalah email dan password. Session dikelola secara server-side dengan cookie sebagai mekanisme penyimpanan token di sisi client.

3.7.2 Route Guard

Pembatasan akses pada antarmuka diterapkan melalui route guard pada file ``routes/admin/route.tsx``. Guard ini memeriksa:

1. Apakah pengguna memiliki session yang valid.
2. Apakah pengguna memiliki role ``admin``.

Route guard pada halaman peta (``/map``) memeriksa apakah pengguna memiliki role ``admin`` atau ``viewer``. Pengguna dengan role ``guest`` atau tanpa session tidak dapat mengakses halaman yang dilindungi.

3.7.3 Protected Procedure

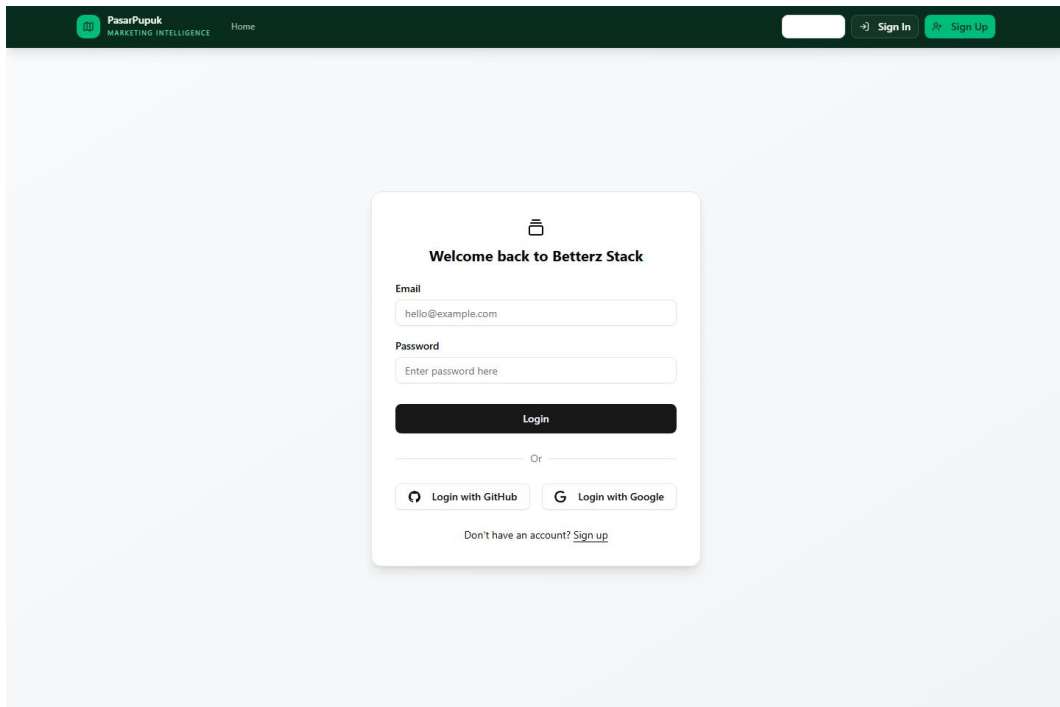
Protected procedure pada oRPC memastikan bahwa request yang masuk memiliki session pengguna yang valid. Namun, berdasarkan hasil analisis, validasi role admin pada lapisan API belum diterapkan secara eksplisit di seluruh endpoint. Protected procedure saat ini hanya memeriksa keberadaan session, bukan role pengguna.

3.7.4 Role Pengguna

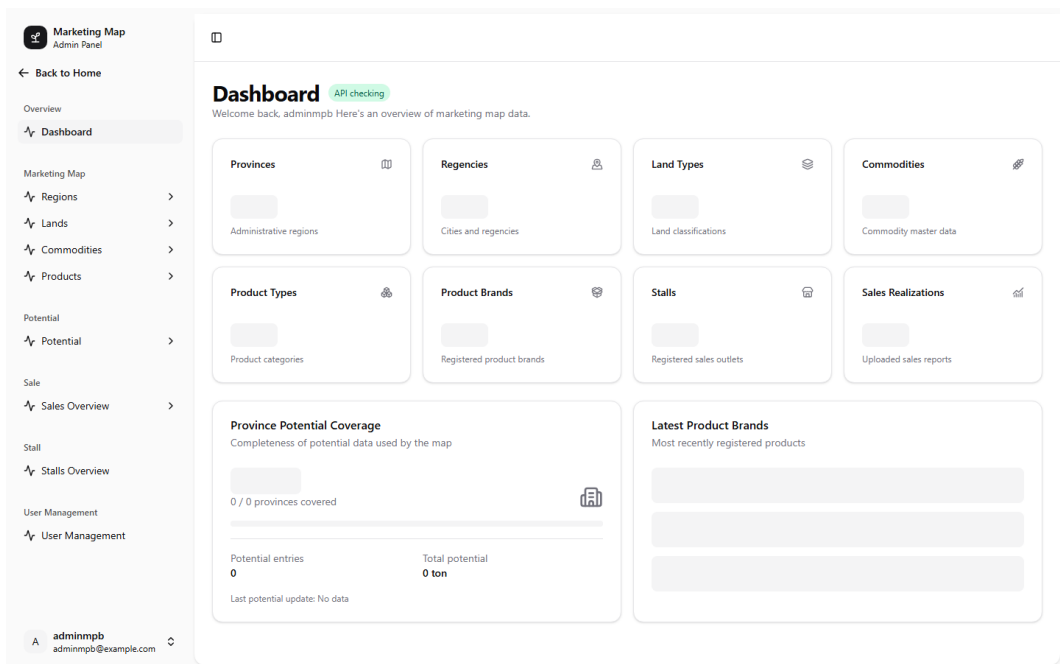
Role	Akses
Viewer	Akses terbatas ke halaman tertentu (misalnya peta). Tidak dapat mengakses panel admin.
Guest	Role default, hanya halaman publik.

3.7.5 Gap Authorization

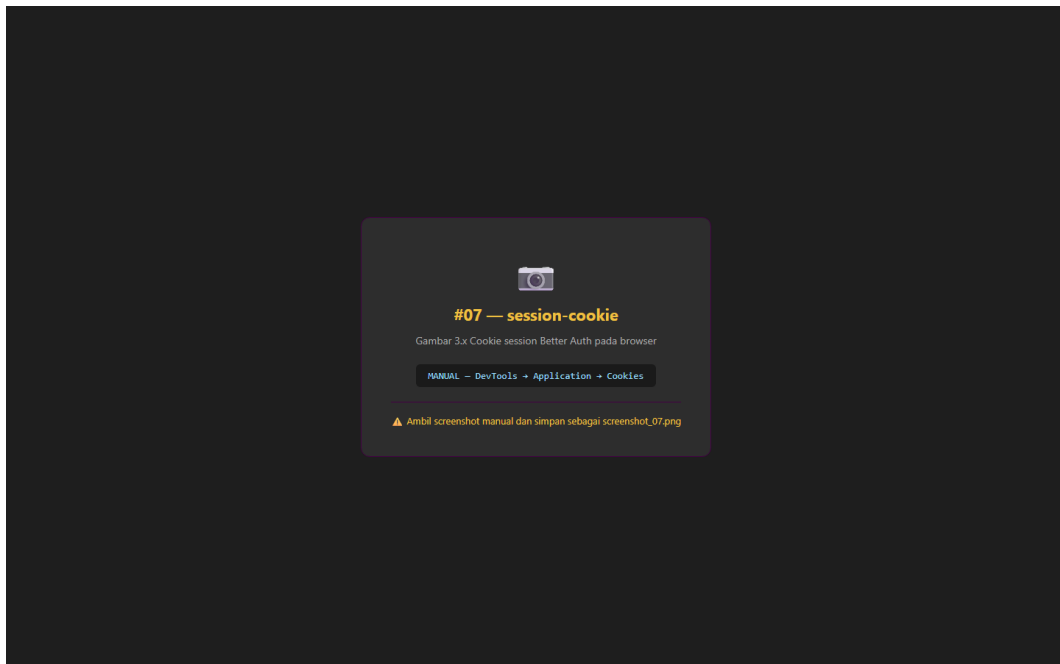
Hasil analisis menunjukkan bahwa mekanisme pembatasan akses telah diterapkan pada route antarmuka, tetapi protected procedure pada lapisan API belum memvalidasi role secara eksplisit. Kondisi ini menyebabkan API masih dapat diakses oleh pengguna dengan role viewer atau guest selama mereka memiliki session yang valid. Penambahan ``adminProcedure`` yang memvalidasi role ``admin`` pada setiap protected endpoint direkomendasikan untuk menutup celah ini.



Gambar 3.24 Halaman login pengguna



Gambar 3.25 Halaman admin setelah login berhasil



Gambar 3.26 Cookie session Better Auth pada browser

```

apps/web/src/routes/admin/route.tsx  typescript · 61 lines

1  import { createFileRoute, Outlet, redirect } from '@tanstack/react-router';
2  import { useSetAtom } from 'jotai';
3  import { useEffect } from 'react';
4  import { orpc } from '@lib/orpc/client';
5  import { AdminLayout } from './components/admin-layout';
6  import { currentUserAtom } from './libs/admin-atoms';
7
8  export const Route = createFileRoute('/admin')({
9    beforeLoad: async ({ context }) => {
10     // Protect admin routes - require authentication
11     if (!context.user) {
12       throw redirect({
13         to: '/auth/login',
14         search: (prev) => ({
15           ...prev,
16           redirect: '/admin',
17         })),
18       });
19     }
20     try {
21       const res = await orpc.admin.user.getById.call({
22         userId: context.user.id,
23       });
24       const isAdmin = res.data?.some((r) => r.role === 'admin');
25       if (!isAdmin) {
26         throw redirect({ to: '/' });
27       }
28     } catch {
29       throw redirect({
30         to: '/',
31       });
32     }
33   },
34   component: AdminLayoutRoute,
35 });
36
37 function AdminLayoutRoute() {
38   const { user } = Route.useRouteContext();
39   const setCurrentUser = useSetAtom(currentUserAtom);
40
41   // Sync user to atom for use in components
42   useEffect(() => {
43     if (user) {
44       setCurrentUser({
45         id: user.id,
46         name: user.name,
47         email: user.email,
48         role: 'admin',
49         avatar: user.image ?? undefined,
50       });
51     }
52   }, [user, setCurrentUser]);
53
54   return (
55     <AdminLayout>
56     <Outlet />
57     </AdminLayout>
58   );
59 }
60
61

```

Gambar 3.27 Implementasi route guard pada halaman admin

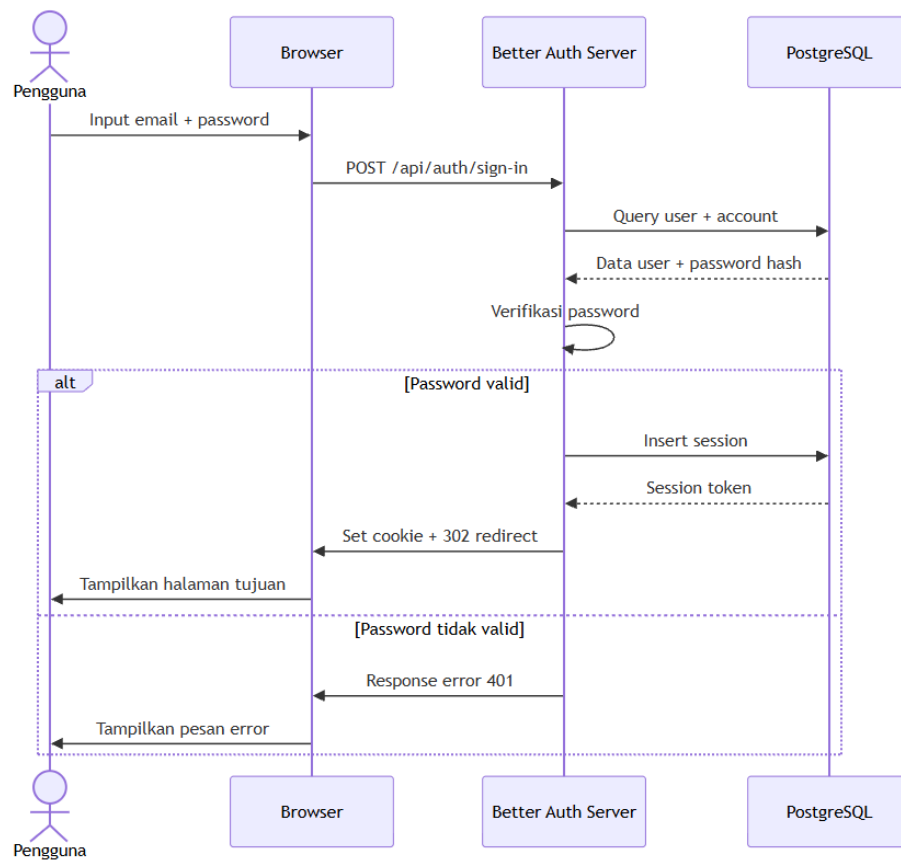

```

apps/web/src/lib/auth/index.ts typescript · 62 lines

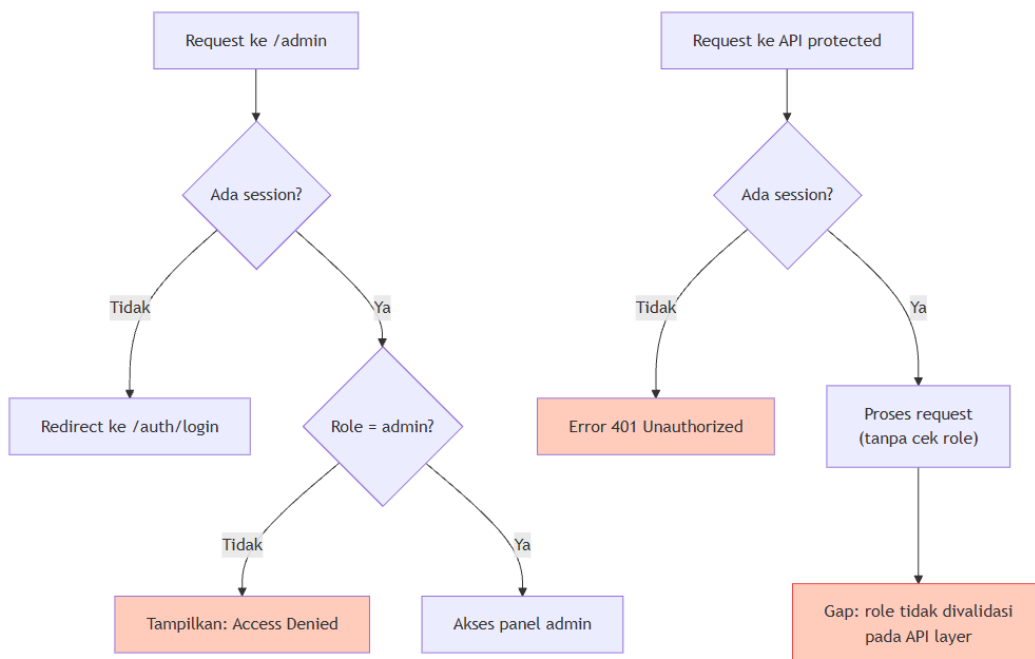
1  import { serverOnly } from 'tanstack/react-start';
2  import { betterAuth } from 'better-auth';
3  import { drizzleAdapter } from 'better-auth/adapters/drizzle';
4  import { reactStartCookies } from 'better-auth/react-start';
5  import { env } from '../../env/server';
6  import { db } from '../../db';
7  import { generateUUID } from '../../db/schema';
8  import { account, session, user, verification } from '../../db/schema/auth';
9
10 const getAuthConfig = serverOnly() => {
11   return betterAuth({
12     baseURL: env.BETTER_AUTH_URL || 'http://localhost:3000',
13     database: drizzleAdapter(db, {
14       provider: 'pg',
15       schema: {
16         user,
17         session,
18         account,
19         verification,
20       },
21     }),
22     // https://www.better-auth.com/docs/integrations/tanstack#usage-tips
23     plugins: [reactStartCookies()],
24
25     // https://www.better-auth.com/docs/concepts/session-management#session-caching
26     session: {
27       // cookieCache: {
28       //   enabled: true,
29       //   maxAge: 5 * 60, // 5 minutes
30       // },
31     },
32
33     // https://www.better-auth.com/docs/concepts/oauth
34     socialProviders: {
35       // github: {
36       //   clientId: env.GITHUB_CLIENT_ID,
37       //   clientSecret: env.GITHUB_CLIENT_SECRET,
38       // },
39       // google: {
40       //   clientId: env.GOOGLE_CLIENT_ID,
41       //   clientSecret: env.GOOGLE_CLIENT_SECRET,
42       // },
43     },
44
45     // https://www.better-auth.com/docs/authentication/email-password
46     emailAndPassword: {
47       enabled: true,
48     },
49
50     trustedOrigins: [env.CORS_ORIGIN || ''],
51     secret: env.BETTER_AUTH_SECRET,
52     advanced: {
53       database: {
54         generateId: generateUUID,
55       },
56     },
57   });
58 };
59
60 export const auth = getAuthConfig();
61
62

```

Gambar 3.28 Konfigurasi Better Auth dengan adapter Drizzle



Gambar 3.4 Sequence diagram alur login pengguna



Gambar 3.5 Diagram authentication dan route guard

3.8 Hasil Implementasi Modul Bisnis

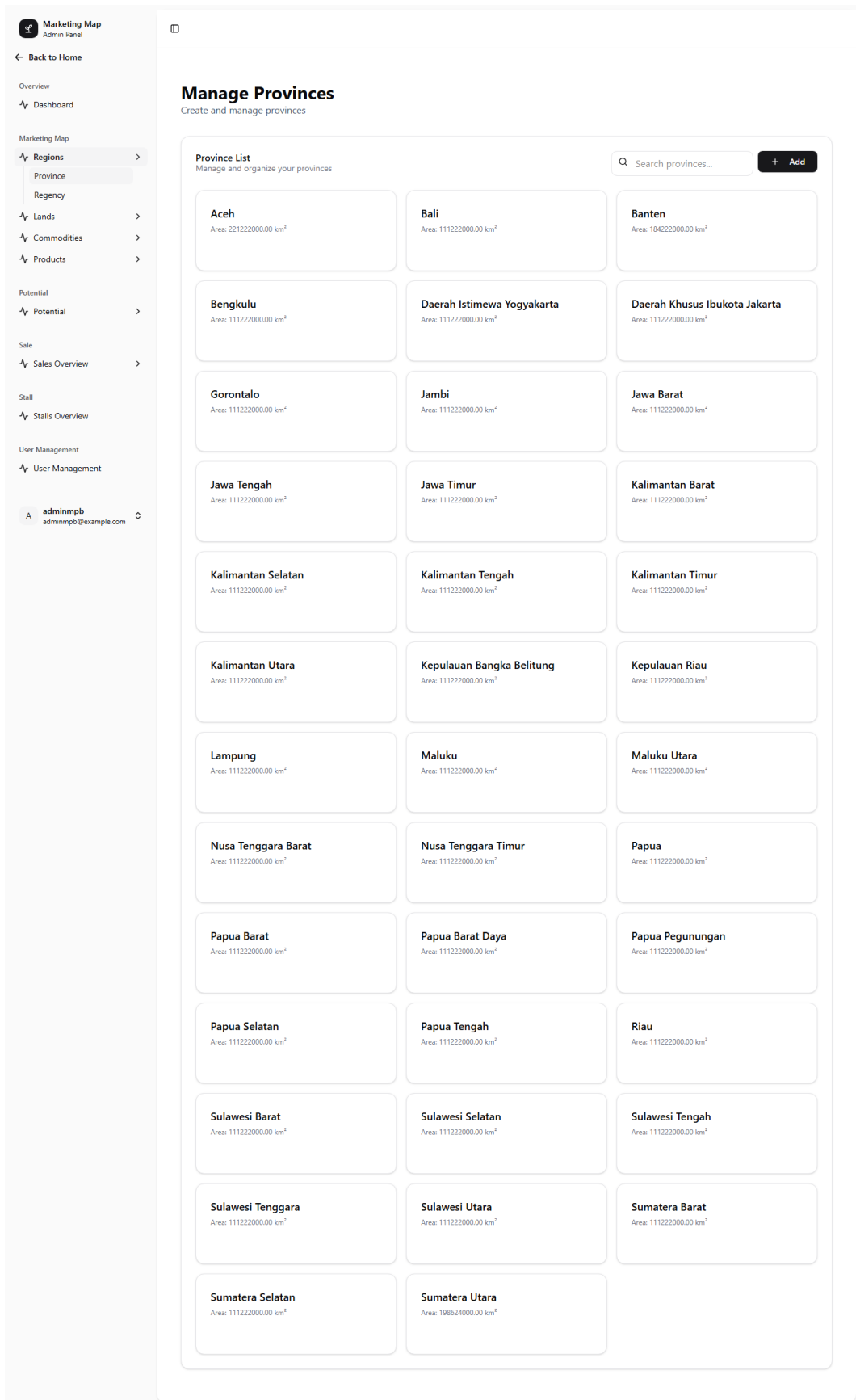
3.8.1 Modul Wilayah (Province dan Regency)

Modul Wilayah digunakan untuk mengelola data master provinsi dan kabupaten yang menjadi referensi geografis bagi seluruh modul lain dalam sistem.

Province menyediakan operasi CRUD untuk data provinsi yang meliputi kode provinsi, nama provinsi, luas wilayah, dan tahun data. Fitur yang tersedia meliputi pencarian berdasarkan nama, pagination, serta validasi kode provinsi yang unik.

Regency menyediakan operasi CRUD untuk data kabupaten atau kota yang terhubung dengan provinsi induk melalui foreign key. Setiap regency mencatat kode wilayah, nama, provinsi induk, luas, dan tahun. Fitur pencarian dan pagination juga tersedia pada modul ini.

Relasi antara province dan regency bersifat one-to-many, di mana satu provinsi dapat memiliki banyak kabupaten atau kota. Validasi memastikan bahwa setiap regency hanya dapat dihubungkan dengan satu provinsi yang valid.



Gambar 3.29 Daftar data provinsi pada panel admin

Marketing Map
Admin Panel

← Back to Home

Overview

Dashboard

Marketing Map

Regions

Province

Regency

Lands

Commodities

Products

Potential

Potential

Sale

Sales Overview

Stall

Stalls Overview

User Management

User Management

adminmpb
adminmpb@example.com

Manage Provinces

Create and manage provinces

Province List

Manage and organize your provinces

Q Search provinces...

+ Add

Aceh

Area: 221222000.00 km²

Bengkulu

Area: 111222000.00 km²

Gorontalo

Area: 111222000.00 km²

Jawa Tengah

Area: 111222000.00 km²

Banten

Area: 184222000.00 km²

Daerah Khusus Ibukota Jakarta

Area: 111222000.00 km²

Jawa Barat

Area: 111222000.00 km²

Kalimantan Barat

Area: 111222000.00 km²

Jambi

Area: 111222000.00 km²

Jawa Timur

Area: 111222000.00 km²

Kalimantan Selatan

Area: 111222000.00 km²

Kalimantan Tengah

Area: 111222000.00 km²

Kalimantan Timur

Area: 111222000.00 km²

Kalimantan Utara

Area: 111222000.00 km²

Kepulauan Bangka Belitung

Area: 111222000.00 km²

Kepulauan Riau

Area: 111222000.00 km²

Lampung

Area: 111222000.00 km²

Maluku

Area: 111222000.00 km²

Maluku Utara

Area: 111222000.00 km²

Nusa Tenggara Barat

Area: 111222000.00 km²

Nusa Tenggara Timur

Area: 111222000.00 km²

Papua

Area: 111222000.00 km²

Papua Barat

Area: 111222000.00 km²

Papua Barat Daya

Area: 111222000.00 km²

Papua Pegunungan

Area: 111222000.00 km²

Papua Selatan

Area: 111222000.00 km²

Papua Tengah

Area: 111222000.00 km²

Riau

Area: 111222000.00 km²

Sulawesi Barat

Area: 111222000.00 km²

Sulawesi Selatan

Area: 111222000.00 km²

Sulawesi Tengah

Area: 111222000.00 km²

Sulawesi Tenggara

Area: 111222000.00 km²

Sulawesi Utara

Area: 111222000.00 km²

Sumatera Barat

Area: 111222000.00 km²

Sumatera Selatan

Area: 111222000.00 km²

Sumatera Utara

Area: 198624000.00 km²

Create New Province

Add a new province

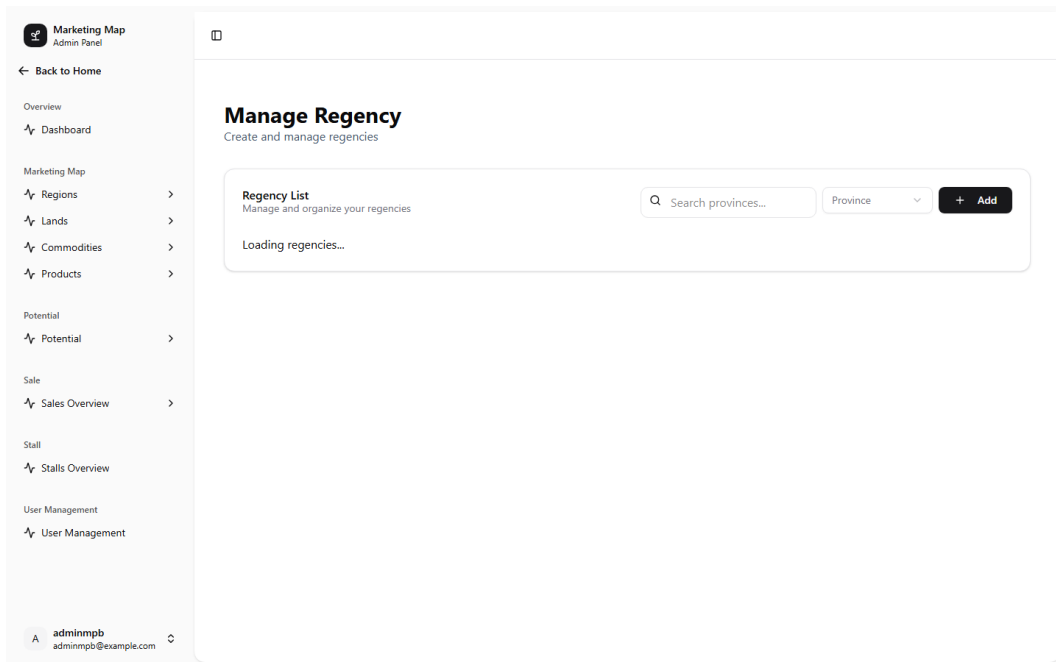
Province Code

Province Name

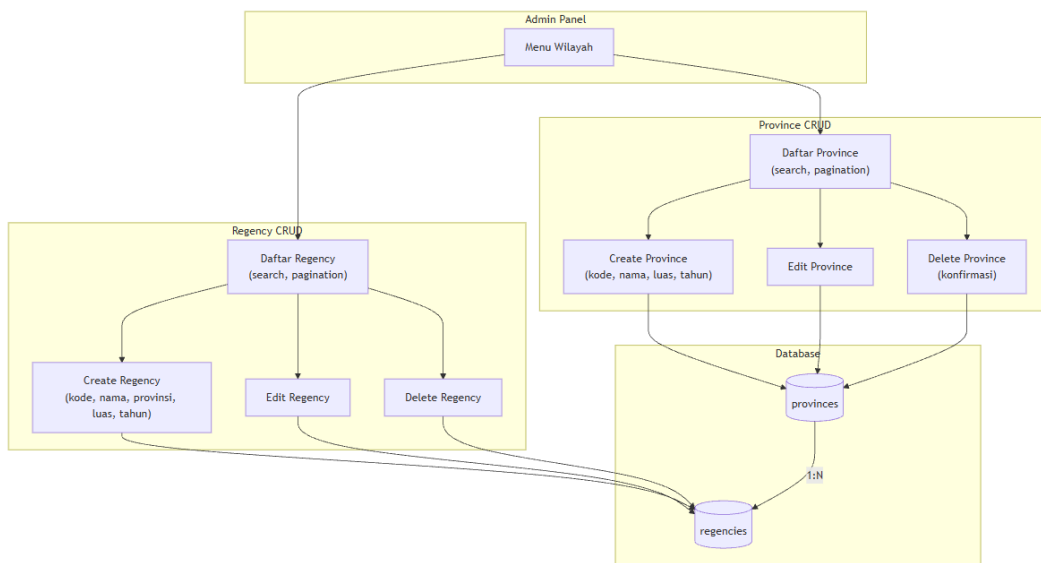
Area

Create

Gambar 3.30 Form penambahan data provinsi



Gambar 3.31 Daftar data kabupaten dengan relasi provinsi



Gambar 3.7 Diagram alur CRUD modul wilayah

3.8.2 Modul Lahan (Land Type, Province Land, Regency Land)

Modul Lahan digunakan untuk mengelola data jenis lahan dan luas lahan per wilayah.

Land Type merupakan data master jenis lahan yang mencakup nama jenis lahan dan tahun. Modul ini menyediakan operasi CRUD penuh.

Province Land menyediakan CRUD untuk data luas lahan pada tingkat provinsi,

mencakup provinsi, jenis lahan, luas, dan tahun. Data ini memungkinkan analisis komposisi lahan per provinsi.

Regency Land saat ini berfokus pada pembacaan data (read-only). Operasi create, update, dan delete pada tingkat kabupaten belum diaktifkan pada router. Modul ini menjadi salah satu area yang dapat dikembangkan lebih lanjut.

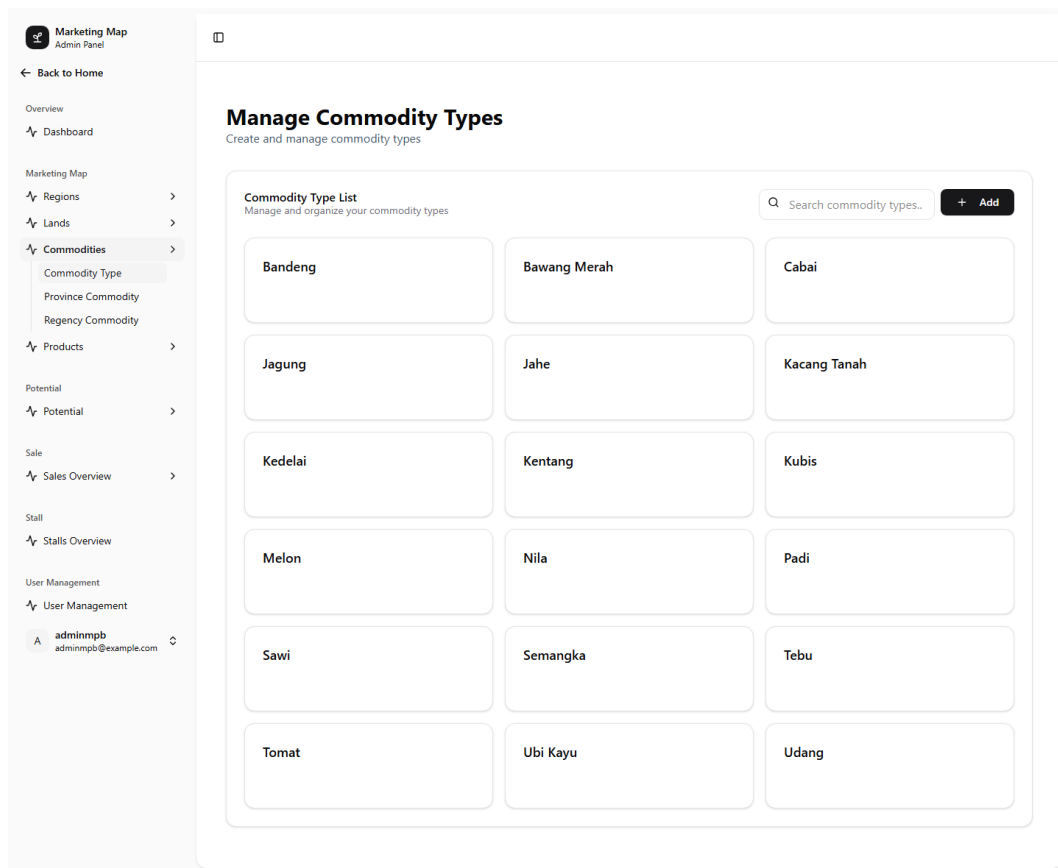
3.8.3 Modul Komoditas (Commodity Type, Province Commodity, Regency Commodity)

Modul Komoditas digunakan untuk mengelola data jenis komoditas pertanian dan hubungannya dengan wilayah.

Commodity Type merupakan data master komoditas yang terhubung dengan jenis lahan melalui foreign key. Setiap komoditas mencatat nama dan tahun. Modul menyediakan CRUD penuh dengan fitur pencarian dan filter berdasarkan jenis lahan.

Province Commodity menyajikan data komoditas pada tingkat provinsi. Informasi yang ditampilkan diperoleh melalui relasi antara provinsi dan jenis komoditas, disertai data luas serta tahun. Berdasarkan hasil analisis, fungsi backend yang aktif berfokus pada pengambilan, pencarian, dan penyaringan data, sedangkan operasi penambahan, perubahan, dan penghapusan belum diaktifkan pada router.

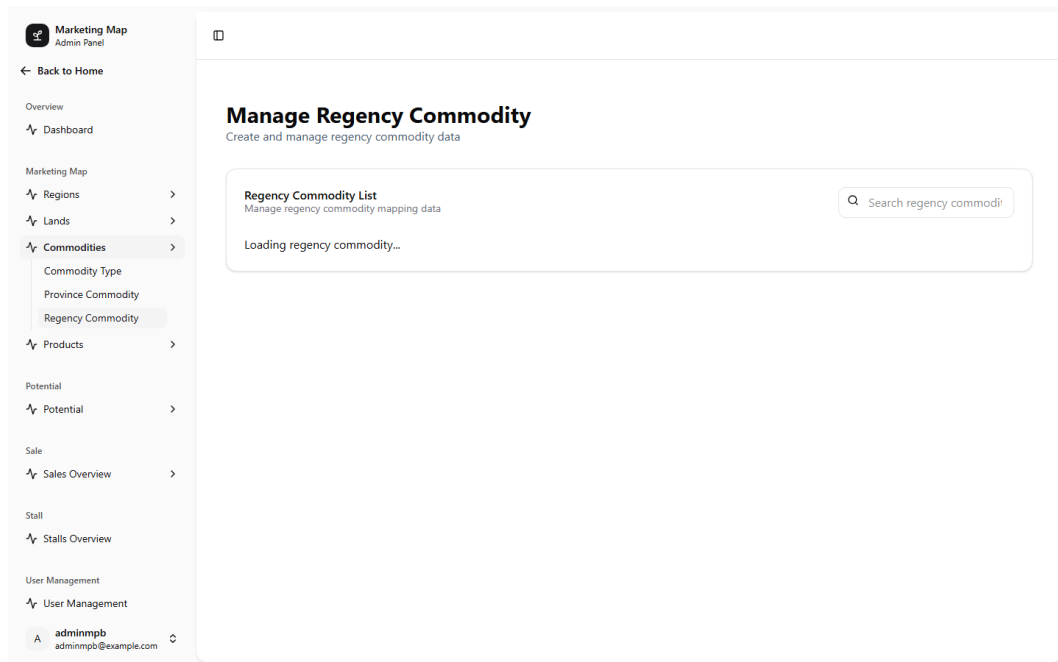
Regency Commodity telah menyediakan operasi CRUD yang lebih lengkap. Modul ini menghubungkan kabupaten atau kota dengan jenis komoditas dan menyimpan informasi luas serta tahun. Perbedaan tingkat kesiapan antara Province Commodity (read-only) dan Regency Commodity (CRUD penuh) menunjukkan bahwa pengembangan pada kedua tingkat wilayah dilakukan dengan prioritas yang berbeda.



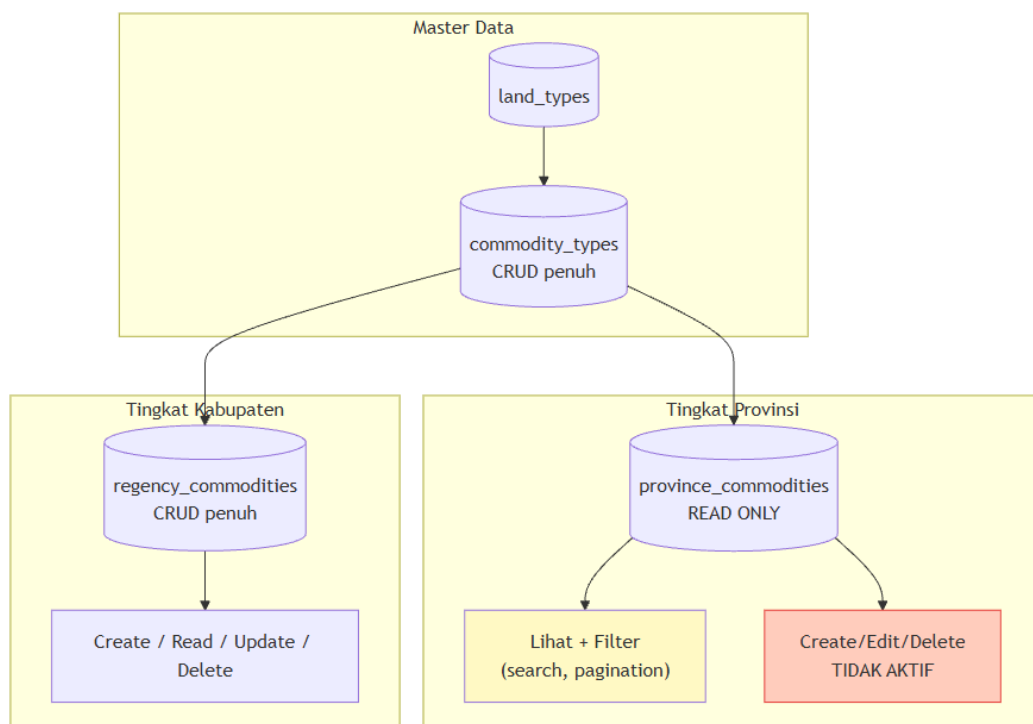
Gambar 3.32 Daftar jenis komoditas dengan filter jenis lahan

Sumatera Selatan	Sawi	0		
Sumatera Selatan	Padi	502162		
Sumatera Selatan	Jahe	0		
Sumatera Selatan	Bandeng	0		
Sumatera Selatan	Bawang Merah	165		
Sumatera Selatan	Kacang Tanah	1012		
Sumatera Selatan	Kentang	91		
Sumatera Selatan	Semangka	0		
Sumatera Selatan	Nila	0		
Sumatera Selatan	Kedelai	33.3		
Sumatera Selatan	Tomat	770		
Sumatera Selatan	Melon	0		
Sumatera Selatan	Jagung	132582		
Sumatera Selatan	Cabai	822		
Sumatera Selatan	Tebu	6614		
Sumatera Utara	Tebu	0		
Sumatera Utara	Tomat	6401		
Sumatera Utara	Cabai	741		
Sumatera Utara	Bawang Merah	4293		
Sumatera Utara	Kedelai	9834		
Sumatera Utara	Nila	0		
Sumatera Utara	Sawi	6565		
Sumatera Utara	Semangka	3971		
Sumatera Utara	Kentang	7210		
Sumatera Utara	Padi	404473		
Sumatera Utara	Kacang Tanah	5926		
Sumatera Utara	Bandeng	0		
Sumatera Utara	Kubis	8304		
Sumatera Utara	Jahe	0		
Sumatera Utara	Ubi Kayu	29243		
Sumatera Utara	Udang	0		
Sumatera Utara	Jagung	304821		
Sumatera Utara	Melon	178		

Gambar 3.33 Data komoditas tingkat provinsi (read-only)



Gambar 3.34 Data komoditas tingkat kabupaten dengan CRUD penuh



Gambar 3.8 Diagram modul komoditas dengan status read-only

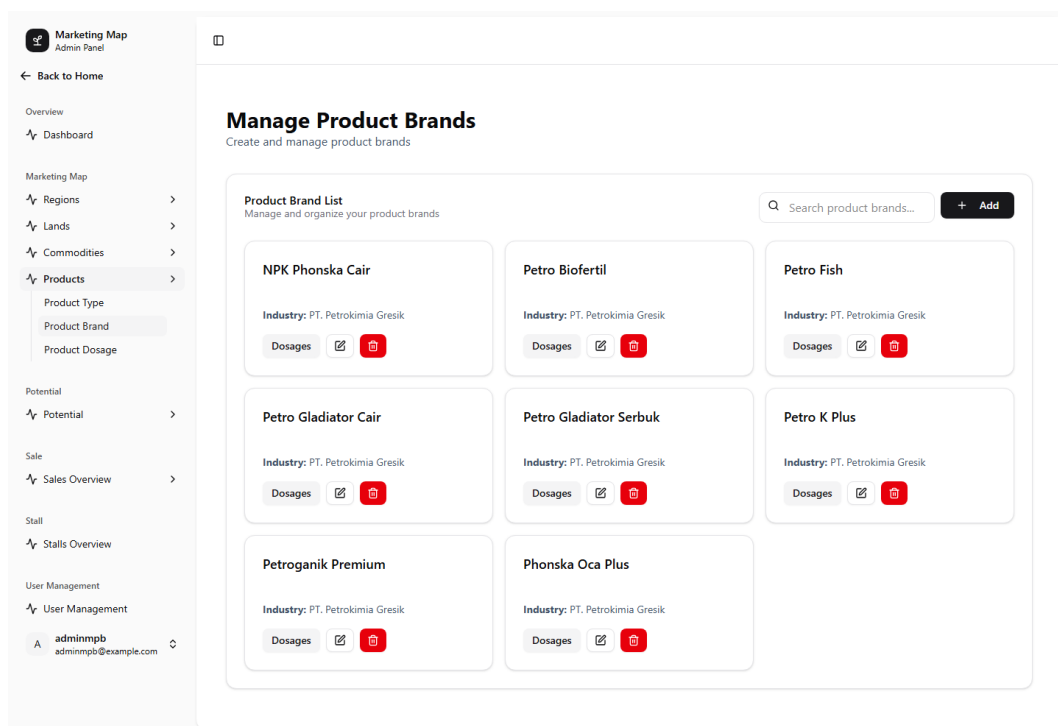
3.8.4 Modul Produk (Product Type, Product Brand, Product Dosage)

Modul Produk digunakan untuk mengelola data produk dengan hierarki tiga tingkat: jenis produk, brand produk, dan dosis produk.

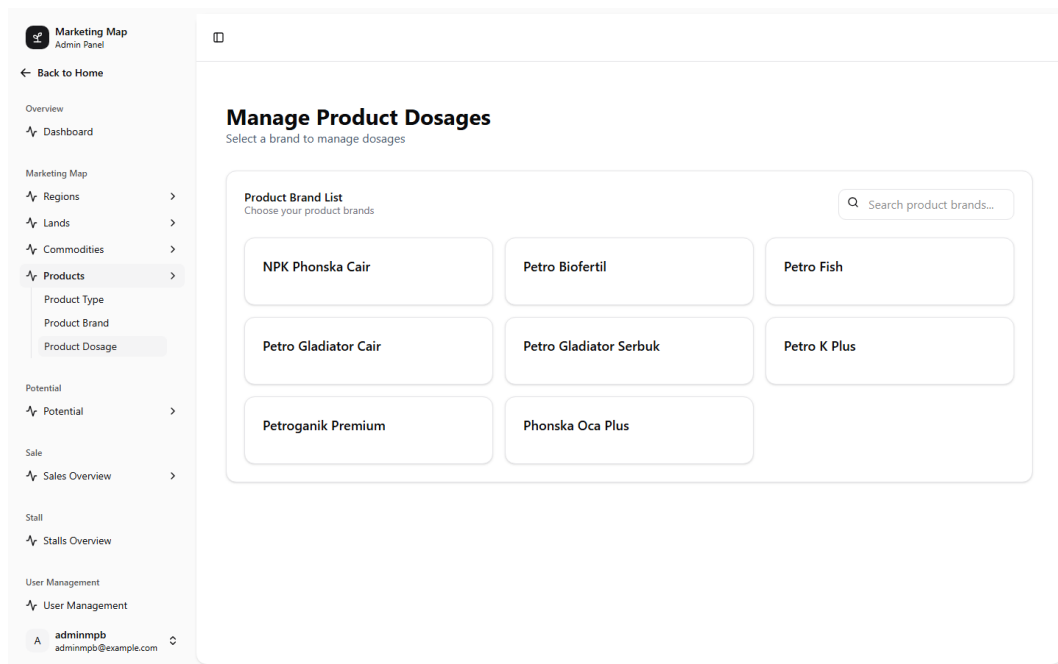
Product Type merupakan data master jenis produk yang mencakup nama, deskripsi, dan tahun. Modul ini menyediakan CRUD penuh dengan penanganan konflik duplikasi data.

Product Brand merupakan data master brand produk yang berada di bawah Product Type. Setiap brand mencatat nama, industri, dan deskripsi. Product Brand memiliki peran sentral karena direferensikan oleh banyak modul lain, termasuk Product Dosage, Province Potential, Sales Realization, Daily Sales, dan Stall. Modul ini menyediakan CRUD penuh dengan fitur pencarian dan filter berdasarkan Product Type.

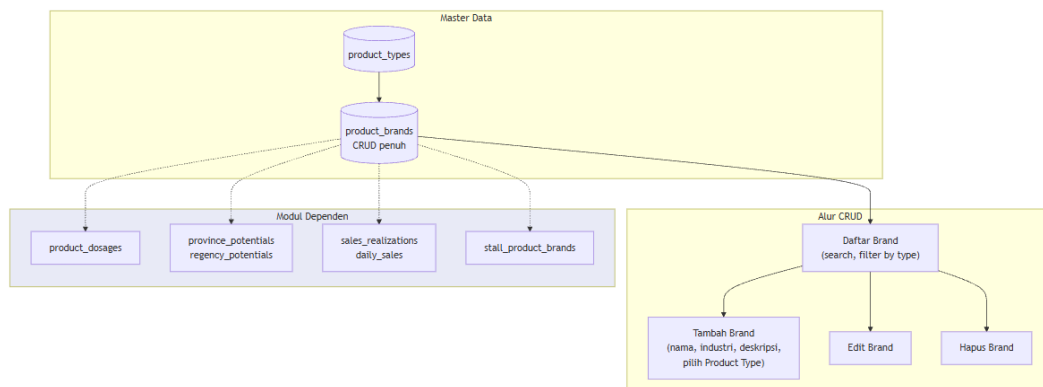
Product Dosage digunakan untuk mencatat dosis penggunaan suatu brand produk pada jenis komoditas tertentu. Data yang dikelola meliputi komoditas, brand produk, nilai dosis, satuan, dan tahun. Pada lapisan backend, modul menyediakan operasi pengambilan, penambahan, perubahan, dan penghapusan data yang terhubung dengan tabel `commodity\_types` dan `product\_brands`.



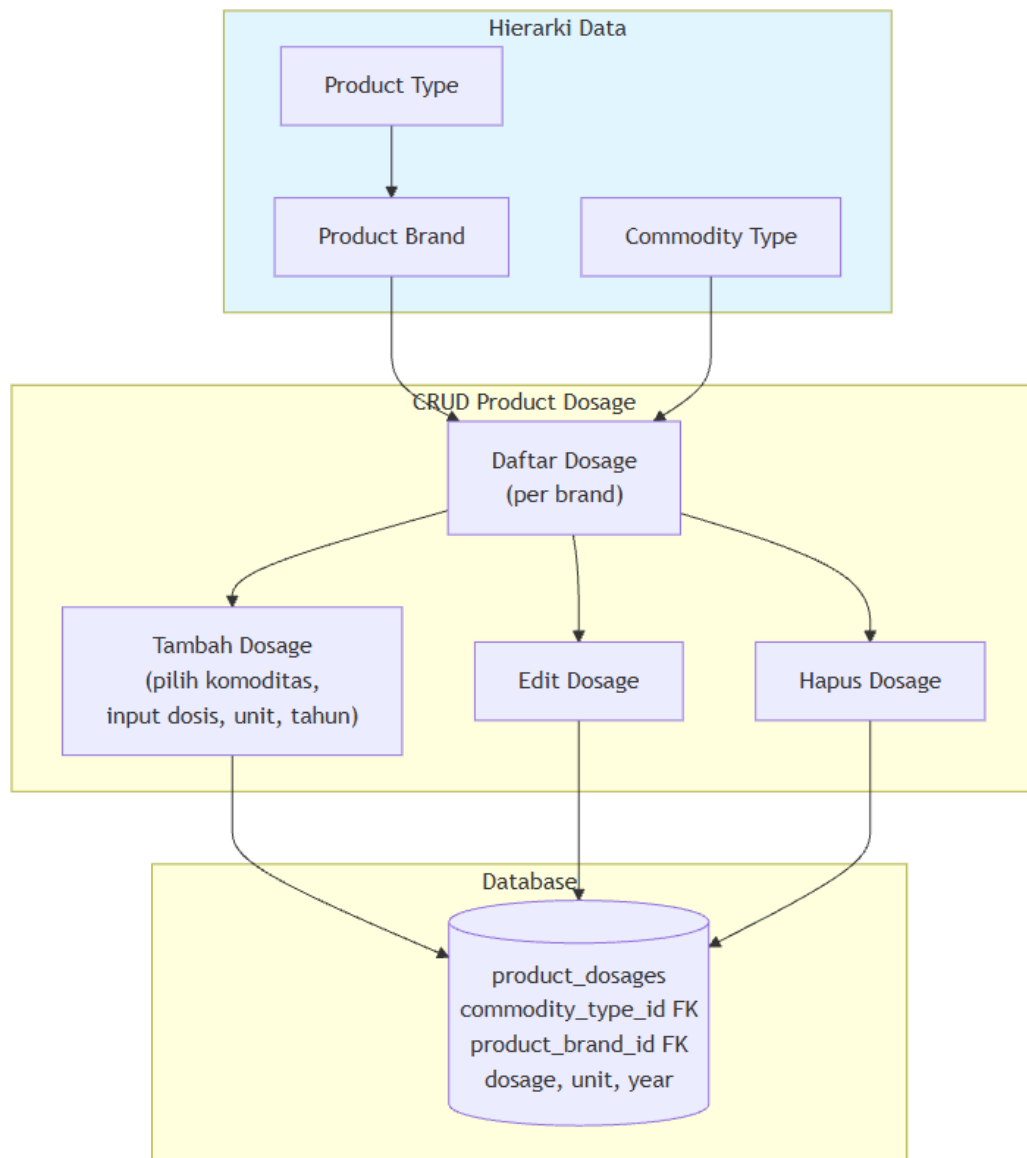
Gambar 3.35 Daftar brand produk yang direferensikan oleh seluruh modul



Gambar 3.36 Daftar dosis produk untuk setiap brand dan komoditas



Gambar 3.9 Diagram alur CRUD Product Brand dan modul dependen



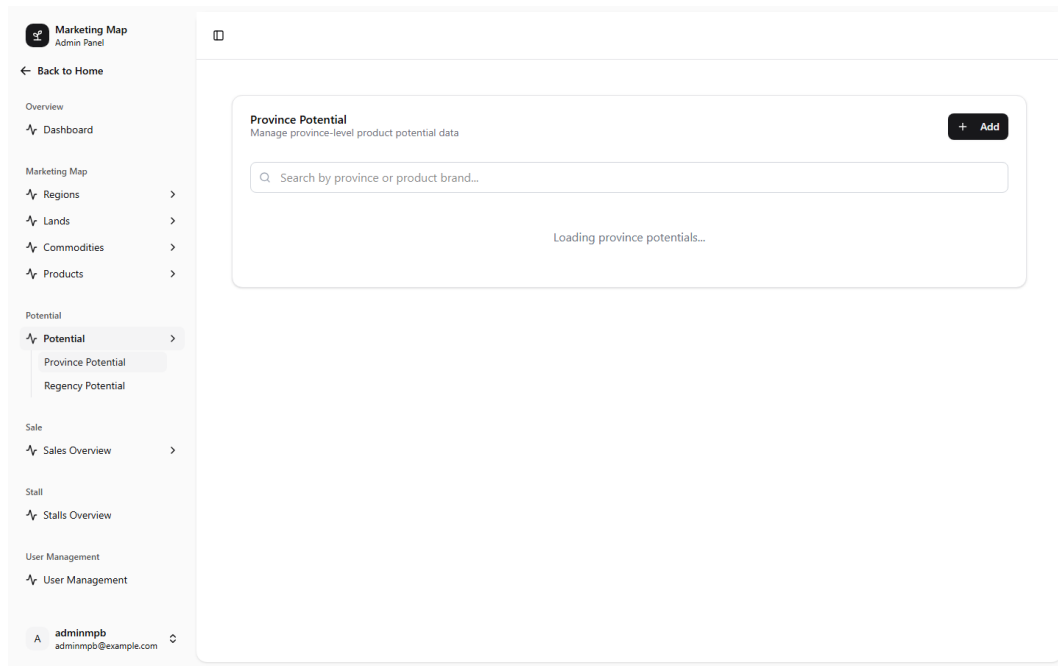
Gambar 3.10 Diagram alur CRUD Product Dosage

3.8.5 Modul Potensi (Province Potential, Regency Potential)

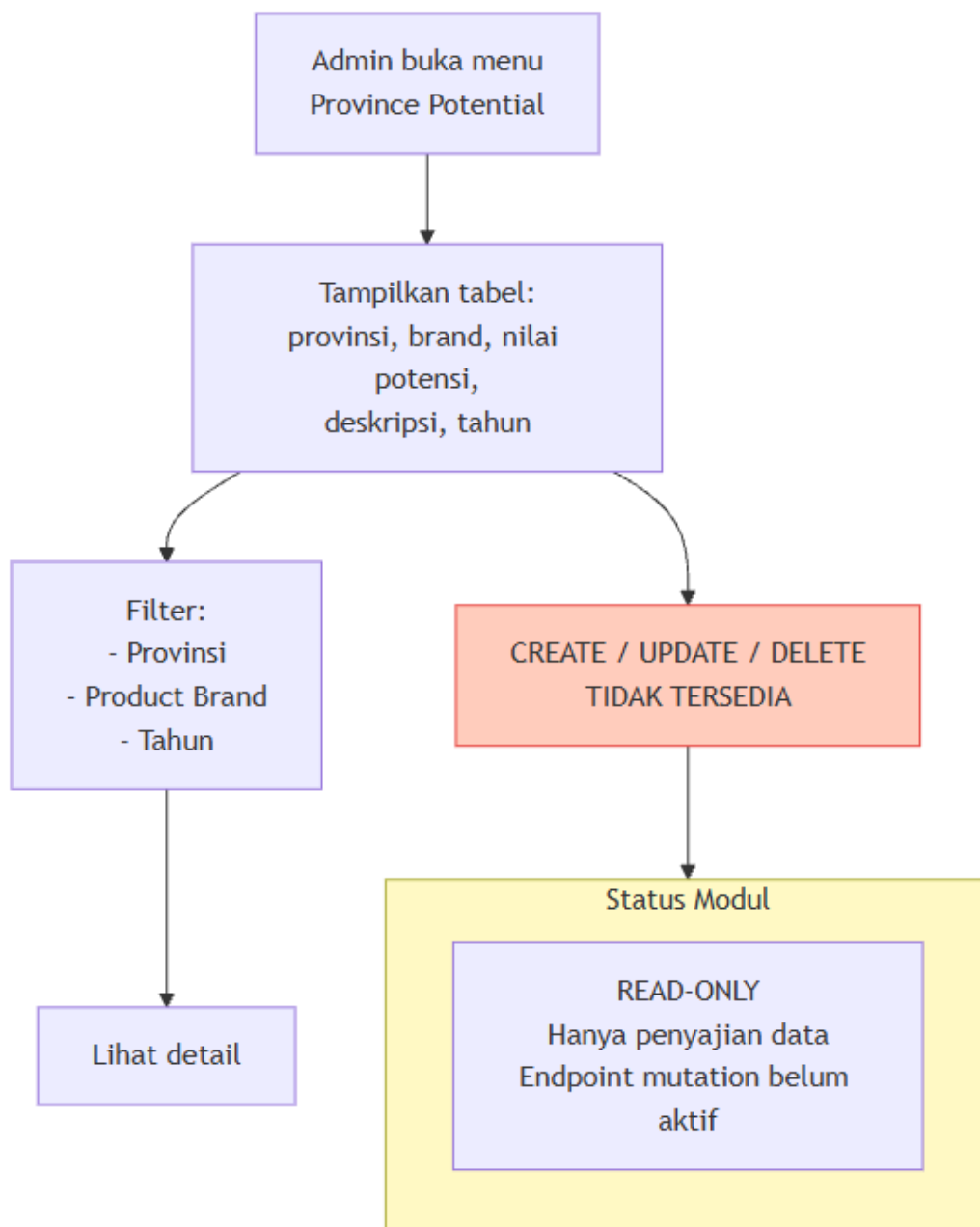
Modul Potensi digunakan untuk menyajikan data potensi pasar suatu brand produk pada tingkat wilayah.

Province Potential menyajikan informasi potensi suatu brand produk pada tingkat provinsi. Backend menggabungkan data provinsi dan Product Brand serta menyediakan filter berdasarkan wilayah, produk, dan tahun. Hasil observasi menunjukkan bahwa fungsi aktif masih berfokus pada penyajian data (read-only). Operasi penambahan, perubahan, dan penghapusan belum diaktifkan pada router.

Regency Potential memiliki tabel database yang telah tersedia, tetapi endpoint aktif belum ditemukan pada router yang dianalisis. Modul ini memerlukan verifikasi lebih lanjut untuk memastikan status implementasinya.



Gambar 3.37 Data potensi provinsi (read-only)



Gambar 3.11 Diagram penyajian data Province Potential (read-only)

3.8.6 Modul Penjualan (Sales Realization, Daily Sales)

Modul Penjualan digunakan untuk mencatat dan memantau realisasi penjualan produk.

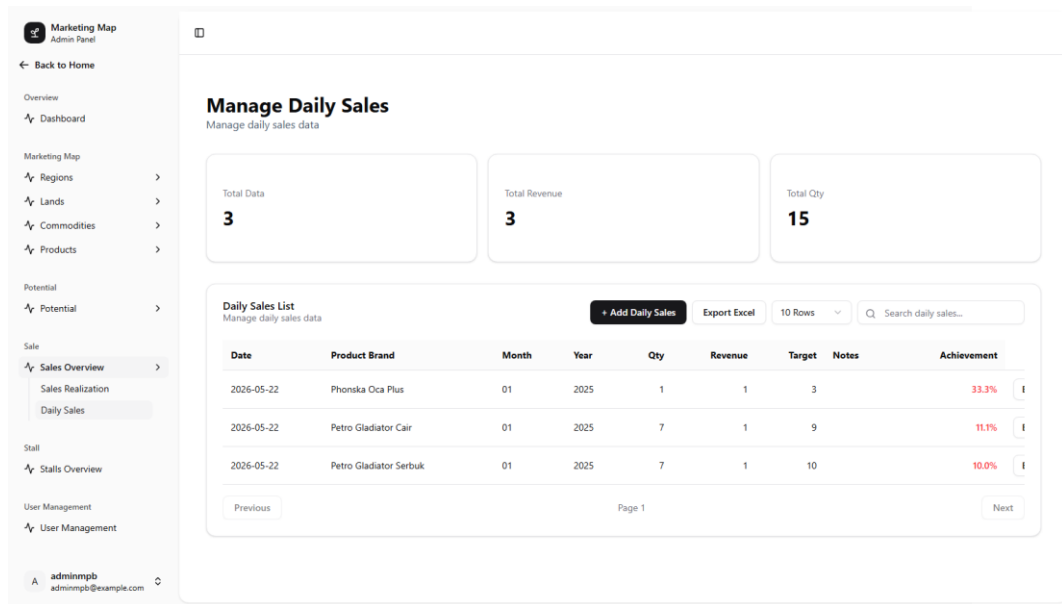
Sales Realization menyediakan CRUD untuk data realisasi dan RKAP (Rencana dan Anggaran Kegiatan) per Product Brand. Data yang dikelola meliputi tanggal laporan, brand produk, realisasi harian, bulanan, year-to-date (YTD), RKAP, dan pembandingan tahun sebelumnya. Modul dilengkapi fitur filter dan pagination. Hasil analisis menunjukkan

indikasi perbedaan penamaan field antara schema (`realization\_ytd`) dan migration (`realizaton\_ytd`) yang perlu diverifikasi terhadap database aktual.

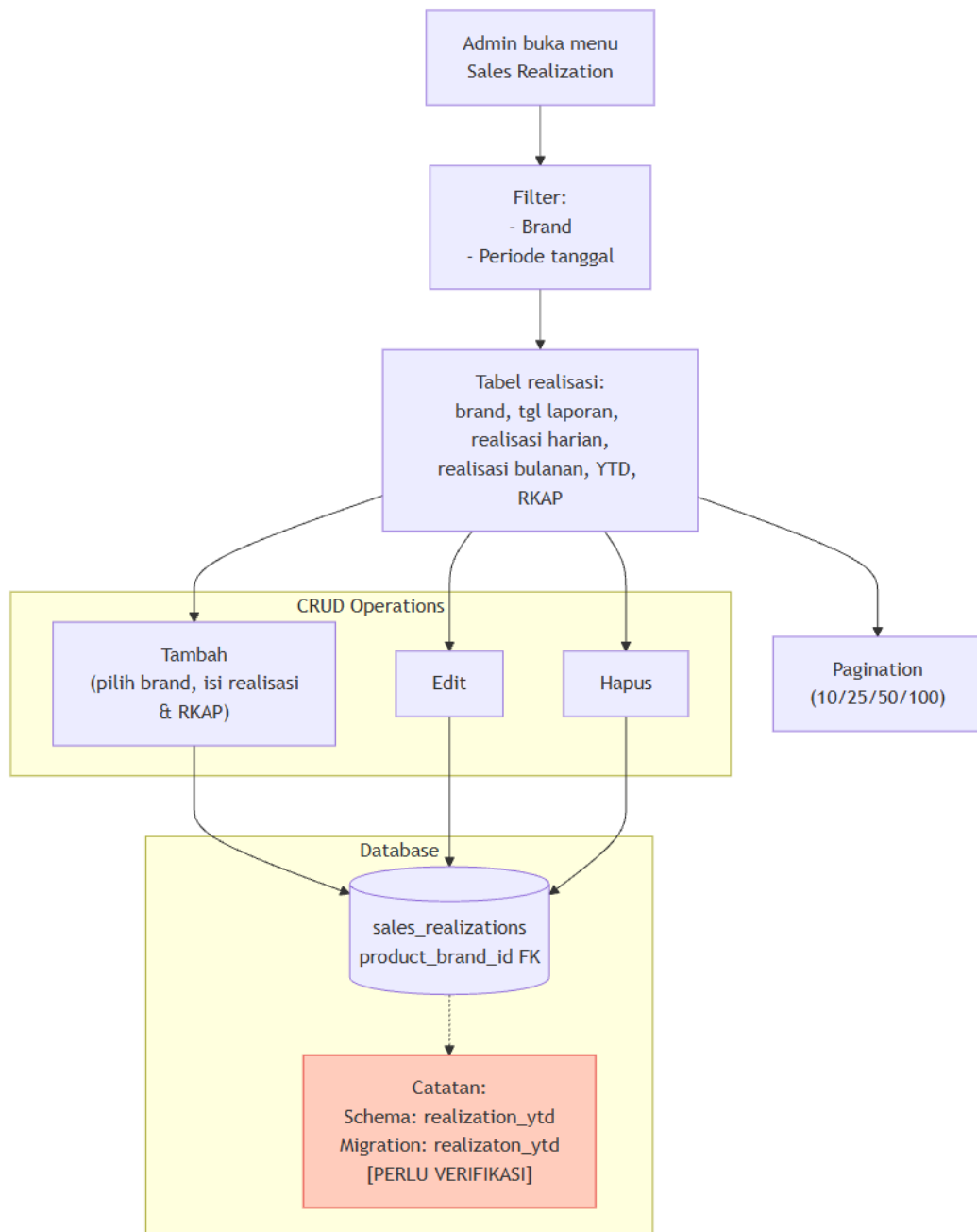
Daily Sales menyediakan CRUD untuk data penjualan harian yang mencakup tanggal, bulan, tahun, Product Brand, provinsi, kuantitas, pendapatan, target, catatan, dan realisasi. Setiap data penjualan harian terhubung dengan satu brand produk. Field `province\_id` telah tersedia untuk menyimpan konteks wilayah, tetapi definisi foreign key belum ditemukan pada schema yang dianalisis.

Product Brand	Month	Year	Realization Monthly	RKAP Monthly	Report Date	Daily	YTD	RKAP YTD	RKAP Yearly	Li
NPK Phonska Cair	01	2025	1	1	2026-05-20	0	1	1	1	1
Petro Fish	01	2026	2	2	2026-05-21	0	2	2	2	2

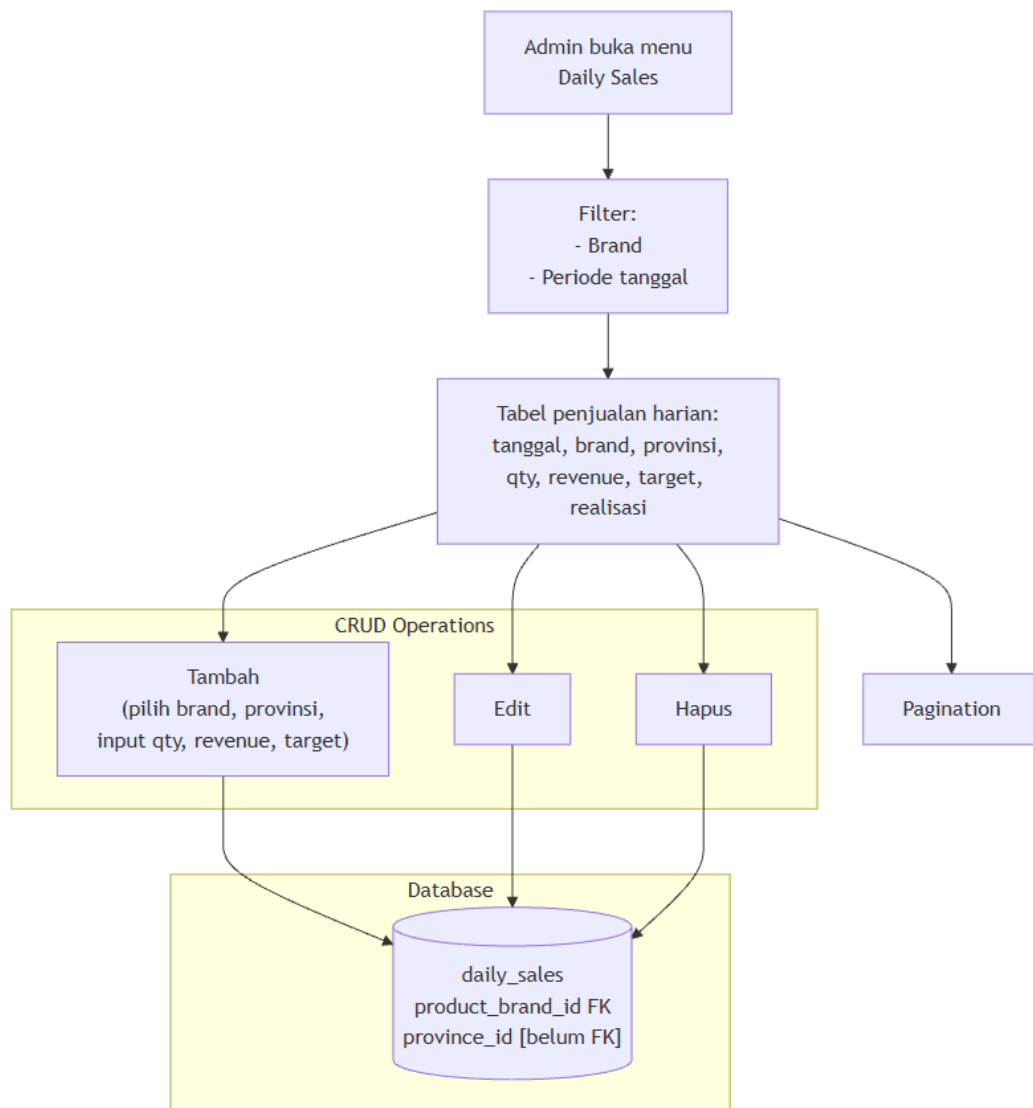
Gambar 3.38 Data realisasi penjualan dengan metrik RKAP dan YTD



Gambar 3.39 Data penjualan harian per brand produk



Gambar 3.12 Diagram alur CRUD Sales Realization



Gambar 3.13 Diagram alur CRUD Daily Sales

3.8.7 Modul Stall

Modul Stall digunakan untuk mengelola data kios yang mencakup nama kios, alamat, provinsi, kabupaten, koordinat geografis (latitude dan longitude), nama pemilik, nomor telepon, kriteria kios, dan tahun.

Setiap Stall berada pada satu provinsi dan satu kabupaten melalui relasi foreign key. Stall dapat terhubung dengan lebih dari satu Product Brand melalui tabel penghubung `stall\_product\_brands`, membentuk relasi many-to-many.

Modul ini menyediakan operasi CRUD lengkap, form pengelolaan data, serta fitur assignment Product Brand. Fitur assignment memungkinkan admin untuk memilih brand

produk yang tersedia pada masing-masing kios melalui antarmuka pengelolaan. Modul Stall juga mendukung import data dari file Excel melalui script `scripts/seed-stalls.ts` yang membaca data dari `data/SURVEY PASAR KIOS (Jawaban).xlsx`.

Modul Stall merupakan modul dengan bukti perubahan repository yang paling jelas. Commit `7a38bb7` menunjukkan penambahan dan perubahan signifikan pada modul ini, mencakup operasi create, update, delete, penyempurnaan form, integrasi procedure pada router oRPC, dan pengembangan fitur assignment Product Brand.

Marketing Map Admin Panel

← Back to Home

Overview

- Dashboard

Marketing Map

- Regions
- Lands
- Commodities
- Products

Potential

- Potential

Sale

- Sales Overview

Stall

- Stalls Overview**

User Management

- User Management

adminmpb
adminmpb@example.com

Manage Stalls

Manage kios and map distribution data

Total Stalls
164

Total Provinces
3

Total Regencies
29

Stall List
Manage kios and map distribution

[+ Add Stall](#) [Export Excel](#) 10 Rows

Name	Province	Regency	Owner	Phone	Latitude	Longitude	Criteria	Action
2 Putra Tani	Jawa Timur	Kabupaten Banyuwangi	Pak Husni Mubarak	081234849877	-8.28816	114.317	Kios	Edit
Abi Tani	Jawa Timur	Kabupaten Magetan			-7.65536	111.28		Edit
Abi Tani Jetak	Jawa Tengah	Kabupaten Sragen	Bu Meyda	082328181015	-7.44099	110.981	Kios	Edit
Abi Tani Ngrampal	Jawa Tengah	Kabupaten Sragen	Vira	085743041353	-7.39647	111.067	Kios	Edit
Agro Makmur	Jawa Tengah	Kabupaten Temanggung	Syarifudin	081227451316	-7.28869	110.182	Kios	Edit
Aji Tani	Jawa Timur	Kabupaten Blitar	Yatno	085790214611	-8.1203	112.195	Kios	Edit
Akar Tani	Jawa Timur	Kabupaten Magetan			-7.68108	111.237		Edit
Akar Tani	Jawa Timur	Kabupaten Magetan	Saifuddin	081237378063	-7.68107	111.237	Kios	Edit
Anugerah Subur	Jawa Timur	Kabupaten Mojokerto	Ibu Aam	081357615836	-7.52794	112.594	Kios	Edit
Anugerah Tani Makmur	Jawa Timur	Kabupaten Mojokerto	Pak Handoyo	081235230177	-7.52456	112.414	Kios	Edit

Previous Page 1 Next

Gambar 3.40 Daftar kios pada modul Stall

Marketing Map

Admin Panel

Back to Home

Overview

Dashboard

Marketing Map

Regions

Lands

Commodities

Products

Potential

Potential

Sale

Sales Overview

Stall

Stalls Overview

User Management

User Management

adminmpb

adminmpb@example.com

Manage Stalls

Manage kios and map distribution data

Total Stalls

164

Total Regencies

29

Stall List

Manage kios and map distribution

+ Add Stall

Export Excel

10 Rows

Q Search stall...

Name	Province	Regency	Owner	Phone	Latitude	Longitude	Criteria	Action
2 Putra Tani	Jawa Timur	Kabupaten Banyuwangi	Pak Husni Mubarak	081234849877	-8.28816	114.317	Kios	Edit
Abi Tani	Jawa Timur	Kabupaten Magetan			-7.65536	111.28		Edit
Abi Tani Jetak	Jawa Tengah	Kabupaten Sragen	Bu Meyda	082328181015	-7.44099	110.981	Kios	Edit
Abi Tani Ngrampal	Jawa Tengah	Kabupaten Sragen	Vira	085743041353	-7.39647	111.067	Kios	Edit
Agro Makmur	Jawa Tengah	Kabupaten Temanggung	Syarifudin	081227451316	-7.28869	110.182	Kios	Edit
Aji Tani	Jawa Timur	Kabupaten Blitar	Yatno	085790214611	-8.1203	112.195	Kios	Edit
Akar Tani	Jawa Timur	Kabupaten Magetan			-7.68108	111.237		Edit
Akar Tani	Jawa Timur	Kabupaten Magetan	Saifuddin	081237378063	-7.68107	111.237	Kios	Edit
Anugerah Subur	Jawa Timur	Kabupaten Mojokerto	Ibu Aam	081357615836	-7.52794	112.594	Kios	Edit
Anugerah Tani Makmur	Jawa Timur	Kabupaten Mojokerto	Pak Handoyo	081235230177	-7.52456	112.414	Kios	Edit

Previous

Page 1

Next

Gambar 3.41 Form penambahan data kios

Marketing Map

Admin Panel

Back to Home

Overview

Dashboard

Marketing Map

Regions

Lands

Commodities

Products

Product Type

Product Brand

Product Dosage

Potential

Potential

Sale

Sales Overview

Stall

Stalls Overview

User Management

User Management

adminmpb

adminmpb@example.com

Manage Stalls

Manage kios and map distribution data

Total Stalls

164

Total Provinces

3

Total Regencies

29

Stall List

Manage kios and map distribution

+ Add Stall

Export Excel

10 Rows

Q Search stall...

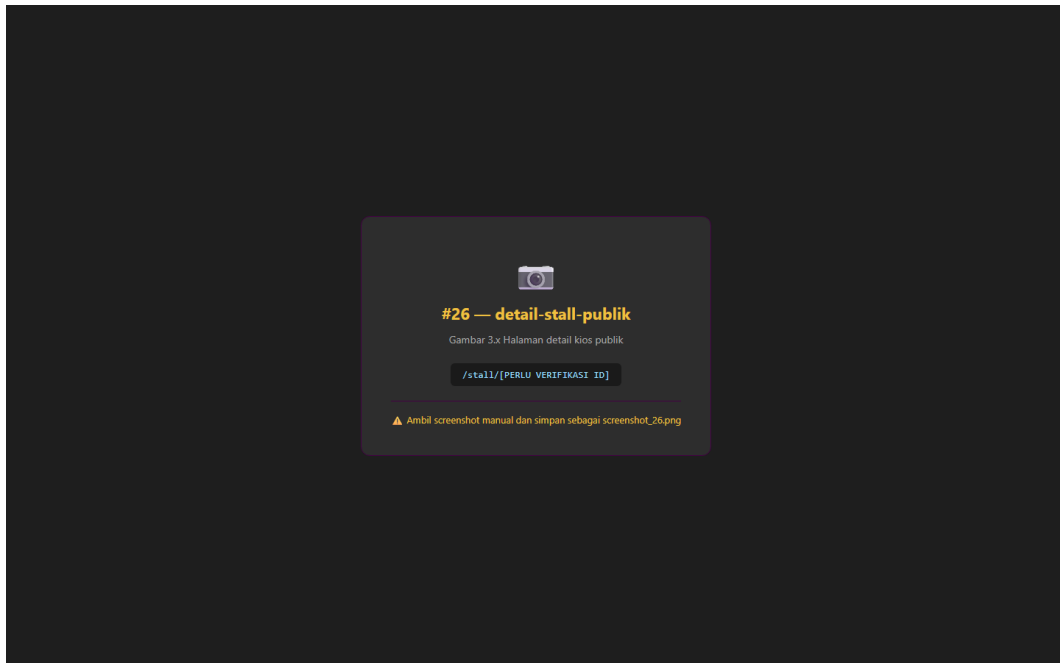
Name	Province	Regency	Owner	Phone	Latitude	Longitude	Criteria	Action
2 Putra Tani	Jawa Timur	Kabupaten Banyuwangi	Pak Husni Mubarak	081234849877	-8.28816	114.317	Kios	Edit
Abi Tani	Jawa Timur	Kabupaten Magetan			-7.65536	111.28		Edit
Abi Tani Jetak	Jawa Tengah	Kabupaten Sragen	Bu Meyda	082328181015	-7.44099	110.981	Kios	Edit
Abi Tani Ngrampal	Jawa Tengah	Kabupaten Sragen	Vira	085743041353	-7.39647	111.067	Kios	Edit
Agro Makmur	Jawa Tengah	Kabupaten Temanggung	Syarifudin	081227451316	-7.28869	110.182	Kios	Edit
Aji Tani	Jawa Timur	Kabupaten Blitar	Yatno	085790214611	-8.1203	112.195	Kios	Edit
Akar Tani	Jawa Timur	Kabupaten Magetan			-7.68108	111.237		Edit
Akar Tani	Jawa Timur	Kabupaten Magetan	Saifuddin	081237378063	-7.68107	111.237	Kios	Edit
Anugerah Subur	Jawa Timur	Kabupaten Mojokerto	Ibu Aam	081357615836	-7.52794	112.594	Kios	Edit
Anugerah Tani Makmur	Jawa Timur	Kabupaten Mojokerto	Pak Handoyo	081235230177	-7.52456	112.414	Kios	Edit

Previous

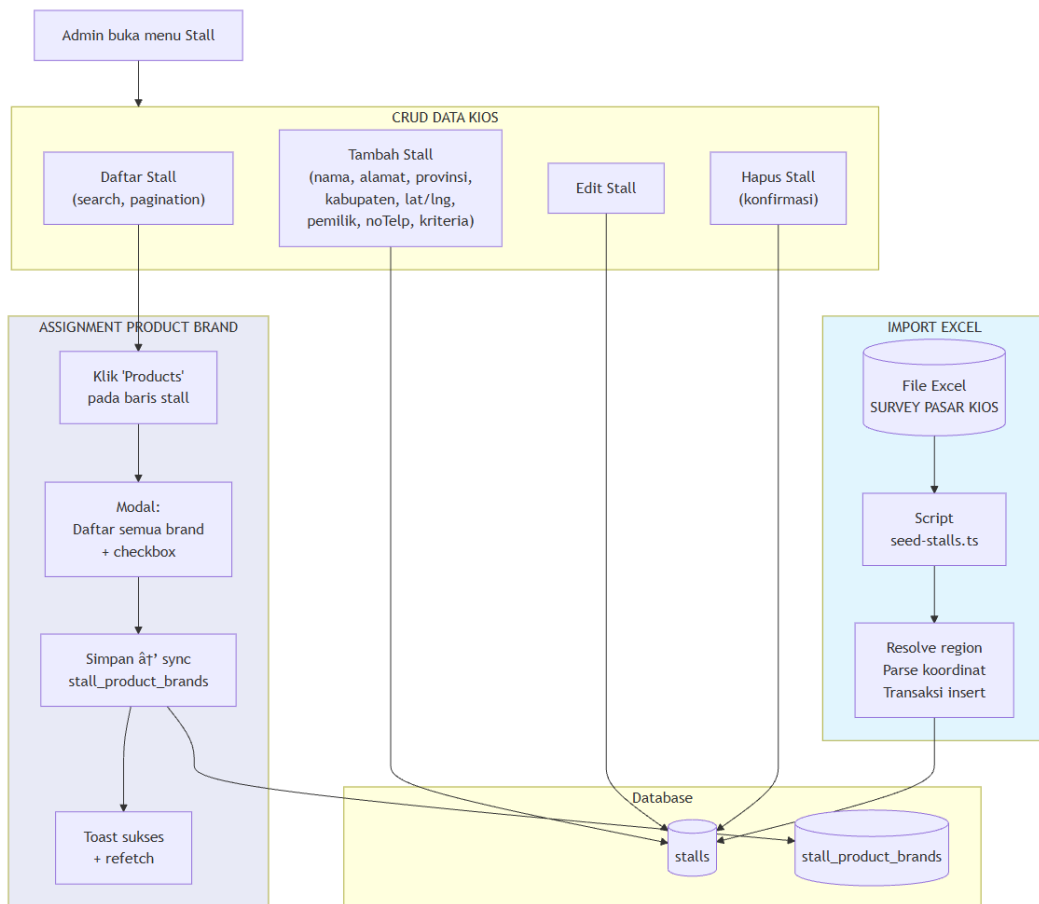
Page 1

Next

Gambar 3.42 Modal assignment brand produk ke kios



Gambar 3.43 Halaman detail kios publik



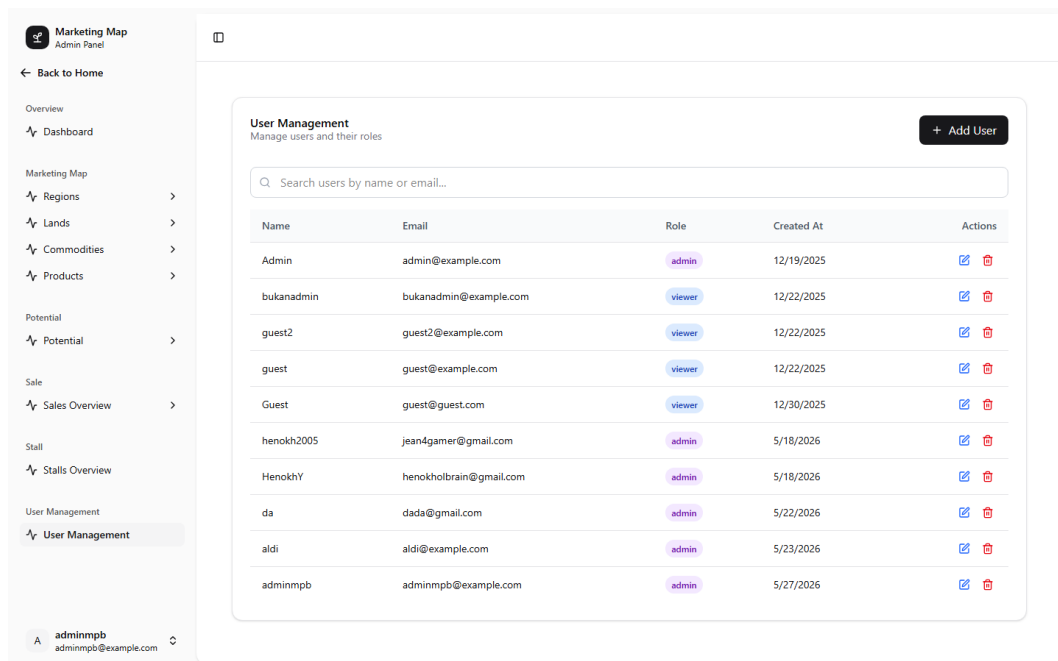
Gambar 3.14 Diagram alur CRUD Stall dan assignment Product Brand

3.8.8 User Management

Modul User Management digunakan untuk mengelola data pengguna dan peran (role) dalam sistem. Data yang dikelola meliputi nama, email, dan role pengguna.

Modul menyediakan operasi pengambilan daftar pengguna, detail pengguna, pembaruan data (termasuk perubahan role), dan penghapusan pengguna. Pembuatan pengguna dilakukan melalui mekanisme signup yang disediakan oleh Better Auth.

Role yang tersedia meliputi admin (akses penuh), viewer (akses terbatas), dan guest (role default).



Name	Email	Role	Created At	Actions
Admin	admin@example.com	admin	12/19/2025	Edit Delete
bukanadmin	bukanadmin@example.com	viewer	12/22/2025	Edit Delete
guest2	guest2@example.com	viewer	12/22/2025	Edit Delete
guest	guest@example.com	viewer	12/22/2025	Edit Delete
Guest	guest@guest.com	viewer	12/30/2025	Edit Delete
henokh2005	jean4gamer@gmail.com	admin	5/18/2026	Edit Delete
HenokhY	henokholtrain@gmail.com	admin	5/18/2026	Edit Delete
da	dada@gmail.com	admin	5/22/2026	Edit Delete
aldi	aldi@example.com	admin	5/23/2026	Edit Delete
adminmpb	adminmpb@example.com	admin	5/27/2026	Edit Delete

Gambar 3.44 Daftar pengguna dengan role masing-masing

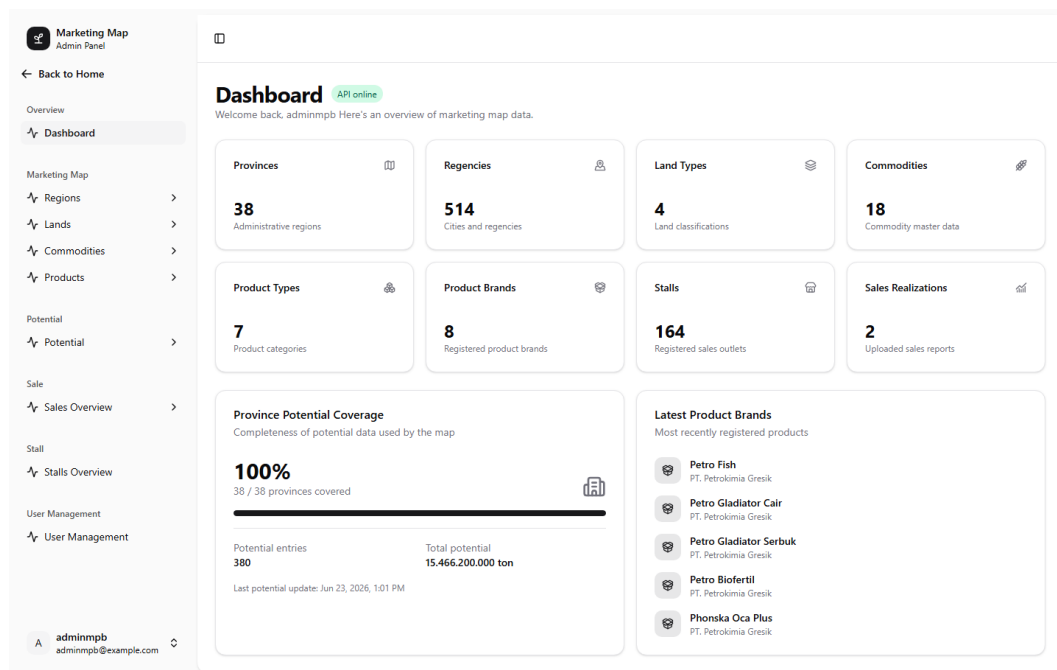
3.8.9 Dashboard Admin

Dashboard Admin menampilkan ringkasan data dalam bentuk cards statistik yang mencakup:

- Jumlah provinsi, kabupaten, jenis lahan, jenis komoditas, jenis produk, brand produk, dan kios yang terdaftar.
- Jumlah data realisasi penjualan.

- Progress bar untuk potensi provinsi (Province Potential Coverage).
- Daftar brand produk terbaru.
- Indikator health check status API.

Dashboard menjadi pusat navigasi bagi admin untuk memantau kondisi data secara cepat dan mengakses modul-modul pengelolaan data.



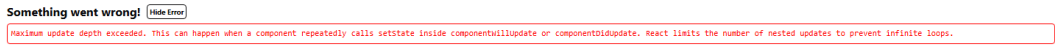
Gambar 3.45 Dashboard admin dengan ringkasan data

3.8.10 Visualisasi Peta

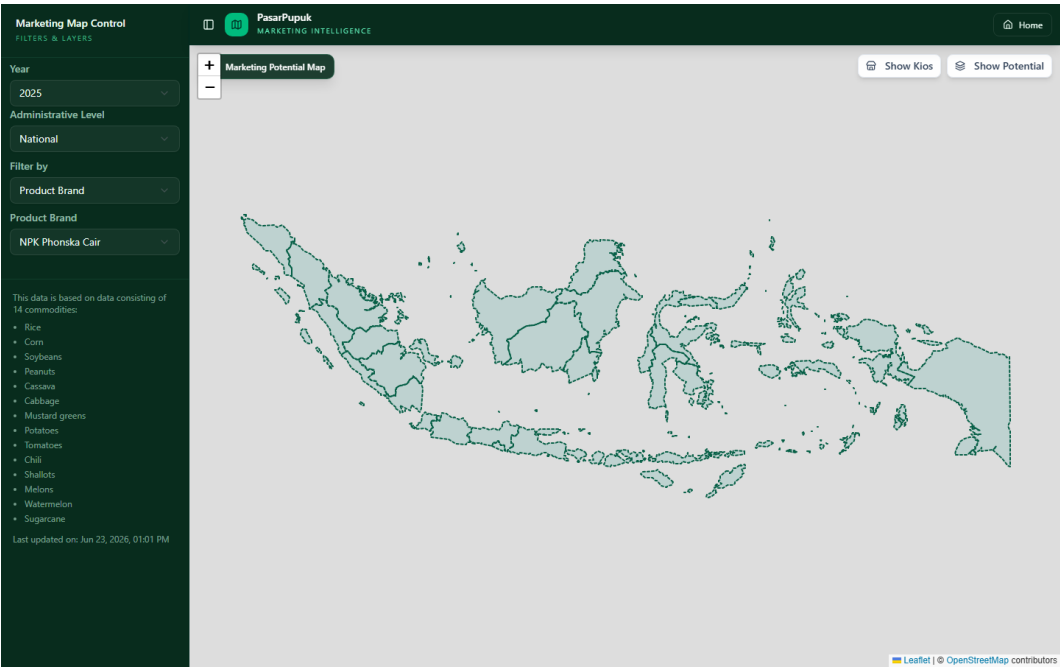
Halaman peta interaktif dibangun menggunakan Leaflet dan React Leaflet untuk menampilkan data spasial secara visual. Komponen utama halaman peta meliputi:

- **Layer batas wilayah:** Data GeoJSON batas provinsi dan kabupaten Indonesia yang dimuat dari folder `public/data/`.
- **Choropleth:** Visualisasi data potensi pasar berdasarkan wilayah dengan gradasi warna.
- **Marker kios:** Titik lokasi kios yang diambil dari data Stall.
- **Sidebar filter:** Panel untuk menyaring data berdasarkan tahun, tingkat administrasi, brand, jenis lahan, dan jenis komoditas.
- **Popup:** Informasi detail provinsi atau kabupaten yang muncul saat wilayah diklik.

Data untuk peta bersumber dari GeoJSON statis untuk batas wilayah dan API oRPC untuk data potensi dan kios.



Gambar 3.46 Peta interaktif dengan batas wilayah Indonesia



Gambar 3.47 Visualisasi choropleth data potensi pasar

3.8.11 Landing Page Publik

Halaman publik (landing page) merupakan antarmuka pertama yang dilihat pengunjung. Halaman ini menampilkan:

- **Hero section:** Judul aplikasi dan statistik ringkas (jumlah provinsi, komoditas, brand produk, kios).
- **Program cards:** Informasi program Market Map, Roadshow, dan Socialization.
- **Monitoring agenda:** Agenda pemantauan tahun 2026.
- **Struktur pimpinan:** Informasi Project Manager (PM) dan Senior Manager Department I (SMD I).
- **Call-to-action:** Tombol navigasi menuju peta potensi dan program kerja.



Gambar 3.48 Halaman utama Satu Peta Pasar

3.9 Pengujian Sistem

3.9.1 Pengujian API

Pengujian API dilakukan untuk memverifikasi bahwa endpoint oRPC berfungsi sesuai dengan yang diharapkan. Skenario pengujian meliputi:

- Health check endpoint untuk memastikan server berjalan.
- Akses endpoint publik tanpa session.
- Akses protected endpoint tanpa session (diharapkan ditolak).

- Validasi input dengan data tidak valid (diharapkan mengembalikan error validasi).
- Operasi CRUD pada setiap modul dengan data valid.

3.9.2 Pengujian Authentication

Pengujian authentication dilakukan untuk memverifikasi mekanisme login dan pembatasan akses:

- Login dengan kredensial valid (diharapkan berhasil dan mendapatkan session).
- Login dengan kredensial tidak valid (diharapkan ditolak).
- Akses route admin tanpa login (diharapkan dialihkan ke halaman login).
- Akses route admin dengan role guest (diharapkan ditolak oleh route guard).

3.9.3 Quality Check

Pemeriksaan kualitas kode dilakukan menggunakan perintah yang tersedia dalam proyek:

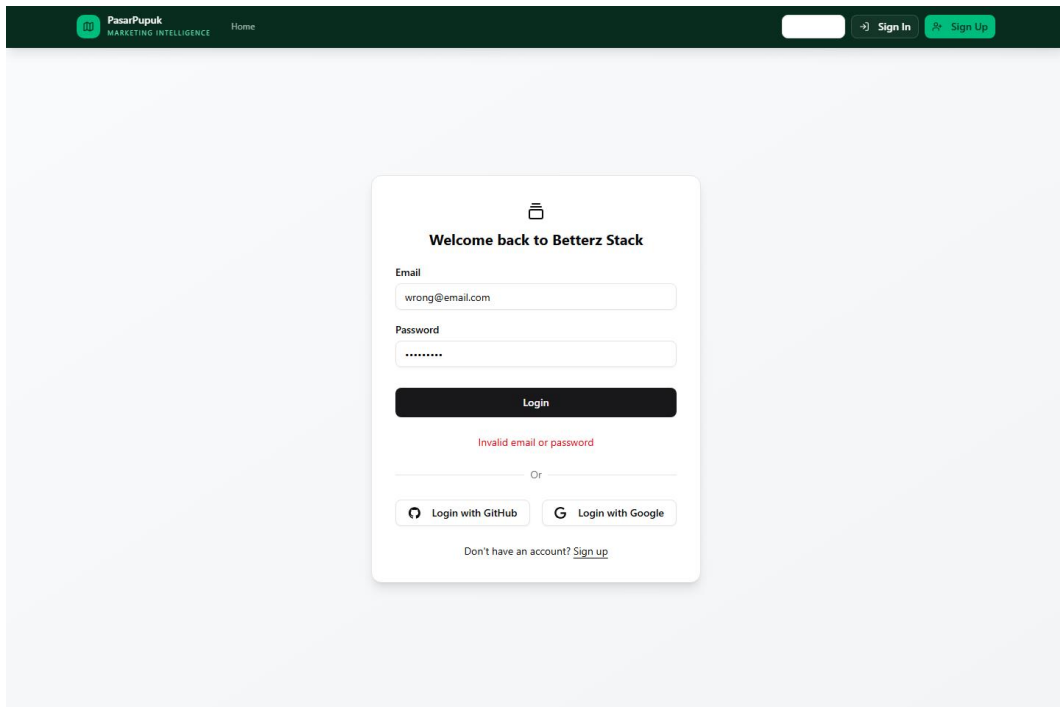
- Type checking dengan `'bun run check-types'` untuk memvalidasi tipe TypeScript.
- Linting dan formatting dengan `'bun run check'` (Biome / Ultracite).

3.9.4 Production Build

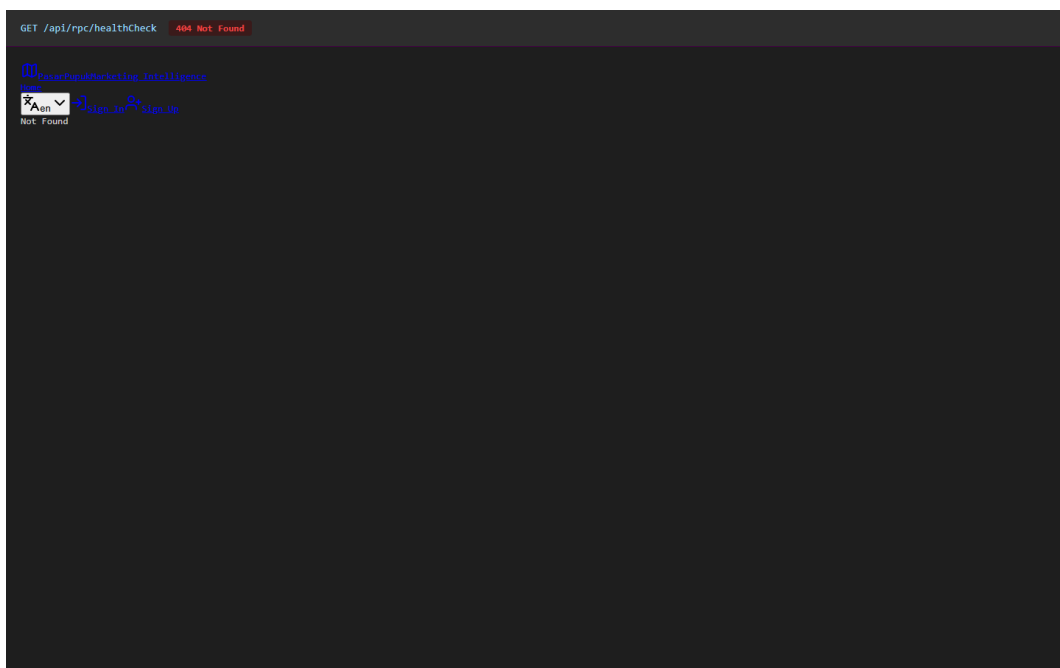
Pengujian build dilakukan untuk memastikan aplikasi dapat dikompilasi ke mode produksi tanpa error. Perintah `'bun run build'` digunakan untuk menjalankan proses build melalui Vite dan Turborepo.

3.9.5 Keterbatasan Pengujian

Pengujian dilakukan dalam lingkup terbatas sesuai dengan waktu dan akses yang tersedia selama kegiatan magang. Pengujian otomatis (unit test dan integration test) dengan Vitest belum dapat dilaksanakan secara menyeluruh karena cakupan test yang masih terbatas dalam repository.



Gambar 3.49 Pesan error saat login gagal



Gambar 3.50 Response endpoint health check



404 Not Found
Not Found

Gambar 3.51 Response validasi error dari Zod

3.10 Build, Testing, dan Deployment

3.10.1 Build

Proses build aplikasi menggunakan Vite sebagai bundler dan Turborepo untuk mengelola task monorepo secara paralel. Target build dikonfigurasi untuk TanStack Start dengan platform Netlify. Konfigurasi build terdapat pada `vite.config.ts` dan `turbo.json` di root proyek.

3.10.2 Linting dan Formatting

Proyek menggunakan Biome dan Ultracite untuk pemeriksaan kualitas kode. Ultracite menyediakan aturan tambahan untuk type safety, aksesibilitas, dan konsistensi kode yang melengkapi aturan bawaan Biome.

3.10.3 Deployment

Beberapa opsi deployment telah dikonfigurasi dalam proyek:

Netlify:

Konfigurasi melalui `netlify.toml` dengan base directory `apps/web`, publish directory `dist`, dan build command `bun run build`.

Vercel:

Konfigurasi alternatif melalui `vercel.json`.

Docker:

Dockerfile untuk membangun aplikasi dalam container dan docker-compose.yml untuk menjalankan layanan PostgreSQL lokal.

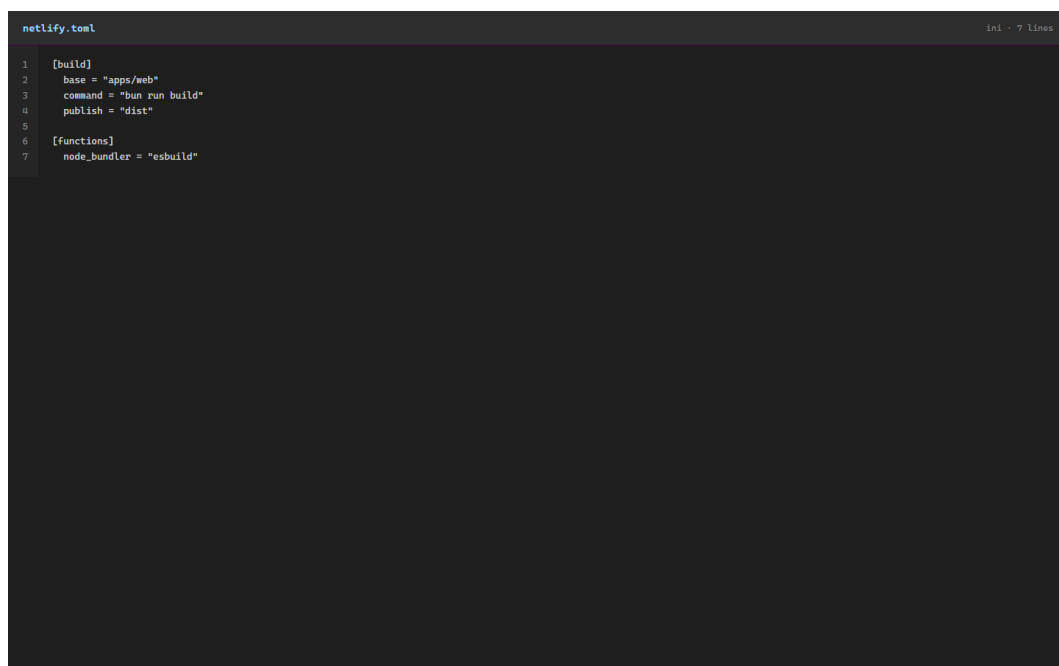
GitHub Actions:

Workflow `Ping Supabase` untuk menjaga koneksi database Supabase tetap aktif. Pipeline build-test penuh belum terdokumentasi.

3.10.4 Environment Variable

Variabel lingkungan yang diperlukan untuk menjalankan aplikasi meliputi:

- `DATABASE\_URL` — URL koneksi database PostgreSQL.
- `BETTER\_AUTH\_SECRET` — Secret key untuk Better Auth.
- `BETTER\_AUTH\_URL` — Base URL aplikasi.
- `CORS\_ORIGIN` — Origin yang diizinkan.
- `VITE\_BETTER\_AUTH\_URL` — URL auth untuk client.



```
netlify.toml
1 [build]
2   base = "apps/web"
3   command = "bun run build"
4   publish = "dist"
5
6 [functions]
7   node_bundler = "esbuild"
ini - 7 lines
```

Gambar 3.52 Konfigurasi deployment Netlify

3.11 Analisis dan Temuan

3.11.1 Status Modul

Berdasarkan hasil analisis, tingkat kesiapan setiap modul belum sepenuhnya seragam. Modul dengan CRUD penuh meliputi Province, Regency, Land Type, Province Land, Commodity Type, Regency Commodity, Product Type, Product Brand, Product Dosage, Sales Realization, Daily Sales, dan Stall. Modul yang masih bersifat read-only meliputi Regency Land, Province Commodity, dan Province Potential. Regency Potential memiliki tabel database yang tersedia, tetapi endpoint aktif belum ditemukan.

3.11.2 Gap Authorization

Mekanisme protected procedure telah memeriksa keberadaan session, tetapi validasi role pada lapisan API belum diterapkan secara eksplisit. Kondisi ini menyebabkan API masih dapat diakses oleh pengguna dengan role viewer atau guest selama mereka memiliki session yang valid. Route guard pada antarmuka telah memeriksa role, tetapi lapisan API belum memiliki mekanisme yang setara.

3.11.3 Mismatch Schema-Migration

Pada tabel `sales\_realizations`, ditemukan indikasi perbedaan penamaan field antara schema dan migration. Schema menggunakan nama `realization\_ytd`, sedangkan migration menggunakan `realizaton\_ytd`. Perbedaan ini perlu diverifikasi terhadap database aktual untuk menentukan kondisi sebenarnya.

3.11.4 Missing Foreign Key

Field `province\_id` pada tabel `daily\_sales` belum didefinisikan sebagai foreign key pada schema yang dianalisis. Penambahan foreign key diperlukan untuk memperkuat integritas referensial antara data penjualan harian dan data provinsi.

3.11.5 Unique Constraint

Kombinasi data yang seharusnya unik, seperti `(province\_id, land\_type\_id, year)` pada `province\_lands` dan `(stall\_id, product\_brand\_id)` pada `stall\_product\_brands`, belum terdokumentasi memiliki constraint unique. Penambahan unique constraint dapat mencegah duplikasi data yang tidak diinginkan.

3.11.6 Coupling Modul Stall

Modul Stall memiliki tanggung jawab yang cukup luas, mencakup pengelolaan data kios, assignment Product Brand, dan integrasi dengan modul lain. Pemisahan komponen dan use case menjadi modul yang lebih terfokus dapat meningkatkan maintainability kode.

3.11.7 Standardisasi

Beberapa aspek implementasi yang belum seragam meliputi:

- Format response API.
- Mekanisme pagination.
- Pola error handling.
- Struktur pemisahan router dan schema per modul.

Standardisasi pada area tersebut dapat meningkatkan konsistensi dan kemudahan pemeliharaan sistem.

3.11.8 Rekomendasi

Berdasarkan temuan di atas, rekomendasi pengembangan meliputi:

1. Menambahkan `adminProcedure` untuk validasi role pada seluruh protected API.
2. Menyinkronkan nama field antara schema dan migration, khususnya pada `sales\_realizations`.
3. Menambahkan foreign key `daily\_sales.province\_id` untuk memperkuat integritas referensial.
4. Menambahkan unique constraint pada kombinasi data yang seharusnya unik.
5. Melakukan standardisasi response API, pagination, error handling, dan struktur router.
6. Memisahkan router oRPC per domain untuk mengurangi kompleksitas.
7. Menambahkan integration test untuk API dan authorization test.

3.12 Kontribusi Mahasiswa

3.12.1 Bukti Repository

Berdasarkan analisis riwayat Git, ditemukan 22 commit oleh author Git `HenokhYeremia <henokholbrain@gmail.com>`. Identitas email `henokholbrain@gmail.com` konsisten dengan nama mahasiswa **Henokh Yeremia Olbrain Perangin Angin**, sehingga kontribusi pada commit dapat diatribusikan kepada mahasiswa.

3.12.2 Commit Modul Stall

Commit `7a38bb7` (26 Mei 2026) dengan judul `feat: complete stall and product brand management` merupakan bukti utama kontribusi implementasi. Perubahan pada commit ini mencakup:

- **Operasi CRUD Stall:** Pengembangan create, update, dan delete pada modul Stall.
- **Form pengelolaan:** Penyempurnaan form input data kios.
- **Integrasi router oRPC:** Penghubungan procedure Stall ke dalam komposisi router.
- **Assignment Product Brand:** Pengembangan fitur pengelolaan relasi Product Brand pada Stall melalui komponen `manage-stall-products.tsx` dan prosedur sinkronisasi relasi.
- **Perubahan query Regency:** Penyesuaian query terkait data Regency.

3.12.3 Commit Konfigurasi Deployment

Sebanyak 12 commit setelah commit fitur Stall menunjukkan perubahan konfigurasi deployment, meliputi:

- Konfigurasi Vercel dan Netlify.
- Redirect SSR.
- Target build.
- Workflow ping Supabase dan keep-alive.

3.12.4 Kegiatan Analisis dan Dokumentasi

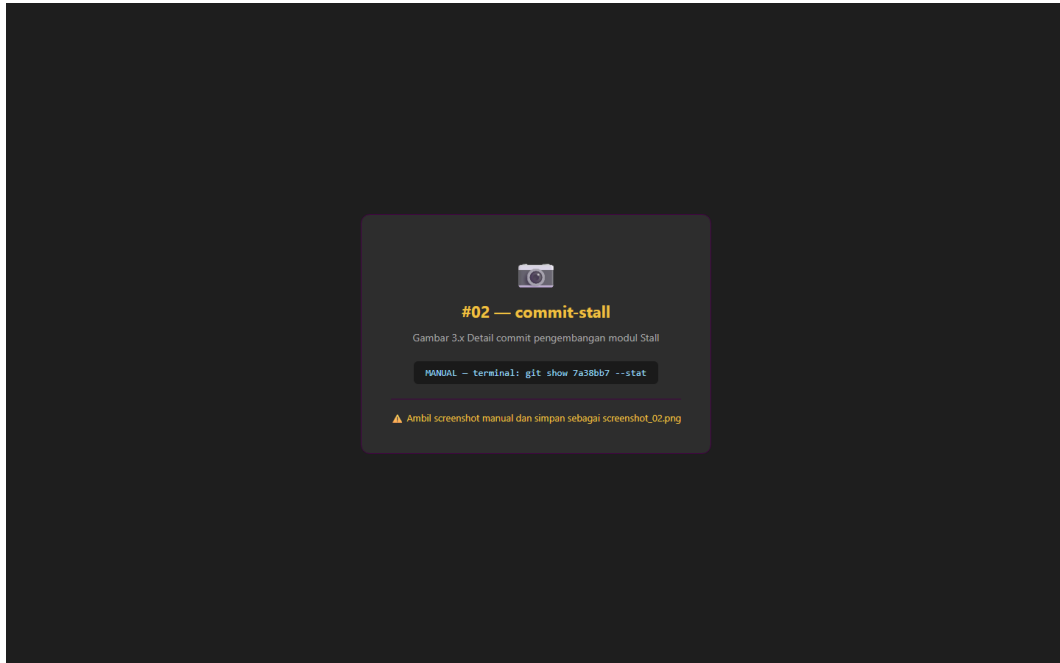
Selain kontribusi implementasi, penulis melakukan kegiatan analisis dan dokumentasi yang mencakup:

- Analisis arsitektur backend, database, API, authentication, dan sembilan modul bisnis.
- Dokumentasi teknis struktur backend, relasi database, endpoint API, dan matriks modul.
- Pemetaan status kesiapan fitur dan identifikasi temuan kualitas.
- Penyusunan rekomendasi pengembangan berdasarkan hasil analisis.

3.12.5 Batasan Kontribusi

Modul-modul berikut tidak diklaim sebagai kontribusi implementasi mandiri mahasiswa dan diposisikan sebagai objek analisis: Product Dosage, Product Brand (CRUD master), Commodity Type, Province Commodity, Regency Commodity, Province Potential, Sales

Realization, dan Daily Sales. Modul-modul tersebut telah tersedia pada snapshot awal repository (commit `3959ce9`) dan tidak ditemukan commit spesifik yang menunjukkan perubahan oleh mahasiswa pada modul-modul tersebut.



Gambar 3.53 Detail commit pengembangan modul Stall

4.1 Kesimpulan

4.1.1 Kesimpulan Arsitektur Sistem

Aplikasi Satu Peta Pasar menerapkan arsitektur full-stack monolith pada workspace `apps/web` yang mengintegrasikan React dan TanStack Start untuk antarmuka pengguna, oRPC untuk lapisan API, Drizzle ORM untuk akses database, dan PostgreSQL untuk penyimpanan data. Berbeda dengan deskripsi template proyek yang membayangkan backend Hono terpisah pada `apps/server`, implementasi aktif menempatkan seluruh lapisan aplikasi dalam satu kesatuan.

Alur data dimulai dari komponen React yang memicu request melalui TanStack Query ke oRPC client. Request diteruskan ke server route TanStack Start yang menjalankan procedure oRPC. Setiap procedure memvalidasi input menggunakan Zod, memeriksa session melalui Better Auth, dan mengakses database melalui Drizzle ORM. Mekanisme ini memastikan type safety di seluruh lapisan aplikasi, mulai dari antarmuka pengguna hingga database.

Better Auth menangani authentication dengan metode email dan password, session berbasis cookie, dan adapter Drizzle ORM untuk penyimpanan data. Route guard pada antarmuka telah memeriksa role pengguna, tetapi protected procedure pada lapisan API belum menerapkan validasi role secara eksplisit.

4.1.2 Kesimpulan Database

Database sistem terdiri dari 22 tabel yang mencakup domain autentikasi (user, session, account, verification), wilayah dan lahan (provinces, regencies, land_types, province_lands, regency_lands), komoditas (commodity_types, province_commodities, regency_commodities), produk dan potensi (product_types, product_brands, product_dosages, province_potentials, regency_potentials), kios (stalls, stall_product_brands), penjualan (sales_realizations, daily_sales), dan demo (todo). Seluruh tabel menggunakan primary key UUID dan dihubungkan melalui foreign key.

Hasil analisis menemukan beberapa area yang memerlukan perhatian: field `province\_id` pada `daily\_sales` belum didefinisikan sebagai foreign key, indikasi perbedaan penamaan

field antara schema dan migration pada `sales\_realizations`, serta unique constraint yang belum terdokumentasi secara eksplisit.

4.1.3 Kesimpulan Modul Bisnis

Tingkat kesiapan setiap modul belum sepenuhnya seragam. Modul dengan CRUD penuh meliputi Province, Regency, Land Type, Province Land, Commodity Type, Regency Commodity, Product Type, Product Brand, Product Dosage, Sales Realization, Daily Sales, dan Stall. Modul yang masih bersifat read-only meliputi Regency Land, Province Commodity, dan Province Potential. Regency Potential memiliki tabel database yang tersedia tetapi endpoint aktif belum ditemukan.

4.1.4 Kesimpulan Kontribusi

Berdasarkan analisis riwayat Git, kontribusi implementasi mahasiswa teridentifikasi pada modul Stall dan assignment Product Brand melalui commit `7a38bb7`, serta rangkaian perubahan konfigurasi deployment Vercel, Netlify, dan workflow Supabase pada 12 commit berikutnya. Identitas author Git `HenokhYeremia <henokholbrain@gmail.com>` telah terverifikasi konsisten dengan nama mahasiswa, sehingga atribusi kontribusi dapat digunakan dalam laporan. Modul bisnis lainnya dibahas sebagai objek analisis dan dokumentasi, bukan sebagai klaim implementasi mandiri mahasiswa.

4.2 Keterbatasan

Kegiatan magang ini memiliki beberapa keterbatasan yang perlu dicatat:

1. Logbook harian, pull request, tiket, atau bukti acceptance test belum tersedia untuk mendukung klaim kegiatan secara lebih rinci.
2. Build, lint, test, dan pengujian API aktual masih perlu dilakukan untuk memvalidasi temuan analisis secara langsung.
3. Database production belum diperiksa; analisis hanya berdasarkan schema dan migration yang terdapat pada repository.
4. Beberapa fitur memiliki endpoint yang belum aktif atau belum terdokumentasi, seperti Regency Potential dan mutation Province Commodity.

5. Perubahan lokal pada Product Dosage (field `year`) dan Sales Realization (page-size selector) belum di-commit dan belum dapat diklaim sebagai kontribusi.

4.3 Saran Pengembangan

4.3.1 Saran Jangka Pendek

1. **Tambahkan adminProcedure:** Implementasi validasi role `admin` pada seluruh protected API untuk menutup gap authorization antara route guard UI dan lapisan API.
2. **Sinkronkan schema dan migration:** Verifikasi dan sinkronkan nama field antara schema dan migration, khususnya `realizaton\_ytd` / `realization\_ytd` pada tabel `sales\_realizations`.
3. **Tambahkan foreign key:** Definisikan foreign key untuk field `province\_id` pada tabel `daily\_sales` untuk memperkuat integritas referensial.

4.3.2 Saran Jangka Menengah

1. **Standardisasi API:** Seragamkan format response, pagination, error handling, dan struktur router schema di seluruh modul.
2. **Pisahkan router per domain:** Kurangi kompleksitas router utama dengan memisahkan definisi router oRPC per domain bisnis.
3. **Tambahkan constraint:** Implementasikan unique constraint pada kombinasi data yang seharusnya unik, seperti `(province\_id, land\_type\_id, year)`.
4. **Lengkapi CRUD modul read-only:** Aktifkan operasi create, update, dan delete pada modul yang masih read-only jika kebutuhan bisnis menghendaki.

4.3.3 Saran Jangka Panjang

1. **Integration test:** Implementasikan integration test untuk API dan authorization test untuk skenario role.
2. **Repository pattern:** Terapkan pola repository untuk memisahkan logika bisnis dari

kueri database.

3. **CI/CD pipeline:** Sempurnakan pipeline dengan build otomatis, lint, test, dan deployment yang terintegrasi.

4.3.4 Saran untuk Kegiatan Magang

1. **Dokumentasi berkala:** Catat kegiatan secara teratur dalam logbook harian untuk mendukung klaim kontribusi.

2. **Gunakan pull request:** Manfaatkan pull request dan tiket untuk melacak perubahan dan diskusi pengembangan.

3. **Konfirmasi identitas:** Pastikan identitas author Git dan hubungannya dengan mahasiswa terverifikasi pada awal kegiatan untuk memudahkan atribusi kontribusi.

4.4 Penutup

Kegiatan magang ini telah memberikan pengalaman berharga dalam mempelajari dan menganalisis pengembangan backend aplikasi web skala industri. Penulis mengucapkan terima kasih kepada PT Petrokimia Gresik, khususnya Departemen Manajemen Produk Baru, yang telah memberikan kesempatan untuk melaksanakan kegiatan magang. Ucapan terima kasih juga disampaikan kepada pembimbing lapangan, dosen pembimbing, dan seluruh pihak yang telah mendukung kelancaran kegiatan magang ini.

Semoga laporan ini dapat memberikan manfaat bagi pengembangan sistem Satu Peta Pasar di masa mendatang serta menjadi referensi bagi kegiatan magang serupa di lingkungan akademik.

DAFTAR PUSTAKA

- Better Auth. *Documentation*. <https://www.better-auth.com/>
- Biome. *Biome CLI Documentation*. <https://biomejs.dev/>
- Bun. *Bun Documentation*. <https://bun.sh/docs>
- Drizzle ORM. *Drizzle ORM Documentation*. <https://orm.drizzle.team/>
- Leaflet. *Leaflet — an open-source JavaScript library for interactive maps*. <https://leafletjs.com/>
- oRPC. *oRPC Documentation*. <https://orpc.unnoq.com/>
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- React. *React Documentation*. <https://react.dev/>
- TanStack. *TanStack Query Documentation*. <https://tanstack.com/query/latest>
- TanStack. *TanStack Router Documentation*. <https://tanstack.com/router/latest>
- TanStack. *TanStack Start Documentation*. <https://tanstack.com/start/latest>
- Ultracite. *Ultracite Documentation*. <https://ultracite.dev/>
- Vite. *Vite Documentation*. <https://vitejs.dev/>
- Zod. *Zod Documentation*. <https://zod.dev/>

LAMPIRAN

Lampiran A — Logbook Kegiatan

Daftar riwayat commit yang menunjukkan aktivitas pengembangan selama kegiatan magang:

No	Tanggal	Commit	Aktivitas
2	26 Mei 2026	`7a38bb7`	Pengembangan modul Stall dan assignment Product Brand
3	26 Mei 2026	`896daf`	Konfigurasi Turborepo output dan environment
4	26 Mei 2026	`e98cc1a`	Konfigurasi deployment Vercel
5	26 Mei 2026	`48cdc1a`	Penambahan konfigurasi Vercel
6	26 Mei 2026	`7626d30`	Perubahan target TanStack ke Vercel
7	26 Mei 2026	`7e0a167`	Rollback konfigurasi deployment
8	26 Mei 2026	`f750d88`	Alih target deployment ke Netlify
9	26 Mei 2026	`057109c`	Penambahan redirect Netlify
10	27 Mei 2026	`f55cf89`	Penambahan SSR redirect Netlify
11	27 Mei 2026	`7e02e4f`	Penggunaan functions-internal server redirect
12	27 Mei 2026	`a34e382`	Penghapusan manual Netlify redirects
13	27 Mei 2026	`19dda4b`	Pembaruan konfigurasi deployment Netlify
14	19 Jun 2026	`71a93b8`	Peningkatan workflow keep-alive Supabase
15	22 Jun 2026	`52dfe56`	Peningkatan admin data dan map loading
16	24 Jun 2026	`83bf8d8`	Pembaruan marketing map
17	24 Jun 2026	`c509687`	Exclude private workspace files
18	24 Jun 2026	`6acf260`	Penambahan potential management dan live dashboard

19	24 Jun 2026	`9446f96`	Pembaruan landing page dan marketing map
20	24 Jun 2026	`9e6cade`	Peningkatan sales dan stall management workflows
21	24 Jun 2026	`d035cbe`	Restorasi product management leadership
22	24 Jun 2026	`eee21e8`	Dokumentasi README dengan animated overview

Lampiran B — Screenshot Tambahan

Catatan: Screenshot akan diambil dari sistem yang berjalan. Berikut daftar screenshot tambahan yang direncanakan untuk lampiran:

No	Nama Screenshot	Keterangan
S2	Response API health check	Endpoint publik
S3	Response validation error Zod	Error validasi input
S4	Environment variable configuration	Tanpa nilai secret
S5	Admin dashboard	Summary cards
S6	Landing page publik	Hero section dan program cards
S7	OpenAPI documentation	Jika tersedia

Lampiran C — Source Code Representatif

C.1 Router oRPC Utama

File: `lib/orpc/router/index.ts`

Router utama yang mengelompokkan seluruh endpoint API berdasarkan domain bisnis. Setiap endpoint didefinisikan sebagai procedure oRPC (public atau protected) yang menghubungkan client dengan handler domain.

Struktur router mencakup:

- `healthCheck` — endpoint publik untuk monitoring
- `auth.getSession` — mendapatkan session pengguna
- `todo.\*` — CRUD todo (demo)
- `map.\*` — data untuk peta
- `admin.\*` — seluruh endpoint administrasi yang dilindungi

C.2 Schema Database

File: `lib/db/schema/`

Schema Drizzle ORM mendefinisikan 22 tabel dalam 5 file:

File	Domain	Tabel
`map-product.ts`	Wilayah, Lahan, Komoditas, Produk, Potensi	`provinces`, `regencies`, `land_types`, `province_lands`, `regency_lands`, `commodity_types`, `province_commodities`, `regency_commodities`, `product_types`, `product_brands`, `product_dosages`, `province_potentials`, `regency_potentials`
`sale.ts`	Penjualan	`sales_realizations`, `daily_sales`
`stall.ts`	Kios	`stalls`, `stall_product_brands`
`todo.ts`	Demo	`todo`

C.3 Route Guard Admin

File: `routes/admin/route.tsx`

Route guard yang memeriksa session dan role admin sebelum mengizinkan akses ke halaman admin. Logika:

1. Periksa session pengguna — jika tidak ada, redirect ke halaman login.
2. Periksa role pengguna — jika bukan admin, redirect ke halaman akses ditolak.

C.4 Better Auth Configuration

File: `lib/auth/index.ts`

Konfigurasi Better Auth dengan provider email/password dan adapter Drizzle ORM.

Mengelola:

- Sign in / Sign up
- Session management
- Cookie handling

C.5 Stall Module — Assignment Product Brand

File: `routes/admin/stall/-app/assign-product-brand.ts`

Procedure untuk menyinkronkan relasi many-to-many antara Stall dan Product Brand.

Logika:

1. Hapus seluruh relasi existing untuk stall yang dipilih.
2. Insert relasi baru berdasarkan daftar brand yang dipilih admin.
3. Menggunakan transaksi database untuk atomicity.

Lampiran D — Surat Keterangan Magang

[Tempatkan surat keterangan magang dari PT Petrokimia Gresik di sini jika tersedia]

Lampiran E — Daftar Lengkap Endpoint API

Berikut daftar seluruh endpoint oRPC yang terdaftar pada router:

Public Endpoints

Path	Method	Auth	Input	Output	Keterangan
`auth.getSession`	handler	public	-	Session user	Mendapatkan session saat ini
`map.getProvinces`	handler	public	-	Daftar provinsi	Data provinsi untuk peta
`map.getStalls`	handler	public	-	Daftar kios	Data kios untuk peta

Protected Endpoints (Admin)

Dashboard

Region

Path	Auth	Input	Output
`admin.region.province.create`	session	`{ code, name, area, year }`	Province
`admin.region.province.update`	session	`{ id, code?, name?, area?, year? }`	Province
`admin.region.province.delete`	session	`{ id }`	-
`admin.region.regency.get`	session	`{ search, page, pageSize }`	Paginated regencies

`admin.region.regency.create`	session	`{ code, name, province_id, area, year }`	Regency
`admin.region.regency.update`	session	`{ id, code?, name?, province_id?, area?, year? }`	Regency
`admin.region.regency.delete`	session	`{ id }`	-

Land

Path	Auth	Input	Output
`admin.land.land_type.create`	session	`{ name, year }`	Land type
`admin.land.land_type.update`	session	`{ id, name?, year? }`	Land type
`admin.land.land_type.delete`	session	`{ id }`	-
`admin.land.province_land.get`	session	`{ search, page, pageSize }`	Paginated province lands
`admin.land.province_land.create`	session	Data province land	Province land
`admin.land.province_land.update`	session	Data province land	Province land
`admin.land.province_land.delete`	session	`{ id }`	-
`admin.land.regency_land.get`	session	`{ search, page, pageSize }`	Paginated regency lands
`admin.land.regency_land.create`	session	Data regency land	Regency land
`admin.land.regency_land.update`	session	Data regency land	Regency land
`admin.land.regency_land.delete`	session	`{ id }`	-

Commodity

Path	Auth	Input	Output
`admin.commodity.commodity_type.create`	session	`{ name, land_type_id, year }`	Commodity type
`admin.commodity.commodity_type.update`	session	`{ id, name?, land_type_id?, year? }`	Commodity type
`admin.commodity.commodity_type.delete`	session	`{ id }`	-
`admin.commodity.province_commodity.get`	session	`{ search, page, pageSize }`	Paginated province commodities
`admin.commodity.province_commodity.create`	session	Data province commodity	Province commodity
`admin.commodity.province_commodity.update`	session	Data province commodity	Province commodity
`admin.commodity.province_commodity.delete`	session	`{ id }`	-
`admin.commodity.regency_commodity.get`	session	`{ search, page, pageSize }`	Paginated regency commodities

`admin.commodity.regency_commodity.create`	session	Data regency commodity	Regency commodity
`admin.commodity.regency_commodity.update`	session	Data regency commodity	Regency commodity
`admin.commodity.regency_commodity.delete`	session	`{ id }`	-

Product

Path	Auth	Input	Output
`admin.product.product_type.create`	session	`{ name, description, year }`	Product type
`admin.product.product_type.update`	session	`{ id, name?, description?, year? }`	Product type
`admin.product.product_type.delete`	session	`{ id }`	-
`admin.product.product_brand.get`	session	`{ search, page, pageSize, product_type_id? }`	Paginated product brands
`admin.product.product_brand.create`	session	`{ name, industry, description, product_type_id }`	Product brand
`admin.product.product_brand.update`	session	`{ id, name?, industry?, description?, product_type_id? }`	Product brand
`admin.product.product_brand.delete`	session	`{ id }`	-
`admin.product.product_dosage.get`	session	`{ search, page, pageSize }`	Paginated product dosages
`admin.product.product_dosage.create`	session	`{ commodity_type_id, product_brand_id, dosage, unit, year }`	Product dosage
`admin.product.product_dosage.update`	session	Data product dosage	Product dosage
`admin.product.product_dosage.delete`	session	`{ id }`	-

Potential

Path	Auth	Input	Output
`admin.potential.province_potential.create`	session	Data province potential	Province potential
`admin.potential.province_potential.update`	session	Data province potential	Province potential
`admin.potential.province_potential.delete`	session	`{ id }`	-
`admin.potential.regency_potential.get`	session	`{ search, page, pageSize }`	Paginated regency potentials

`admin.potential.regency_potential.create`	session	Data regency potential	Regency potential
`admin.potential.regency_potential.update`	session	Data regency potential	Regency potential
`admin.potential.regency_potential.delete`	session	`{ id }`	-

Sales

Path	Auth	Input	Output
`admin.sale.sales_realization.get`	session	`{ search, page, pageSize }`	Paginated sales realizations
`admin.sale.sales_realization.create`	session	Data sales realization	Sales realization
`admin.sale.sales_realization.update`	session	Data sales realization	Sales realization
`admin.sale.sales_realization.delete`	session	`{ id }`	-
`admin.sale.daily_sales.get`	session	`{ search, page, pageSize }`	Paginated daily sales
`admin.sale.daily_sales.create`	session	Data daily sales	Daily sales
`admin.sale.daily_sales.update`	session	Data daily sales	Daily sales
`admin.sale.daily_sales.delete`	session	`{ id }`	-

Stall

Path	Auth	Input	Output
`admin.stall.getStallProduct`	session	`{ stall_id }`	Brands for a stall
`admin.stall.create`	session	Data stall	Stall
`admin.stall.update`	session	Data stall	Stall
`admin.stall.delete`	session	`{ id }`	-
`admin.stall.stall_product_brand.get`	session	`{ stall_id }`	Stall product brands
`admin.stall.stall_product_brand.get_product_brands`	session	-	All product brands
`admin.stall.stall_product_brand.assign`	session	`{ stall_id, brand_ids }`	Sync assignment

User

Path	Auth	Input	Output
`admin.user.getById`	session	`{ id }`	User detail
`admin.user.create`	session	`{ name, email, password, role }`	User
`admin.user.update`	session	`{ id, name?, email?, role? }`	User

`admin.user.delete`	session	`{ id }`	-
---------------------	---------	----------	---

Lampiran F — Daftar Lengkap Tabel Database

F.1 Domain Autentikasi

Tabel	Kolom	Tipe	Constraint
	name	text	-
	email	text	unique
	emailVerified	boolean	default false
	image	text	nullable
	role	text	default 'guest'
	createdAt	timestamp	-
	updatedAt	timestamp	-
`session`	id	UUID	PK
	userId	UUID	FK → user.id
	token	text	unique
	expiresAt	timestamp	-
	createdAt	timestamp	-
`account`	id	UUID	PK
	userId	UUID	FK → user.id
	accountId	text	-
	providerId	text	-
	password	text	nullable
	createdAt	timestamp	-
`verification`	id	UUID	PK
	value	text	-
	expiresAt	timestamp	-
	createdAt	timestamp	-

F.2 Domain Wilayah

Tabel	Kolom	Tipe	Constraint
	code	text	-
	name	text	-

	area	numeric	nullable
	year	integer	nullable
	createdAt	timestamp	-
`regencies`	id	UUID	PK
	province_id	UUID	FK → provinces.id
	code	text	-
	name	text	-
	area	numeric	nullable
	year	integer	nullable
	createdAt	timestamp	-

F.3 Domain Lahan

Tabel	Kolom	Tipe	Constraint
	name	text	-
	year	integer	nullable
	createdAt	timestamp	-
`province_lands`	id	UUID	PK
	province_id	UUID	FK → provinces.id
	land_type_id	UUID	FK → land_types.id
	area	numeric	-
	year	integer	nullable
	createdAt	timestamp	-
`regency_lands`	id	UUID	PK
	regency_id	UUID	FK → regencies.id
	land_type_id	UUID	FK → land_types.id
	area	numeric	-
	year	integer	nullable
	createdAt	timestamp	-

F.4 Domain Komoditas

Tabel	Kolom	Tipe	Constraint
	land_type_id	UUID	FK → land_types.id

	name	text	-
	year	integer	nullable
	createdAt	timestamp	-
`province_commodities`	id	UUID	PK
	province_id	UUID	FK → provinces.id
	commodity_type_id	UUID	FK → commodity_types.id
	area	numeric	nullable
	year	integer	nullable
	createdAt	timestamp	-
`regency_commodities`	id	UUID	PK
	regency_id	UUID	FK → regencies.id
	commodity_type_id	UUID	FK → commodity_types.id
	area	numeric	nullable
	year	integer	nullable
	createdAt	timestamp	-

F.5 Domain Produk dan Potensi

Tabel	Kolom	Tipe	Constraint
	name	text	-
	description	text	nullable
	year	integer	nullable
	createdAt	timestamp	-
`product_brands`	id	UUID	PK
	product_type_id	UUID	FK → product_types.id
	name	text	-
	industry	text	nullable
	description	text	nullable
	createdAt	timestamp	-
`product_dosages`	id	UUID	PK
	commodity_type_id	UUID	FK → commodity_types.id
	product_brand_id	UUID	FK → product_brands.id
	dosage	numeric	-
	unit	text	-

	year	integer	nullable
	createdAt	timestamp	-
`province_potentials`	id	UUID	PK
	province_id	UUID	FK → provinces.id
	product_brand_id	UUID	FK → product_brands.id
	potential_value	numeric	nullable
	description	text	nullable
	year	integer	nullable
	createdAt	timestamp	-
`regency_potentials`	id	UUID	PK
	regency_id	UUID	FK → regencies.id
	product_brand_id	UUID	FK → product_brands.id
	potential_value	numeric	nullable
	description	text	nullable
	year	integer	nullable
	createdAt	timestamp	-

F.6 Domain Kios

Tabel	Kolom	Tipe	Constraint
	province_id	UUID	FK → provinces.id
	regency_id	UUID	FK → regencies.id
	name	text	-
	address	text	nullable
	latitude	numeric	nullable
	longitude	numeric	nullable
	owner	text	nullable
	noTelp	text	nullable
	criteria	text	nullable
	year	integer	nullable
	createdAt	timestamp	-
`stall_product_brands`	id	UUID	PK
	stall_id	UUID	FK → stalls.id
	product_brand_id	UUID	FK → product_brands.id

	createdAt	timestamp	-
--	-----------	-----------	---

F.7 Domain Penjualan

Tabel	Kolom	Tipe	Constraint
	product_brand_id	UUID	FK → product_brands.id
	report_date	date	-
	realization_daily	numeric	nullable
	realization_monthly	numeric	nullable
	realization_ytd	numeric	nullable
	rkap_monthly	numeric	nullable
	rkap_ytd	numeric	nullable
	realization_previous_year	numeric	nullable
	createdAt	timestamp	-
`daily_sales`	id	UUID	PK
	product_brand_id	UUID	FK → product_brands.id
	province_id	UUID	FK belum didefinisikan
	date	date	-
	month	integer	-
	year	integer	-
	quantity	integer	nullable
	revenue	numeric	nullable
	target	numeric	nullable
	realization	numeric	nullable
	notes	text	nullable
	createdAt	timestamp	-

F.8 Domain Demo

Tabel	Kolom	Tipe	Constraint
	text	text	-
	done	boolean	default false
	createdAt	timestamp	-

LEMBAR BIMBINGAN KERJA PRAKTEK

Semester Gasal / Genap Tahun 2025/2026 Periode : Genap

Pas Photo 4 x 6	Nama	: Henokh Yeremia Olbrain Perangin Angin
	NBI	: 1462300075
	Alamat Rumah / Kost	: Dusun : , Jl.Sk Rd Syahbudin no.14 komp rsud abdul manap, RT/RW : 6/1, Kelurahan : Mayang Mangurai, Kecamatan : 106001
	No Telp. / Hp	: 082164819086
	Pembimbing	: Ahmad Habib (20460150665)
Mulai Bimbingan	Judul KP	: PENGEMBANGAN BACKEND SISTEM INFORMASI MANAJEMEN PRODUK BARU BERBASIS TANSTACK START DAN POSTGRESQL PADA PT PETROKIMIA GRESIK

PERSETUJUAN DOSEN PEMBIMBING	NILAI
Tanggal : 25 Juni 2026 Ttd. Pembimbing  (Ahmad Habib)	

LEMBAR BIMBINGAN KERJA PRAKTEK

NO	HARI / TGL	URAIAN MATERI	TT.DOSEN
1	4 Juni 2026	Bimbingan penentuan judul serta penyusunan BAB I (Pendahuluan) mengenai pengembangan backend Sistem Informasi Manajemen Produk Baru pada PT Petrokimia Gresik.	
2	8 Juni 2026	Bimbingan penyusunan BAB II (Tinjauan Pustaka) meliputi konsep backend, TanStack Start, PostgreSQL, dan teori pendukung lainnya.	
3	20 Juni 2026	Bimbingan penyusunan BAB III (Pelaksanaan Magang) yang membahas profil perusahaan, arsitektur sistem, implementasi backend menggunakan TanStack Start dan PostgreSQL, serta hasil pekerjaan selama magang.	
4	23 Juni 2026	Bimbingan penyusunan BAB IV (Penutup) yang meliputi kesimpulan, saran, dan evaluasi hasil pengembangan sistem.	
5	25 Juni 2026	Revisi akhir laporan magang, pemeriksaan kesesuaian format penulisan, sitasi, lampiran, serta persetujuan untuk mengikuti sidang.	

JUDUL KERJA PRAKTEK SETELAH DIREVISI

PENGEMBANGAN BACKEND SISTEM INFORMASI MANAJEMEN PRODUK BARU UNTUK Mendukung Pengelolaan Data dan Monitoring Produk Berbasis Web pada PT Petrokimia Gresik

LEMBAR PENGESAHAN JUDUL KERJA PRAKTEK

Tanggal : 25 Juni 2026 Ttd. Pembimbing  Ahmad Habib NIP : 20460150665	Ttd. Koordinator Anton Brevia Yunanda S.T., M.MT NPP. 20450020554
--	---

* Cetak dilembar buffalo kuning

CHECKLIST PROPOSAL KERJA PRAKTEK

Semester Gasal / Genap Tahun 2025/2026 Periode : Genap

Nama	: Henokh Yeremia Olbrain Perangin Angin
NBI	: 1462300075
Alamat Rumah / Kost	: Dusun : , Jl.Sk Rd Syahbudin no.14 komp rsud abdul manap, RT/RW : 6/1, Kelurahan : Mayang Mangurai, Kecamatan : 106001
No Telp. / Hp	: 082164819086
Pembimbing	: Ahmad Habib (20460150665)
Judul KP	: PENGEMBANGAN BACKEND SISTEM INFORMASI MANAJEMEN PRODUK BARU BERBASIS TANSTACK START DAN POSTGRESQL PADA PT PETROKIMIA GRESIK

Dosen Pembimbing wajib memberikan check (✓) untuk tiap point yang telah dipenuhi.

Ketentuan umum yang harus dipenuhi

- ☒ Mahasiswa telah lulus mata kuliah minimal 72 sks
- ☒ Mahasiswa mempunyai IPK minimal 2.50
- ☒ Mahasiswa sudah mencantumkan mata kuliah Kerja Praktek dalam KRS
- ☒ Kerja Praktek sudah sesuai dengan bidang ilmu pada program studi Teknik Informatika
- ☒ Mahasiswa sudah melakukan pembayaran untuk mengikuti mata kuliah Kerja Praktek pada periode saat ini

Sistematika Penulisan Proposal

- ☒ Font yang digunakan adalah Times New Roman dengan ukuran 12
- ☒ Jarak baris pada proposal KP adalah 1 spasi
- ☒ Ukuran kertas yang digunakan adalah A4 dengan minimal 10 halaman
- ☒ Ukuran margin yang digunakan sudah sesuai aturan, yaitu right, top, bottom adalah 3 cm, dan left 4 cm
- ☒ Halaman Sampul sampai Daftar Isi diberi nomor halaman dengan huruf: i, ii, iii, dst dan diletakkan pada sudut kanan bawah
- ☒ Halaman Pendahuluan sampai Daftar Pustaka diberi nomor halaman dengan angka arab: 1, 2, 3, ...dst yang diletakkan pada sudut kanan atas
- ☒ Deskripsi kegiatan kerja praktek sudah 1 halaman atau lebih

Surabaya, 4 juni 2026

Mengetahui,
Koordinator KP

Anton Breva Yunanda S.T., M.MT
NIP : 20450020554

Dosen Pembimbing


Ahmad Habib
NIP : 20460150665

Gresik, 20 Januari 2026

Nomor : 00035/C/SM/D3210/PK/2026
Lampiran : -
Perihal : Surat Konfirmasi Penerimaan Kerja Praktek Kelompok

Yth, Bapak/Ibu Yusrida Muflihah, S.Kom., M.Kom
UNIVERSITAS 17 AGUSTUS 1945 SURABAYA

Dengan hormat,

Menindaklanjuti surat nomor 11/K/FTEIC/Akd/1/2026, tanggal 13 Januari 2026 perihal Permohonan Kerja Praktek Kelompok, bersama ini kami sampaikan bahwa perihal pengajuan tersebut dapat Diterima, terhitung mulai tanggal 01 Februari 2026 s.d 31 Maret 2026 untuk Mahasiswa/Siswa :

Nama	Prodi	Area Pabrik	Pembimbing
Daffa Raditya	Informatika	Mitra Bisnis Layanan TI PKG	2166544 - Anugrah Rinaldy, S.ST.
Henokh Yeremia Olbrain Perangin Angin	Informatika	Mitra Bisnis Layanan TI PKG	2166544 - Anugrah Rinaldy, S.ST.

Ketentuan terkait dengan pelaksanaan tersebut adalah sebagai berikut :

1. Pelaksanaan kegiatan kerja praktik dilakukan secara offline.
2. Peserta diwajibkan:
 - Melakukan pembuatan **KIKP (Kartu Identitas Kerja Praktik)** dengan cara melakukan Konfirmasi kesediaan website.
 - Mengisi surat pernyataan terkait aturan K3 & Cyber Security di lingkungan Petrokimia Gresik melalui link <https://petro.kim/SuratPernyataanPesertaProgram>. Bukti pengisian bisa discreenshot/capture dan ditunjukkan saat pengambilan KIKP dan APD
 - Bergabung **WAG Prakerin** yang dikirimkan melalui email pendaftar.
3. Mengikuti Induksi Prakerin secara online via zoom pada:
 - Tanggal : 02 Februari 2026
 - Pukul : 07.30 WIB
 - Link : Untuk seluruh peserta (ketua & anggota) yang sudah diterima & konfirmasi diwebsite. Silahkan bergabung WAG di <https://petro.kim/WAGprakerinfebruari2026> ,untuk informasi pelaksanaan Prakerin di Petrokimia Gresik
4. Peraturan pelaksanaan kerja praktik:
 1. Ketentuan seragam/pakaian pada saat pelaksanaan kerja praktik adalah sebagai berikut
 - SMK : Mengikuti aturan seragam sekolah
 - Universitas : Bebas, sopan dan rapi
 2. Mentaati dan mematuhi arahan serta petunjuk yang diberikan oleh pembimbing selama pelaksanaan kerja praktek/penelitian

Demikian hal ini disampaikan, atas perhatian serta kerja sama-nya kami ucapkan terima kasih.

Hormat Kami,
VP Departemen Manajemen & Pengembangan SDM



EDWYN CHARISMA PUTRA, S.Psi, M.M.,
2115322

Tembusan:

1. Pembimbing Kerja Praktik
2. Arsip